

15 · Supervised Learning (KI und ML)

Modul: KI und ML — Supervised Learning **Dozent:** Martin Zaefferer (zaefferer@dhbw-ravensburg.de , Raum 206, Marienplatz 2) **Studiengang:** DHBW Ravensburg · WDS224 · Data Science **Lehrbuch:** James, G.; Witten, D.; Hastie, T.; Tibshirani, R. (2021): *An Introduction to Statistical Learning – with Applications in R*. 2nd Edition, Springer. → **ISL**, frei verfügbar unter www.statlearning.com **Prüfung:** Klausur · **Erlaubte Hilfsmittel:** nur nicht-programmierbarer Taschenrechner · Formelsammlung wird gestellt!

Quellenlage dieser Analyse

Datei	Seite n	Wörte r	Inhalt
Folien/00_Intro.pdf	39	1.276	Organisatorisches, Motivation, CRISP-DM, DASC-PM, Lab R
Folien/10_Ch2_statistical_learning.pdf	50	1.796	ISL Kap. 2 — Grundlagen, Bias-Variance, kNN
Folien/20_Ch3_linear_regression.pdf	63	3.815	ISL Kap. 3 — Lineare Regression
Folien/30_Ch4_classification.pdf	75	4.148	ISL Kap. 4 — Klassifikation, Log. Reg., LDA/QDA
Folien/40_Ch9_support_vector_machines.pdf	48	2.189	ISL Kap. 9 — Hyperebenen, Margins, Kernel, SVM
Folien/50_Ch5_resampling.pdf	59	2.686	ISL Kap. 5 — Validation, CV, Bootstrap
Folien/60_Ch8_trees.pdf	59	3.037	ISL Kap. 8 — Bäume, Bagging, RF, Boosting
Folien/80_Ch10_NN.pdf	69	2.453	ISL Kap. 10.1/10.2/10.7 — Neuronale Netze
Übung/UebungML.html	—	~9.000	62 Übungsaufgaben mit Musterlösungen (Kap. 2,3,4,9,5,8,10)
Übung/Uebungsklausur.pdf	4	1.071	Übungsklausur, 50 Punkte, 10 Aufgaben
Übung/Uebungsklausur_LSG.pdf	6	1.825	Musterlösung der Übungsklausur
Labs/...LabScripts.zip	—	—	7 R-Skripte + 1 Python-Skript (identisch zum Folien-Code)

Themenliste laut Dozent (Folie „Themen“):

1. Motivation, Terminologie und Grundlagen (hauptsächlich **ISL Kap. 2**)
2. Lineare Regression (**Kap. 3**)
3. Klassifikation (**Kap. 4**)
4. Support Vector Machines (**Kap. 9**)
5. Re-sampling / Validation (**Kap. 5**)
6. Entscheidungsbäume, Ensembles (**Kap. 8**)
7. Einführung in Neuronale Netze (**Kap. 10**)

Lernziele laut Dozent:

- Allgemeines Verständnis von ML und verwandten Gebieten
- Probleme/Aufgabenstellungen des maschinellen Lernens
- Grundlegende Methoden und Algorithmen
- Eigenschaften von Algorithmen und Datensätzen
- Beziehungen zwischen Problem, Daten und Algorithmen/Modellen
- Anwendung von Algorithmen
- Interpretation von Modellen / Ergebnissen, Evaluierung

⚠ Wichtig zur Prüfungsrelevanz: Die Programmbeispiele in R sind laut Dozent ausdrücklich „**nur indirekt prüfungsrelevant**“. Sie dienen der Demonstration. Die Klausur prüft **Konzepte, Interpretation und Rechnen von Hand**.

TEIL 0 — DAS KLAUSURFORMAT (zuerst lesen!)

0.1 Struktur der Übungsklausur — 10 Aufgaben, 50 Punkte

Nr.	Punkte	Thema	Aufgabentyp
1	6	Grundlagen (Kap. 2)	Definieren + Beispiele nennen (Klassifikation vs. Regression; kategorische Features)
2	6	Bias-Variance (Kap. 2)	Erklären + Schlussfolgern (Underfit → Komplexität, Bias/Varianz, Lösung)
3	6	Lineare Regression (Kap. 3)	RECHNEN (Interaktionsterm, Ableiten, Gleichsetzen)
4	6	Logistische Regression (Kap. 4)	RECHNEN ($p(X)$ berechnen, Intercept interpretieren, Grenzwert)
5	5	SVM (Kap. 9)	Fließtext erklären (SVM ↔ Hyperebenen ↔ Kernel-Trick)
6	6	Resampling (Kap. 5)	Begründen (Warum Validierungsdaten?)
7	4	Bäume (Kap. 8)	Baum ablaufen (Vorhersage aus gegebenem Baum)
8	2	Boosting (Kap. 8)	Kurzenscheid + Begründung (λ erhöhen/verringern)
9	4	Random Forest (Kap. 8)	Zweck erklären (zufällige Feature-Auswahl)
10	5	Neuronale Netze (Kap. 10)	RECHNEN (Forward-Pass + Gleichung lösen)

🔑 Muster erkennen: Jedes der 7 Kapitel kommt vor. Etwa **die Hälfte der Punkte ist Rechnen**, die andere Hälfte **Konzept-Erklärung mit Begründung**. Bei fast jeder Aufgabe steht „Begründen Sie“ — reine Antworten ohne Begründung geben Punktabzug.

0.2 ★ DIE FORMELSAMMLUNG — wird in der Klausur gestellt

Zitat aus der Übungsaufgaben-Datei: „Die folgenden Formeln werden Ihnen als Teil der Aufgabenstellung in der Klausur zu Verfügung gestellt.“

#	Formel	Verwendung
1	$\ \vec{x} - \vec{x}'\ = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2}$	Euklidische Distanz (kNN)
2	$[\beta - 2 SE(\beta), \beta + 2 SE(\beta)]$	95 % Konfidenzintervall für β
3	$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Residual Sum of Squares
4	$TSS = \sum_{i=1}^n (y_i - \bar{y})^2$	Total Sum of Squares
5	$R^2 = (TSS - RSS)/TSS$	Bestimmtheitsmaß
6	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Mean Squared Error
7	$\vec{c} = (X^T X)^{-1} X^T \vec{y}$ bzw. $(X^T X) \vec{c} = X^T \vec{y}$	Least-Squares-Koeffizienten
8	$n_0 + \vec{n}^T \vec{x} = 0$	Hyperebenengleichung
9	$p(x) = \frac{s}{1+s}$ mit $s = e^a$, $a = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n$	Logistische Regression
10	$\ln\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n$	Log-Odds / Logit
11	$d = \frac{ \vec{p}^T \vec{n} + n_0 }{ \vec{n} }$	Abstand Punkt ↔ Ebene
12	$K(\vec{x}, \vec{x}') = e^{-\gamma \sum_{j=1}^p (x_j - x'_j)^2}$	Radial-Kernel (RBF)
13	$f(\vec{x}) = n_0 + \sum_{i=1}^n \alpha_i K(\vec{x}, \vec{x}_i)$	SVM-Vorhersage
14	$G = 1 - \sum_{k=1}^K \hat{p}_{mk}^2$	Gini-Index
15	$\text{ReLU } h(x) = \begin{cases} x & x > 0 \\ 0 & \text{sonst} \end{cases}$ · Sigmoid $h(x) = 1/(1 + e^{-x})$ · Tanh $h(x) = \tanh(x)$	Aktivierungsfunktionen

⚠ **Was NICHT in der Formelsammlung steht** (und Sie also können müssen):

- Die Formeln für **SE(β_0)**, **SE(β_1)** (werden aber auch nicht verlangt — nur die Interpretation)
- **RSE** = $\sqrt{\frac{1}{n-2} RSS}$
- Der **Boosting-Algorithmus** (auswendig, siehe Teil F)
- Die **Case-Control-Korrektur** $\hat{\beta}_0^* = \hat{\beta}_0 + \ln \frac{a}{1-a} - \ln \frac{\tilde{a}}{1-\tilde{a}}$
- Formel 3.4 (einfache LinReg): $\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$, $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$ (wurde in Übung 3.15 aber mitgeliefert)

0.3 Die 6 wiederkehrenden Aufgabentypen (Muster über Klausur + Übungen)

1. „**Erläutern Sie den Unterschied zwischen X und Y**“ → immer beide Seiten definieren + je ein Beispiel
2. „**Steigt/sinkt X? Begründen Sie.**“ → Ableiten oder Vorzeichen betrachten, dann in Worten interpretieren
3. „**Berechnen Sie p(X) / f(x) / RSS / R²**“ → Werte einsetzen, Taschenrechner, **3 Nachkommastellen** wenn verlangt
4. „**Zeichnen Sie ...**“ → Baum, Featureraum-Aufteilung, Entscheidungsgrenze, Netzwerk-Skizze
5. „**Welches Modell/Maß würden Sie wählen? Warum?**“ → immer über Bias-Variance-Tradeoff argumentieren
6. **Fangfragen** → „100 % Wahrscheinlichkeit?“ → **nur als Grenzwert, praktisch unmöglich**; „Baum zu dieser Aufteilung?“ → **nicht möglich, wenn Regionen nicht achsenparallel-rekursiv entstehen können**

TEIL A — GRUNDLAGEN DES MACHINE LEARNING (ISL Kap. 2)

A.1 Motivation — wozu Machine Learning?

Der Dozent nennt sieben Anwendungsfelder als Einstieg:

Anwendung	Beschreibung
Translation	Übersetzung zwischen Sprachen (Text und/oder Audio)
OCR (Optical Character Recognition)	Erkennung von Text in Bildern (z. B. vor der Übersetzung)
Language ↔ Code	Copilot, ChatGPT; auch: strukturierte Dokumente (HTML, Markdown, LaTeX), Musik (Noten, MIDI), Vektorgrafiken (SVG)
Predictive Maintenance	Geräuschfrequenzen rotierender Elemente, Verfärbung von Materialien, Mikrorisse/Ablösungen auf Bildern, Trenderkennung in Sensordaten
Spam-Erkennung	„Spam“ (unerwünscht) vs. „Ham“ (erwünscht), anhand von Absendername/-adresse, Titel, CC-Einträgen, Hyperlinks, Text, Metadaten
Sicherheit	Anomalien in Systemen, Einbrüche in digitale Netze, Manipulation physischer Infrastruktur
Recommender Systems	Empfehlung von Produkten/Artikeln/Musik auf Basis persönlicher Daten und Kaufverhalten

A.2 ★ Statistical Learning = Machine Learning

🔑 **Kernaussage des Dozenten:** „Statistical Learning ist ein Synonym für Maschinelles Lernen (ML). Statistik ist eine wichtige Grundlage für ML.“

„Lernen“ bedeutet hier in der Regel:

- **Lernen aus Daten**
- oder: **Ermitteln von Beziehungen zwischen Objekten anhand von Datenbeispielen**

Statistik vs. Machine Learning

	Statistik	Machine Learning
Grundschema	Daten $X \rightarrow$ Ausgabe Y	Daten $X \rightarrow$ Ausgabe Y
Beispiel	Lineare Regression $\hat{f}(x) = c_0 + c_1x_1 + c_2x_2 + \dots$	Entscheidungsbäume, SVM, Ensembles, NN
Komplexität	einfacher	komplexer
Beispiel-Beziehungen	—	Maschinendaten $\rightarrow$ Ausfälle; Sprache $\rightarrow$ Sentiment; Bilder $\rightarrow$ erkennbare Objekte

🔑 **Merksatz:** „Kein grundsätzlicher Unterschied zu ‚statistischen‘ Methoden“ — nur komplexere Modelle, die komplexere Beziehungen lernen können.

A.3 ★★ Supervised vs. Unsupervised Learning

Supervised Learning

Begriff	Symbol	Synonyme (alle klausurrelevant!)
Label	Y	Abhängige Variable, Output, Antwort, Zielvariable
Features	X (Anzahl: p)	Prädiktoren, Inputs, unabhängige Variablen, Kovariate, Merkmale
Beobachtungen	$(x_1, y_1), \dots, (x_N, y_N)$	Datenproben, Datenpunkte, Beobachtungen, Stichproben

Zwei Problemtypen:

	Regression	Klassifikation
Y ist ...	quantitativ / numerisch	qualitativ / kategorisch
Wertebereich	reelle Zahlen	endliche Anzahl von Werten aus einer Menge C
Beispiele	Preis, Blutdruck, Niederschlagsmenge	überlebt/verstorben, spam/ham, Ausfall/kein Ausfall
kNN-Aggregation	Mittelwert der Nachbarn	Modus (häufigste Klasse) der Nachbarn

Zielsetzung im Supervised Learning (3 Punkte, wörtlich):

1. Genaue **Vorhersage** von Y für neue Testdaten X^*
2. **Verstehen**, wie Y mit X zusammenhängt
3. **Bestimmen der Unsicherheit** von Vorhersagen

Unsupervised Learning

- **Kein Label Y** , nur Features X — oder: Features X sind auch selbst Label
- Zielsetzung ist **weniger spezifisch**:
 - Erkennen von **Gruppen** ähnlicher Beobachtungen (Clustering)
 - Erkennen von **Korrelationen** zwischen Features, Muster
 - **Komprimierung** von Daten / Dimensionsreduktion, Visualisierung
 - Lernen des „**normalen Verhaltens**“ von Systemen, die Daten erzeugen
 - **Vorverarbeitung** von Daten vor Supervised Learning
- **⚠ Oft schwieriger zu bewerten** (kein Vergleich der Genauigkeit über Abgleich mit Y möglich)

A.4 ML in der Praxis: Sprachen, Werkzeuge, Prozesse

Bereich	Werkzeug
„klassisches“ ML / Statistik	R
Deep Learning / Neuronale Netze	Python
„Code-less“	knime, orange, RapidMiner
Manchmal auch	Excel 📊

Zu berücksichtigende Prozesse rund um ML:

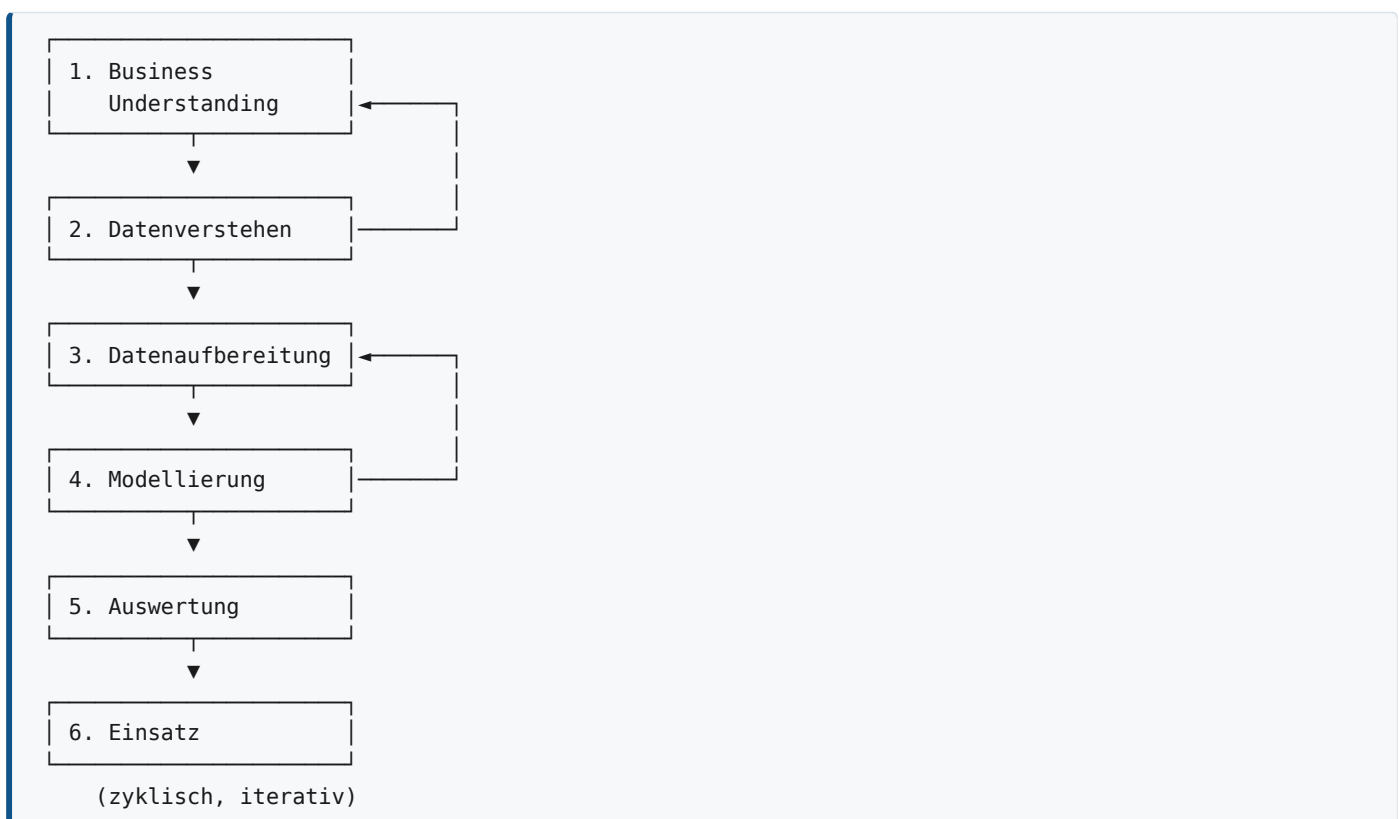
- **Pre-processing:** Datenerfassung (Sensoren, Webscraping), Datenspeicherung (Datenbanken), Datenübertragung zum ML-Prozess (DB-Zugriff), Datenaufbereitung (fehlende Werte ersetzen, Umcodierung)
- **Post-processing:** Übertragung der Ergebnisse in andere Systeme, Speicherung der Ergebnisse, Interpretation und Visualisierung

🔑 „Algorithmen allein sind nicht genug.“

A.5 ★ Prozessmodelle: CRISP-DM und DASC-PM

CRISP-DM = **C**ross **I**ndustry **S**tandard **P**rocess for **D**ata **M**ining — Standardprozessmodell für Datenanalyseprojekte.

Die 6 Phasen von CRISP-DM



Phase	Inhalt (wörtlich vom Dozenten)
1. Verständnis des Business-Case	Welches Problem zu lösen? Mit welchen Zielen? Welche Grenzen beachten?
2. Datenverstehen	Sammeln, Erkunden, Beschreiben verfügbarer Daten, Qualität bestimmen
3. Datenaufbereitung	Umgang mit Ausreißern, fehlenden Daten, Rauschen; Integration verschiedener Quellen
4. Modellierung	Kern des Prozesses. Auswählen, Anwenden, Verbessern von ML-Modellen
5. Auswertung	Ergebnisinterpretation, wie gut wird Problem gelöst, Anwendbarkeit prüfen
6. Einsatz	Implementierung in technologische/geschäftliche Prozesse, Überwachung, Wartung

DASC-PM = **D**ata **S**cience **P**rocess **M**odel — ein neueres Prozessmodell, das Ähnlichkeiten mit CRISP-DM aufweist (Schulz et al. 2021, v1.1).

💡 Der Dozent merkt an: Prozessmodelle sind **hilfreich für die Bachelorarbeit** — sie liefern einen methodischen Rahmen. Die Vorlesung konzentriert sich aber auf **Datenverständnis, Algorithmen und Auswertung**.

A.6 ★★ Die ideale Regressionsfunktion $f(X)$

Grundmodell

$$Y = f(X) + \epsilon$$

- $f(X)$ — die (unbekannte) wahre Beziehung zwischen X und Y
- ϵ — **Fehler**, z. B. aufgrund von Messrauschen in den Daten

Wozu kann $f(X)$ verwendet werden?

1. **Vorhersage** von Y für neue Datenpunkte X
2. **Welche Features haben Einfluss** auf Y , welche nicht?
3. Abhängig vom Modell: **Erläuterung des Einflusses**

★ Die ideale Regressionsfunktion

Beispiel des Dozenten: Bei $X = 4$ liefern die Daten verschiedene Y -Werte. Guter Wert für $f(X)$: **Durchschnitt der Beobachtungen Y bei genau $X = 4$:**

$$f(4) = E(Y \mid X = 4)$$

$E(Y \mid X = 4)$ = **Erwartungswert (Mittelwert) von Y bei $X = 4$.**

🔑 **Eigenschaft:** Die ideale Regressionsfunktion liefert den **minimalen quadratischen Fehler** — sie macht bei Vorhersage minimale Fehler.

⚠️ Warum kann man f nicht direkt bestimmen?

In der Regel gibt es **keine oder sehr wenige Datenpunkte mit exakt $X = 4$** . Daraus folgt: die Regressionsfunktion kann in der Regel **nicht** bestimmt werden → **Schätzung $\hat{f}(X)$ notwendig**. Die Schätzung bringt **zusätzliche Fehler** mit sich.

A.7 ★★★ Die Fehlerzerlegung

Nicht-reduzierbarer Fehler ϵ

$$\epsilon = Y - f(x)$$

- Fehler der **idealen** Regressionsfunktion
- **Auch bei idealem $f(x)$ ist der Fehler nicht Null!**
- **Grund:** Y entspricht einer **Verteilung** möglicher Werte für einen bestimmten Wert von X (nicht nur einem Wert Y)
- 📝 **Beispiel des Dozenten:** Zwei Personen mit gleichem Alter, gleicher Ausbildung, gleichem Beruf (Features X) können **unterschiedliche Gehälter** haben (Label Y).

Vorhersagefehler des Schätzwertes

$$\text{Fehler von } \hat{f}(X) = \underbrace{\hat{f}(X) - f(x)}_{\text{reduzierbar}} + \underbrace{\epsilon}_{\text{nicht reduzierbar}}$$

☆☆☆ Bias-Variance-Zerlegung (KLAUSURKERN!)

$$\text{Error}(x^*, y^*) = \underbrace{\text{Variance of } \hat{f}(x^*)}_{\text{Varianz}} + \underbrace{\text{Bias of } \hat{f}(x^*)}_{\text{Bias}} + \underbrace{\epsilon}_{\text{nicht-reduzierbar}}$$

Komponente	Ursache (wörtlich)
Variance	Fehler der Modellvorhersagen aufgrund kleiner Änderungen in den Trainingsdaten
Bias	Fehler der Modellvorhersagen aufgrund der Modellstruktur
ϵ	Fehler in den Daten selbst , kann nicht durch das Modell „behoben“ werden

★ Der Trade-off — auswendig!

Wenn die Flexibilität bzw. Komplexität von $\hat{f}$ steigt $\uparrow$:

- Variance erhöht $\uparrow$
- Bias vermindert $\downarrow$
- Nicht-reduzierbarer Fehler konstant $\circ$

🔑 **Schlussfolgerung des Dozenten:** „Die richtige Wahl der Modellkomplexität auf Grundlage des durchschnittlichen Testfehlers impliziert die Auflösung des Bias-Variance Trade-off.“

Die klassische 2×2-Zielscheiben-Darstellung

	LOW VARIANCE	HIGH VARIANCE
HIGH BIAS	 (dicht, aber daneben)	 (gestreut und daneben)
LOW BIAS	 IDEAL: dicht und im Zentrum	 gestreut, aber im Mittel richtig

A.8 ☆☆☆ Under-, Good- und Overfit

Zustand	Definition (wörtlich)	Bias	Varianz	Trainingsfehler	Testfehler
Underfit	„Das Modell ist zu einfach und funktioniert schlecht“ — lernt nicht genug, ist selbst auf Trainingsdaten schlecht angepasst und auf Testdaten ähnlich schlecht	hoch	gering	hoch	hoch
Good Fit	„Komplexität ist ausgewogen“ — erfasst die Zusammenhänge in den Daten gut, generalisiert gut	mittel	mittel	mittel	minimal
Overfit	„Das Modell ist zu komplex und funktioniert schlecht“ — passt sich zu nah an die Trainingsdaten an, lernt Rauschen , generalisiert schlecht	gering	hoch	sehr gering	hoch

⚠️ Wie erkennt man Overfit? — Die häufigste Falle

❌ FALSCH: „Der Trainingsfehler ist geringer als der Testfehler, daraus folgt Overfit.“ Das ist **nur ein schwacher Indikator, kein Beweis** für Overfitting! Kann andere Gründe haben:

- zufällige Verschiebungen / Ausreißer
- unterschiedliche Skalen aufgrund unterschiedlicher Stichprobengrößen

🟢 BESSER: Trainingsfehler nimmt ab, während der Testfehler zunimmt (bei zunehmender Modellkomplexität/Flexibilität) — siehe Bias-Variance-Tradeoff. ⚠️ Aber auch das ist **nur ein Hinweis, kein Beweis**.

🔑 ZUSATZBEDINGUNG: Test- und Trainingsdaten müssen **einigermaßen unkorreliert** sein! Redundanz / hohe Korrelation kann zu **ununterscheidbaren** Test-/Trainingsdatensätzen führen.

📝 Beispiel: Redundanz (Folie 35)

Gesamtdatensatz				50 % Training			50 % Test		
Nr	x1	x2	y	x1	x2	y	x1	x2	y
1	-1	10	1	-1	10	1	-1	10	1
3	-1	10	1	-1	10	1	-1	10	1
5	-1	10	1	-1	10	1	0	5	2
7	-1	10	1	0	5	2	0	5	2
9	-1	10	1	0	5	2	0	5	2
2,4,6,8,10	0	5	2						

⚠️ Der Test- und der Trainingssatz enthalten **exakt dieselben Feature-Kombinationen** → der „Testfehler“ ist wertlos.

A.9 ★ Trade-off: Vorhersagegenauigkeit vs. Interpretierbarkeit

Modell	Genauigkeit	Interpretierbarkeit
Lineare Modelle	niedriger	leicht zu interpretieren
Entscheidungsbäume (einzeln)	niedrig	am leichtesten interpretierbar
Support Vector Machines	hoch	schwer zu interpretieren
Ensembles (RF, Boosting)	sehr hoch	schwer
Neuronale Netze	sehr hoch	sehr schwer

🔑 „Manchmal: Einfacheres Modell mit weniger Variablen bevorzugen (erklärbar, interpretierbar).“

A.10 ★★ Neighborhood Averaging / k-Nearest Neighbors

Motivation

Man kann den Durchschnitt bei $X = 4$ nicht berechnen, wenn es dort keine Daten gibt. **Abhilfe:** nicht mehr den exakten Wert X betrachten, sondern **in der Nachbarschaft suchen**.

Annahme: Ähnliche X haben vermutlich ähnliche Y .

Problem mit Intervallen

Selbst mit einem Intervall/Volumen als Nachbarschaft: **möglicherweise sind KEINE Daten enthalten.**

Lösung: Berechnung der **Abstände** zwischen den Beobachtungen und Definition der Nachbarschaft als **die k Beobachtungen, die sich am nächsten sind** → **k-Nearest-Neighbors (kNN)**.

$$\|\vec{x} - \vec{x}'\| = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2}$$

★ Wann funktioniert kNN gut?

Bedingung	Größenordnung
Wenige Features (p klein)	einstellig
Viele Beobachtungen (N groß)	Hunderte

⚠️ Curse of Dimensionality (Fluch der Dimensionalität)

Intuitiv: Selbst der nächste Nachbar ist im hochdimensionalen Raum tendenziell weit entfernt.

Argumentation des Dozenten:

- Wir brauchen einen angemessenen Anteil von N Beobachtungen y_i zur Berechnung des Durchschnitts, z. B. **10 % von N**
- Eine 10 %-Nachbarschaft in hohen Dimensionen ist **wahrscheinlich nicht mehr lokal!**

📝 Rechenbeispiel aus der Übung (Kap. 4, Aufg. 3+4):

- $p = 1$, X gleichverteilt auf $[0, 1]$, Nachbarschaft $X \pm 0.05 \rightarrow$ **10 %** der Beobachtungen
- $p = 2$, $[0, 1] \times [0, 1]$, Nachbarschaft $X_1 \pm 0.05$ **und** $X_2 \pm 0.05 \rightarrow$ **1 %** (= 10 % von 10 %)
- Allgemein: $0.1^p \rightarrow$ bei $p = 10$ nur noch 10^{-10} = praktisch keine Daten!

kNN für Klassifikation

- Vorhersage:** häufigste Klasse in der Nachbarschaft (Modus)
- Wird mit zunehmender Dimension ebenfalls schlechter
- 🔑 Aber: **Die Auswirkungen auf die geschätzte Klasse $\hat{c}(x)$ sind geringer als auf die geschätzte Wahrscheinlichkeit $\hat{p}_k(x)$**
- Kleines k** → sehr nichtlineare, „wackelige“ Entscheidungsgrenze (flexibel, hohe Varianz)
- Großes k** → glatte Entscheidungsgrenze (unflexibel, hoher Bias)

A.11 ★ Klassifikation: Bayes-optimaler Klassifikator

- $p_k(x)$: Häufigkeit jeder Klasse k bei x
- Bayes-optimaler Klassifikator $c(x)$:** die Klasse mit dem **größten** $p_k(x)$
- Vergleichbar mit der idealen Regressionsfunktion $f(x)$
- 🔑 **Der ideale Bayes-Klassifikator hat die minimale Fehlklassifizierungsrate**

Fehlklassifizierungsrate:

$$\text{Error}_{X_{\text{test}}} = \frac{\text{Anzahl der falschen Vorhersagen}}{\text{Anzahl der Vorhersagen}}$$

Ziele der Klassifikation (wörtlich):

1. Erstellen eines Klassifikators $Y = c(X)$, der jedem neuen X eine Klasse zuordnet
2. Einfluss der Features bei dieser Zuordnung verstehen
3. Bewertung der Unsicherheit des Klassifikators $Y = c(X)$

A.12 Modellgenauigkeit: MSE

$$MSE = \frac{1}{n} \sum_{i=1}^n \left[y_i - \hat{f}(x_i) \right]^2$$

- **Auf Trainingsdaten berechnet:** kann Overfit möglicherweise **nicht erkennen** — das Modell scheint gut zu sein, ist aber bei neuen Daten schlecht
- **Daher:** Berechnung des MSE mit **neuen Testdaten**, die im Trainingsprozess nicht berücksichtigt wurden
- ⚠ Die Testdaten müssen **vor dem Training** abgetrennt werden (oder nach dem Training erhoben werden)
- 🔑 **Idealerweise:** sehr geringe Überschneidung zwischen Test- und Trainingsdaten, mit **guter Repräsentation der vollständigen Daten in beiden Sätzen**

▼ ? Selbsttest Teil A (aufklappen)

1. Nennen Sie 5 Synonyme für „Label“ und 5 für „Feature“.
2. Warum ist der nicht-reduzierbare Fehler nicht Null, selbst wenn f ideal ist? Geben Sie das Gehaltsbeispiel wieder.
3. Wie lautet die Bias-Variance-Zerlegung? Was passiert mit jedem der drei Terme, wenn die Modellkomplexität steigt?
4. Warum ist „Trainingsfehler < Testfehler“ **kein** Beweis für Overfitting?
5. Ein Datensatz hat $p = 3$ Features, gleichverteilt in $[0, 1]^3$. Welchen Anteil der Daten enthält eine Nachbarschaft $X_j \pm 0.05$? (*Antwort: $0.1^3 = 0.1 \%$*)
6. Was ist der Bayes-optimale Klassifikator und welche Eigenschaft hat er?
7. Nennen Sie die 6 Phasen von CRISP-DM in korrekter Reihenfolge.
8. Was ist die ideale Regressionsfunktion, formal ausgedrückt?
9. Warum ist Unsupervised Learning schwerer zu bewerten als Supervised Learning?
10. Welche zwei Bedingungen müssen erfüllt sein, damit Neighborhood Averaging gut funktioniert?

TEIL B — LINEARE REGRESSION (ISL Kap. 3)

B.1 Grundidee und Zitat

- **Einfacher Ansatz** für Supervised Learning
- **Annahme:** Y ist Linearkombination von $X_1, X_2, \dots$
- **Realität:** Zusammenhänge sind **nie** linear
-  **Trotzdem: lineare Regression kann sehr nützlich sein**

 **Bekanntes Zitat (George Box):** „Essentially, all models are wrong, but some are useful.“

B.2 Der Beispieldatensatz: Advertising

- Daten aus mehreren Marketingkampagnen
- Budget für Marketing in **3 verschiedenen Medien** (TV, Radio, Newspaper)
- Erfasste Verkäufe (**sales**) für jede Kampagne
- **Fragestellung:** Was ist der Zusammenhang von sales und den Marketingbudgets?

Erste Zeilen:

	TV	radio	newspaper	sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
5	180.8	10.8	58.4	12.9
6	8.7	48.9	75.0	7.2

Typische Fragen an die Daten (Folie 6):

1. Beziehung zwischen Werbebudget und Umsatz?
2. Wie stark ist diese Beziehung?
3. Ist die Beziehung linear?
4. Welche Medien tragen zum Umsatz bei?
5. Wie genau können wir zukünftige Verkäufe vorhersagen?
6. Gibt es **Synergien** zwischen den verschiedenen Werbemedien?

B.3 ★ Einfaches lineares Modell (1 Feature)

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Symbol	Bedeutung
β_0	Y-Achsenabschnitt (Intercept)
β_1	Steigung
β_0, β_1	zusammen: Koeffizienten oder Parameter — unbekannte konstante Werte
ϵ	Modellfehler

Vorhersage mit geschätzten Koeffizienten:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

B.4 ★★ Least Squares (Methode der kleinsten Quadrate)

Größe	Formel
Vorhersage für den i -ten Trainingspunkt	$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$
i -tes Residuum	$e_i = y_i - \hat{y}_i$
Residual Sum of Squares	$RSS = e_1^2 + e_2^2 + \dots + e_n^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Least Squares (LS): Finde die Koeffizienten β , die den **kleinsten** RSS-Wert ergeben.

🔑 **Wichtige Eigenschaft:** „RSS kann **analytisch** optimiert werden!“ — d. h. es müssen nur einige Gleichungen gelöst werden, **kein rechenintensiver Optimierungsprozess**.

⚠️ **Beachten:** RSS wird auf **Trainingsdaten** und **nicht** auf Testdaten optimiert.

Alternative: Maximum-Likelihood-Estimation (MLE)

- Finde die Koeffizienten β , die die **größte Wahrscheinlichkeit** ergeben, die Trainingsdaten zu beobachten
- 🔑 Für „klassische“ lineare Regression liefern MLE und LS das gleiche Ergebnis

★ Die Normalengleichung (in der Formelsammlung!)

$$\vec{\beta} = (X^T X)^{-1} X^T \vec{y}$$

Anmerkung 1: Die Inverse wird in der Praxis **meist nicht berechnet**. Stattdessen: Lösung des **LGS**

$$(X^T X) \vec{\beta} = X^T \vec{y}$$

Anmerkung 2: Die **Designmatrix** X enthält meist auch eine **Spalte nur aus Einsen** (für den Y-Achsenabschnitt β_0).

📝 Herleitungsaufgabe (Übung 3.17) — Modell ohne Intercept

Für $f(x) = \beta x$ (ein Feature, **kein** Y-Achsenabschnitt):

- X ist ein **Spaltenvektor** aus allen Trainingswerten von x
- $X^T X = \sum_{i=1}^n x_i^2$ (**Skalar!**)
- $X^T \vec{y} = \sum_{i=1}^n x_i y_i$ (**Skalar!**)
- Da beides Skalare sind, ist **keine Matrixrechnung nötig** — die Inverse eines Skalars ist der Kehrwert:

$$\beta = \frac{\sum_{i=1}^n x_i y_i}{\sum_{i=1}^n x_i^2}$$

Weiterführung: Die Vorhersage am j -ten Punkt ist eine **gewichtete Summe aller Beobachtungen**:

$$\hat{y}_j = \beta x_j = \frac{\sum_i x_i y_i}{\sum_i x_i^2} x_j = \sum_{i=1}^n \underbrace{\frac{x_j x_i}{\sum_k x_k^2}}_{a_i} y_i$$

🔑 Interpretation (klausurrelevant!):

- Liegt x_j nahe Null, gehen **alle Gewichte gegen Null** → die beobachteten Daten haben kaum Einfluss auf die Vorhersage
- Beobachtungen mit **großem** x_i haben **deutlich größeren Einfluss** auf $\hat{y}_j$
- **Folgerung:** Trainingsdatenpunkte, die **weit vom Ursprung (bzw. nach Verschiebung: vom Mittelpunkt der Daten)** entfernt liegen, haben einen **größeren Einfluss** auf Vorhersagen (→ Leverage!)

B.5 ★ Genauigkeit der Koeffizientenschätzungen

Standardfehler (SE)

Der **Standardfehler** spiegelt die **Variation bei wiederholten Stichproben** wider:

$$SE(\hat{\beta}_1)^2 = \frac{\sigma^2}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad SE(\hat{\beta}_0)^2 = \sigma^2 \left[\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^n (x_i - \bar{x})^2} \right]$$

→ wird von Software (R, Python) berechnet.

★ Konfidenzintervall (Formelsammlung!)

$$95\text{-KI: } \left[\hat{\beta} - 2 \cdot SE(\hat{\beta}), \hat{\beta} + 2 \cdot SE(\hat{\beta}) \right]$$

Interpretation (wörtlich): Gibt einen Wertebereich an; **95 % Wahrscheinlichkeit, dass der Bereich den wahren Wert enthält** (unter der Annahme wiederholter Stichproben).

📝 **Beispiel:** Für die Advertising-Daten ist das 95 %-KI für β_1 (TV): **[0.042, 0.053]**

📝 **Übung 3.3:** $\beta = 1.13$, $SE(\beta) = 0.47$ → KI = $[1.13 - 0.94, 1.13 + 0.94] = [0.19, 2.07]$

📝 **Übung 3.10:** Welches Intervall ist breiter: 80 % oder 90 % KI? → **Das 90 %-KI ist breiter.** Ein breiteres Intervall hat eine höhere Wahrscheinlichkeit, bei Wiederholung des Zufallsexperiments den beobachteten Wert zu beinhalten.

B.6 ★★★★★ Hypothesentests und p-Werte (SEHR klausurrelevant!)

Die Nullhypothese

$$H_0 : \beta_1 = 0$$

Bedeutung: „**Es gibt keinen Zusammenhang zwischen X und Y** “ bzw. der Koeffizient β ist Null.

★★ Interpretation des p-Werts — die wichtigste Tabelle des Kapitels

p-Wert	Entscheidung	Interpretation
klein (normalerweise $p < 0.05$)	H_0 kann abgelehnt werden	Hinweise für Zusammenhang zwischen Feature und Label liegen vor. Hinweise, dass der Koeffizient β_i nicht Null ist.
groß ($p > 0.05$)	H_0 kann nicht abgelehnt werden	KEIN Hinweis für einen Zusammenhang — aber auch KEIN Hinweis dagegen! Koeffizient β_i kann folglich Null sein.

⚠️ Die drei klassischen Fallen

1. „Der p-Wert ist die Wahrscheinlichkeit, dass H_0 wahr ist“ → **FALSCH!** „Ja, der p-Wert ist tatsächlich eine Wahrscheinlichkeit. Aber: Der p-Wert ist **nicht** die Wahrscheinlichkeit, dass H_0 wahr ist!“
2. **Kausalität** → *, „sagt nichts über Kausalität des Zusammenhangs aus. **Korrelation ist nicht Kausalität**“. Kausalität lässt sich aus dem **Systemverständnis** ableiten, normalerweise nicht aus Statistik oder ML-Modellen.“*
3. **Hinweis ≠ Beweis** → wörtliche Anmerkung des Dozenten: „**Hinweis ≠ Beweis**“

📝 Übung 3.6 (Musterantwort auswendig lernen!):

Aussage: „Das Modell zeigt, dass höhere Temperaturen eine geringere Effizienz **verursachen**.“

Lösung: Das Modell zeigt **nur, dass Hinweise für einen Zusammenhang vorliegen**. Bei höheren Temperaturen ist also geringere Effizienz **zu erwarten**. Zur **Ursache** dieses Zusammenhangs kann **keine Aussage** getroffen werden. Ursachen/Kausalität lassen sich aus der Modellierung nicht schließen.

📝 Übung 3.4: $\beta = 0.2$ mit 95 %-KI $[-1.8, 2.2]$. Kann H_0 abgelehnt werden?

Nein — das Konfidenzintervall **beinhaltet den Wert** $\beta = 0$. Wir können also nicht ausschließen, dass der wahre Wert Null ist.

🔑 **Merkregel:** KI enthält 0 $\iff$ p-Wert > Signifikanzniveau $\iff H_0$ nicht ablehnbar.

📝 **Übung 3.2c:** „Da der Koeffizient für den GPA/IQ-Interaktionsterm sehr klein ist, gibt es nur sehr wenige Belege für eine Wechselwirkung.“ → **Nicht richtig**. Die **Größe des Koeffizienten** gibt **keine Aussage** darüber, wie gut dieser nachgewiesen/belegt ist. Dafür müsste man **Konfidenzintervall oder p-Wert** betrachten.

B.7 ★ Genauigkeit des Gesamtmodells

Maß	Formel	Interpretation
RSS	$\sum_{i=1}^n (y_i - \hat{y}_i)^2$	Residual Sum of Squares
TSS	$\sum_{i=1}^n (y_i - \bar{y})^2$	Total Sum of Squares
RSE	$\sqrt{\frac{1}{n-2} \text{RSS}}$	Residual Standard Error — nicht direkt interpretierbar (benötigt einen Vergleich)
R^2	$1 - \frac{\text{RSS}}{\text{TSS}} = \frac{\text{TSS} - \text{RSS}}{\text{TSS}}$	Anteil der erklärten Varianz — idealerweise nahe 1

! Interpretation von R^2

- Hängt **von den Daten UND vom Modell** ab
- **Kaum Daten erlauben es, $R^2 = 1$ zu erreichen** → nicht-reduzierbarer Fehler!
- 📝 Beispiel Advertising (nur TV): $R^2 = 0.6091$ → „Modell erklärt ca. 60 % der Varianz, wir wissen aber nicht, was den Rest der Varianz verursacht“
- 🗝️ R^2 **kann NEGATIV werden** — nämlich dann, wenn das Modell **schlechter** ist als eines, das einfach nur den **Mittelwert** vorhersagt (siehe Übung 3.16: $R_2^2 = -1.09$)

📝 **Übung 3.5:** Modell A hat $R^2 = 0.87$; Modell B hat $RSS = 1$ bei $TSS = 10$.

$$R_B^2 = \frac{10 - 1}{10} = 0.9 > 0.87 \Rightarrow \text{Modell B ist besser}$$

📝 **Übung 3.16 (vollständig durchgerechnet):**

Daten (9 Punkte): $x = \{(1, 1), (2, 2), (3, 3), (4, 1), (5, 2), (6, 3), (7, 1), (8, 2), (9, 3)\}$, $y = \{-2, -1, 0, 4, 5, 6, 10, 11, 12\}$

Modelle: $\hat{f}_1(x) = 2x_1 - x_2 + 1$ und $\hat{f}_2(x) = 2x_1 + x_2$

Ergebnis	Wert
RSS_1	144
RSS_2	465
TSS	222
R_1^2	0.35
R_2^2	-1.09

→ **Modell 1 ist deutlich besser** — für beide Metriken. R_2^2 wird **negativ**, da Modell 2 schlechter ist als eines, das nur den Mittelwert vorhersagt.

📝 **Übung 3.14 (RSS von Hand):** $\hat{Y} = (1, 4, 2, 2, 1, 1, 1, 2, 4, 5, 2, 3, 0)$, $Y = (2, 4, 3, 2, 2, 2, 1, 4, 3, 4, 1, 2, 1)$

Differenzen: $(-1, 0, -1, 0, -1, -1, 0, -2, 1, 1, 1, 1, -1)$ → Quadrate: $(1, 0, 1, 0, 1, 1, 0, 4, 1, 1, 1, 1, 1)$ → **RSS = 13**

B.8 ★★ Multiple lineare Regression

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon$$

★★★ Die Standard-Interpretationsformel (AUSWENDIG!)

β_j ist die durchschnittliche Auswirkung auf Y eines Einheitsanstiegs in X_j , **WENN ALLE ANDEREN FEATUREWERTE GLEICH BLEIBEN.**

📝 **Advertising-Modell:**

```

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  2.590680   0.406805   6.368 6.57e-09 ***
TV           0.044550   0.001725  25.831 < 2e-16 ***
radio        0.206230   0.010906  18.910 < 2e-16 ***
newspaper    0.002187   0.007080   0.309  0.758
Multiple R-squared:  0.9214, Adjusted R-squared:  0.9189

```

Interpretation (wörtlich vom Dozenten):

- Intercept, TV, radio: **sehr kleine p-Werte** → H_0 ablehnen → Hinweis, dass TV und radio einen Zusammenhang mit sales haben
- newspaper: $p = 0.758$ → **Kein Hinweis für/gegen** eine Beziehung zwischen newspaper & sales
- „Wenn TV um 1 zunimmt, steigt sales um **0.04455** (wenn andere Features gleich bleiben)“
- „Wenn radio um 1 zunimmt, steigt sales um **0.20623** (wenn andere Features gleich bleiben)“

B.9 ★★ Kollinearität (Multikollinearität)

Kollinearität = Korrelation von Features untereinander.

Problem	Beschreibung
Varianz	Die Varianz der Koeffizienten nimmt tendenziell zu, manchmal dramatisch
Numerik	Probleme mit der numerischen Stabilität des Modelltrainings
Interpretation	„wenn sich X_j ändert, ändert sich alles andere!“

⚠ **Der Widerspruch:** Der Koeffizient $\hat{\beta}_j$ schätzt die erwartete Änderung in Y pro Änderung in X_j , **wenn alle anderen Features unverändert bleiben** 🤔 — **aber korrelierte Features ändern sich gleichzeitig!** 🤔

Mögliche Lösung: Principal Component Regression (PCR)

- Allerdings: nur eine **numerische** Lösung, um stabile Ergebnisse zu erhalten
- ⚠ **Die Interpretation bleibt auch mit PCR schwierig**

📝 ★ Das Münzbeispiel (Folie 26) — Klassiker!

Y = Wert der Münzen in einer Tasche X_1 = Anzahl der 10-, 5-, 2- und 1-Cent-Stücke X_2 = Anzahl **aller** Münzen

Frage 1: Welches Vorzeichen hat β_1 im Modell $f(X) = \beta_0 + \beta_1 X_1$? → **Positiv** (mehr Kleinmünzen = mehr Geld)

Frage 2: Welches Vorzeichen hat β_1 im Modell $f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2$? → **Negativ!** Bei **konstanter Gesamtzahl** von Münzen bedeutet mehr Kleingeld weniger große Münzen → weniger Wert.

📝 Übung 3.7 (Variante mit Schrauben):

Y = Preis eines Produktes, X_1 = Anzahl Schrauben, X_2 = Gesamtzahl der Bauteile. Ohne X_2 : mehr Schrauben → teureres Produkt → $\beta_1 > 0$. Mit X_2 : bei **gleicher Gesamtanzahl** von Bauteilen und steigender Zahl von Schrauben werden weniger **teurere** Bauteile benötigt → $\beta_1 < 0$.

🔑 **Prüfungsmerksatz:** Ein Vorzeichenwechsel eines Koeffizienten bei Hinzunahme eines Features ist ein **klassisches Kollinearitäts-/Confounder-Symptom**.

B.10 ★★ Feature Selection

Ansatz 1: p-Wert-Filterung

Alle Features mit hohen p-Werten ausschließen. ⚠ **Nicht immer ideal**, kann zu suboptimalen Ergebnissen führen (z. B. bei Kollinearität).

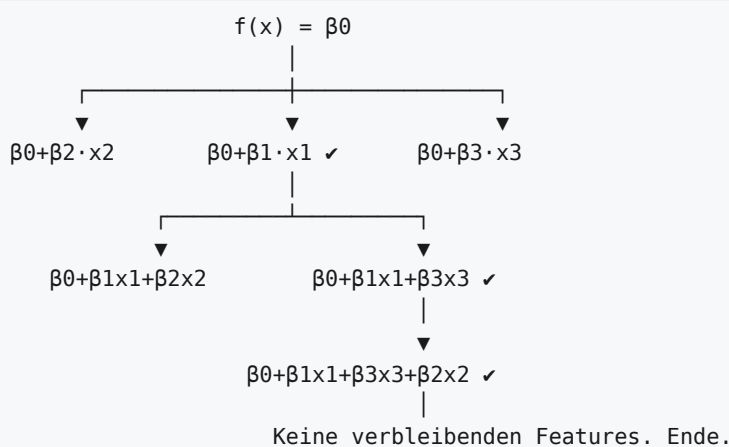
Ansatz 2: Brute Force

- Anpassung **jeweils eines Modells für alle möglichen Featurekombinationen**
- Auswahl des Modells, das ein Kriterium optimiert (z. B. MSE, RSS)
- ⚠ **Anzahl der Modelle:** 2^p (bei rein linearen Effekten)
- 📝 Für $p = 40$: **über eine Billion Modelle!**

★ Ansatz 3: Stepwise Forward Selection (auswendig!)

1. Beginne mit "leerem" Modell – nur Intercept, kein Feature X_i
2. Trainiere alle Modelle, bei denen jeweils nur EIN Feature dem aktuellen Modell hinzugefügt wird
3. Wähle das Modell, das ein Auswahlkriterium optimiert
4. Wenn Abbruchkriterium erfüllt: Ende. Sonst: gehe zu 2.

Ablaufdiagramm (Folie 33):



(✓ = bestes Modell in jedem Schritt)

Alternative: Stepwise Backward Selection — Start mit „vollem“ Modell (alle Features), schrittweise entfernen.

★ Auswahlkriterien

„Idealerweise **‚bestrafen‘** die Kriterien solche Modelle, die zwar genauer, aber **zu komplex** sind.“

- Mallow's C_p
- Akaike Information Criterion (AIC)
- Bayesian Information Criterion (BIC)
- adjusted R^2

Weitere Alternativen

- Formulierung als **Optimierungsproblem** (z. B.: Minimierung von Feature-Feature-Korrelationen, Maximierung von Feature-Label-Korrelationen)
- **Lasso-Regression**

 **Übung 3.8 (Musterantwort):** *Forward Selection ist eine Art der schrittweisen Regression, bei der man mit einem leeren Modell beginnt und eine Variable nach der anderen hinzufügt. In jedem Vorwärtsschritt wird die Variable hinzugefügt, die die größte Verbesserung des Modells bewirkt. [...] Sobald sich das Modell durch Hinzufügen weiterer Variablen nicht mehr (oder nur sehr wenig) verbessert, wird der Prozess gestoppt.*

B.11 ★★ Kategorische Features / Dummy-Codierung

Kategorische Features (auch: qualitative Features, **Faktoren / Faktorvariablen**) stellen eine Auswahl von Kategorien dar. Die spezifischen Kategorien heißen **Ausprägungen**.

 **Beispiele des Dozenten:**

- Feature „Beruf“ ↔ „Einkommen“
- Feature „Materialart“ ↔ „Härte“ — Ausprägungen: Plastik, Stahl, Holz, Keramik, Aluminium
- Feature „Automarke“ ↔ „Preis“

Dummy-Variable bei 2 Ausprägungen

$$x_i = \begin{cases} 1 & \text{if student} \\ 0 & \text{else} \end{cases} \Rightarrow \hat{y}_i = \begin{cases} \hat{\beta}_0 + \hat{\beta}_1 & \text{if student} \\ \hat{\beta}_0 & \text{else} \end{cases}$$

 **Kreditkartendaten:** StudentYes = 362.72 mit $p = 0.000898 \rightarrow$ „Balance für Studenten ist +363 höher als für Nicht-Studenten“

★★ Kategorische Features mit > 2 Ausprägungen

Für „Region“ mit 3 Ausprägungen (south, west, east):

$$x_{i1} = \begin{cases} 1 & \text{if south} \\ 0 & \text{else} \end{cases} \quad x_{i2} = \begin{cases} 1 & \text{if west} \\ 0 & \text{else} \end{cases}$$

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} = \begin{cases} \hat{\beta}_0 + \hat{\beta}_1 & \text{if south} \\ \hat{\beta}_0 + \hat{\beta}_2 & \text{if west} \\ \hat{\beta}_0 & \text{if east} \end{cases}$$

 **DIE REGEL:** Immer eine Dummy-Variable weniger als die Anzahl der Ausprägungen! (k Ausprägungen $\rightarrow k - 1$ Dummies)

- Die Ausprägung **ohne** zugehörige Dummy-Variable (hier: *east*) heißt **Baseline**
- **Jeder Modellkoeffizient erfasst die Differenz des Labels im Vergleich zu dieser Baseline**

 **Übung 3.9:** Kategorisches Feature mit **10 Leveln** $\rightarrow 10 - 1 = 9$ Dummy-Variablen

★ Dummy vs. One-Hot

	Dummy-Codierung	One-Hot-Codierung
Anzahl Variablen	$k - 1$	k (eine je Ausprägung)
Effizienz	etwas effizienter und interpretierbarer	mehr Spalten
Lineare Abhängigkeit	keine	ja! — zwischen Y-Achsenabschnitt und der Summe der one-hot-codierten Spalten
Für lineare Regression	✓ geeignet	✗ problematisch — ein Koeffizient kann nicht geschätzt werden (NA , „1 not defined because of singularities")
Für andere Modelle	ok	nicht unbedingt ein Problem

R-Beispiel des Dozenten:

```

Coefficients: (1 not defined because of singularities)
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.3662     0.1703   2.151  0.0599 .
X.1           0.1046     0.2085   0.502  0.6279
X.2          -0.0705     0.2408  -0.293  0.7763
X.3              NA          NA      NA      NA    ← nicht schätzbar!

```

⚠️ Die feine Interpretationsfalle bei mehreren Dummies

 **Region-Modell (Kreditdaten):** RegionSouth $p = 0.849$, RegionWest $p = 0.874$

Kein Nachweis, dass „Region“ mit „Balance“ zusammenhängt. **AUCH KEIN Nachweis**, dass die „Balance“ in den verschiedenen Regionen **gleich** ist!

🔑 **Wichtiger Hinweis des Dozenten:** „Es könnten auch **einzelne** Kategorien signifikant sein ($p < 0.05$). Theoretisch könnte nur **eine** der Regionen ‚south‘ und ‚west‘ unterschiedliche balance im Vergleich zu ‚east‘ aufweisen. **Wir haben eine H_0 für JEDE Dummyvariable.**“*

B.12 ★★★ Interaktionen / Wechselwirkungen (Klausur-Aufgabe 3!)

Motivation

- **Ohne Interaktion:** Auswirkung jedes Features auf sales **unabhängig** von den anderen Features
- **Annahme mit Interaktion:** Die Ausgaben für Radio erhöhen die Wirksamkeit der TV-Werbung → die Steigung β_1 für TV sollte **zunehmen**, wenn Radio zunimmt
-  Bei einem festen Budget von 100.000 \$: **sinnvoller 50 % Radio + 50 % TV** als 100 % für nur ein Medium
- **Im Marketing:** Synergieeffekt · **In der Statistik:** Interaktionseffekt oder Wechselwirkung

⚠️ **Hinweis: Interaktionen sind NICHT dasselbe wie Korrelationen / Kollinearität!**

Indikator für eine Interaktion (Folie 49)

- Wenn TV **und** radio **gleichzeitig hoch** oder **gleichzeitig niedrig** sind: **Modell unterschätzt** sales
- Wenn **nur eines** der Features niedrig ist (das andere hoch): **Modell überschätzt** sales
- 🗝️ **Das ist der typische Indikator für Interaktion.**

★★ Die Umformung (KLAUSUR-TECHNIK!)

$$\begin{aligned}\text{sales} &= \hat{\beta}_0 + \hat{\beta}_1 \cdot TV + \hat{\beta}_2 \cdot \text{radio} + \hat{\beta}_3 \cdot (TV \cdot \text{radio}) \\ &= \hat{\beta}_0 + \underbrace{(\hat{\beta}_1 + \hat{\beta}_3 \cdot \text{radio})}_{\text{effektive Steigung für TV}} \cdot TV + \hat{\beta}_2 \cdot \text{radio}\end{aligned}$$

📊 Ergebnis für Advertising:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	6.467e+00	3.613e-01	17.90	< 2e-16 ***
TV	1.998e-02	2.092e-03	9.55	1.37e-15 ***
radio	4.386e-02	1.366e-02	3.21	0.0018 **
TV:radio	1.021e-03	7.598e-05	13.43	< 2e-16 ***
Multiple R-squared: 0.9727 (vorher ohne Interaktion: 0.9214!)				

- **Positive Interaktion:** mehr Radio-Anteil bedeutet einen **stärkeren TV-Effekt** auf sales (und umgekehrt)
- TV-Einheitsanstieg (Rest konstant): sales steigt um $0.02 + 0.001 \cdot \text{radio}$
- radio-Einheitsanstieg (Rest konstant): sales steigt um $0.04 + 0.001 \cdot TV$

★★ Das Hierarchieprinzip

Wenn wir Interaktionen einbeziehen, sollten wir auch die Haupteffekte einbeziehen — trotz größerer p-Werte!

Grund: Interaktionen sind in einem Modell ohne Haupteffekte **schwer zu interpretieren** (ihre Bedeutung wird verändert!). Konkret: **Interaktionsterme enthalten Haupteffekte**, wenn das Modell keine Haupteffekte hat.

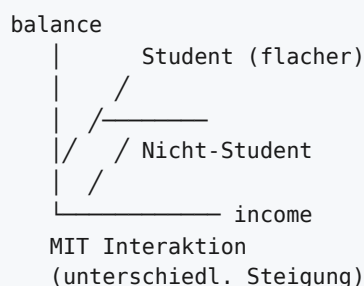
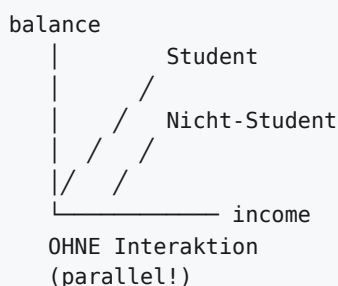
★ Interaktion numerisch × kategorisch (Kreditdaten)

Ohne Interaktion — zwei **parallele** Geraden:

$$\text{balance}_i \approx \beta_1 \cdot \text{income}_i + \begin{cases} \beta_0 + \beta_2 & \text{if student} \\ \beta_0 & \text{else} \end{cases}$$

Mit Interaktion — zwei Geraden mit **unterschiedlicher Steigung**:

$$\text{balance}_i \approx \begin{cases} (\beta_0 + \beta_2) + (\beta_1 + \beta_3) \cdot \text{income}_i & \text{if student} \\ \beta_0 + \beta_1 \cdot \text{income}_i & \text{else} \end{cases}$$



MUSTERAUFGABE (= Klausuraufgabe 3 & Übung 3.2 & 3.13)

Modell: $y = 50 - 2.5x_1 + 8x_2 - 1.2x_1x_2$, alle p-Werte < 0.05 . y = Blutdruck (diastolisch, mmHg), x_1 = Alter (Jahre), $x_2 = 1$ (Raucher) / 0 (Nichtraucher)

(a) Wie ändert sich der Blutdruck pro Jahr? Bei wem stärker?

Schritt 1 — Ableiten:

$$\frac{\partial y}{\partial x_1} = -2.5 - 1.2x_2$$

Schritt 2 — Einsetzen:

- Nichtraucher ($x_2 = 0$): $\partial y / \partial x_1 = -2.5$
- Raucher ($x_2 = 1$): $\partial y / \partial x_1 = -2.5 - 1.2 = -3.7$

Schritt 3 — Interpretieren: Nach diesem Modell **nimmt der Blutdruck mit dem Alter ab** (negatives Vorzeichen) — und zwar **stärker bei Rauchern** (-3.7 mmHg/Jahr) als bei Nichtrauchern (-2.5 mmHg/Jahr).

(b) In welchem Alter haben Raucher und Nichtraucher denselben Blutdruck?

Schritt 1 — beide Geraden aufstellen:

- Raucher: $y_R(x_1) = 50 - 2.5x_1 + 8 - 1.2x_1 = 58 - 3.7x_1$
- Nichtraucher: $y_N(x_1) = 50 - 2.5x_1$

Schritt 2 — gleichsetzen:

$$58 - 3.7x_1 = 50 - 2.5x_1 \Rightarrow 8 = 1.2x_1 \Rightarrow x_1 = \frac{8}{1.2} \approx 6,67$$

Antwort: Bei einem Alter von ca. **6,67 Jahren** sind die vorhergesagten Blutdruckwerte gleich.

MUSTERAUFGABE 2 (Übung 3.2 — GPA/IQ/Level)

X_1 =GPA, X_2 =IQ, X_3 =Level (1=College, 0=Highschool), X_4 =GPA·IQ, X_5 =GPA·Level $\hat{\beta}_0 = 50$, $\hat{\beta}_1 = 20$, $\hat{\beta}_2 = 0.07$, $\hat{\beta}_3 = 35$, $\hat{\beta}_4 = 0.01$, $\hat{\beta}_5 = -10$

Modellgleichung: $Y = 50 + 20 \cdot GPA + 0.07 \cdot IQ + 35 \cdot \text{Level} + 0.01 \cdot GPA \cdot IQ - 10 \cdot GPA \cdot \text{Level}$

Level-Terme ausklammern:

$$Y = \text{Konstante} + \text{Level} \cdot (35 - 10 \cdot GPA)$$

GPA	Vorzeichen von $(35 - 10 \cdot GPA)$	Wer verdient mehr?
> 3.5	negativ	Highschool-Absolventen
< 3.5	positiv	College-Absolventen
$= 3.5$	0	gleich viel

→ **Nur Aussage iii. ist richtig** („Highschool verdient mehr, vorausgesetzt der GPA ist hoch genug“).

Gehaltsschätzung (College, IQ=110, GPA=4.0):

$$\begin{aligned}
 Y &= 50 + 20 \cdot 4 + 0.07 \cdot 110 + 35 \cdot 1 + 0.01 \cdot 4 \cdot 110 - 10 \cdot 4 \cdot 1 \\
 &= 50 + 80 + 7.7 + 35 + 4.4 - 40 = \mathbf{137,1} \text{ (Tausend \$)}
 \end{aligned}$$

MUSTERAUFGABE 3 (Übung 3.13 — Koeffizienten aus einer Zeichnung ablesen)

Modell mit Interaktion: $\hat{Y} = \beta_0 + \beta_1x_1 + \beta_2x_2 + \beta_3x_1x_2$, x_2 dummy-codiert.

- **Rote Gerade** ($x_2 = 0$): $\hat{Y} = \beta_0 + \beta_1 x_1$
- **Blaue Gerade** ($x_2 = 1$): $\hat{Y} = (\beta_0 + \beta_2) + (\beta_1 + \beta_3) x_1$

🔑 Ablese-Regeln:

Koeffizient	Wie ablesen
β_0	Y-Achsenabschnitt der roten Geraden
β_1	Steigung der roten Geraden
β_2	Y-Achsenabschnitt blau – Y-Achsenabschnitt rot
β_3	Steigung blau – Steigung rot

Beispiellösung: $\hat{\beta}_0 \approx 1.1$, $\hat{\beta}_1 \approx 0.7$, $\hat{\beta}_2 \approx 1.2$, $\hat{\beta}_3 \approx -2.3$

B.13 ★ Nichtlinearität durch Polynomterme

Beobachtung: Das lineare Modell **überschätzt** die sales bei kleinen Werten von TV → nicht-lineare Wirkung.

$$\text{sales} = \beta_0 + \beta_1 \cdot TV + \beta_2 \cdot TV^2$$

oder mit höheren Graden:

$$\text{sales} = \beta_0 + \beta_1 TV + \beta_2 TV^2 + \beta_3 TV^3 + \beta_4 TV^4 + \dots$$

R-Syntax:

```
fit <- lm(sales ~ TV + I(TV^2), data=ads_train)      # Grad 2
fit <- lm(sales ~ TV+I(TV^2)+I(TV^3)+I(TV^4)+I(TV^5), ...) # Grad 5
fit <- lm(sales ~ poly(TV, 50, raw=TRUE), data=ads_train) # Grad 50 → OVERFIT!
```

🔑 **Wichtig:** Das Modell bleibt **linear in den Parametern** — es ist immer noch „lineare Regression“! Grad 50 zeigt eindrucksvoll **Overfitting**.

★ **Das ist die Antwort auf Klausuraufgabe 2c:** Underfit lösen **ohne Modelltypwechsel** → **polynomielle Terme und Interaktionsterme hinzufügen** oder **neue Features aus anderen Datenquellen ergänzen**.

B.14 📝 Nützliche Zusatz-Übungen

📝 **Übung 3.15 (Beweisaufgabe):** Zeigen Sie, dass ein least-squares-gefittetes lineares Modell (ein Feature) immer durch den Punkt $(\bar{x}, \bar{y})$ verläuft.

Lösung: Aus $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$ folgt durch Umstellen $\bar{y} = \hat{\beta}_0 + \hat{\beta}_1 \bar{x}$ — das ist genau die Modellgleichung, wenn für x und y jeweils der Mittelwert eingesetzt wird. ■

📝 **Übung 3.11:** Ein neuer Datenpunkt ($x = 1, y = 4$) kommt hinzu, der **oberhalb** der Geraden liegt.

Lösung: Die Gerade müsste bei $x = 1$ **nach oben gedrückt** werden, während sie bei $x = 0$ unverändert bleiben sollte (sonst würde dort der Fehler größer). Die Gerade **dreht sich also um den Punkt bei $x = 0$: Steigung steigt, Y-Achsenabschnitt bleibt (annähernd) gleich**.

▼ ? **Selbsttest Teil B (aufklappen)**

1. Wie interpretiert man einen Koeffizienten β_j in der multiplen linearen Regression? (Wortlaut!)
2. Was bedeutet ein p-Wert von 0.7 für einen Koeffizienten? Und was bedeutet er **nicht**?
3. Ein Modell hat $TSS = 200$, $RSS = 250$. Berechnen Sie R^2 und interpretieren Sie das Ergebnis.
4. Warum kann One-Hot-Encoding in der linearen Regression zu NA -Koeffizienten führen?
5. Ein kategorisches Feature hat 7 Ausprägungen. Wie viele Dummy-Variablen brauchen Sie? Was ist die Baseline?
6. Erklären Sie das Hierarchieprinzip und begründen Sie es.
7. Gegeben $y = 10 + 3x_1 - 2x_2 + 0.5x_1x_2$. Wie stark ändert sich y pro Einheit x_1 , wenn $x_2 = 4$?
8. Was ist der Unterschied zwischen Interaktion und Kollinearität?
9. Warum liefern MLE und Least Squares bei der klassischen linearen Regression dasselbe Ergebnis?
10. Wie viele Modelle müsste eine Brute-Force-Feature-Selection bei $p = 10$ Features testen? (1024)
11. Wann wird R^2 negativ?
12. Erklären Sie das Münzbeispiel zur Kollinearität.

TEIL C — KLASSIFIKATION (ISL Kap. 4)

C.1 Ausgangspunkt: Der Default-Datensatz

Credit Card Default data:

Variable	Typ	Wertebereich
default (Label Y)	kategorisch	$\{Yes, No\}$
student (X_1)	kategorisch	$\{Yes, No\}$
balance (X_2)	numerisch	$\mathbb{R}$
income (X_3)	numerisch	$\mathbb{R}$

⚠ **Wichtig:** Dieser Datensatz ist stark **unausgewogen (unbalanced)** — nur ~3 % Defaults. Darauf baut später eine zentrale Lehre auf (siehe C.10).

C.2 ★★☆☆ Hyperebenen (Grundlage für ALLES Lineare!)

Aufsteigende Definition

Objekt	Raum	Dimension	Parameterform	Koordinatenform
Gerade	$\mathbb{R}^2$	1	$\vec{p} + \lambda \vec{q}$	$n_0 + n_1x_1 + n_2x_2 = 0$
Ebene	$\mathbb{R}^3$	2	$\vec{p} + \lambda \vec{q} + \mu \vec{r}$	$n_0 + n_1x_1 + n_2x_2 + n_3x_3 = 0$
Hyperebene	$\mathbb{R}^d$	$d - 1$	—	$n_0 + n_1x_1 + \dots + n_dx_d = 0$

★ Definition (auswendig!)

Eine Hyperebene in $\mathbb{R}^d$ ist ein affiner Unterraum mit Dimension $d - 1$.

Vektorschreibweise:

$$n_0 + \vec{n}^T \vec{x} = 0$$

Dimension d	Hyperebene ist ein/eine
1	Punkt
2	Gerade
3	Ebene
d	Hyperebene ($d - 1$ -dimensional)

Wichtige Eigenschaften:

- $\vec{n} = (n_1, n_2, \dots, n_d)^T$ heißt **Normalenvektor** und ist **orthogonal zur Hyperebene**
- Falls $n_0 = 0$: Hyperebene verläuft **durch den Ursprung** (Untervektorraum), sonst nicht (affiner Unterraum)
- 🗝️ Für uns nützlich, weil: Sie teilt den zugehörigen Raum in ZWEI HÄLFTEN — also zwei Klassen!

📝 **Beispiel des Dozenten:** $\vec{n} = (1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{3})^T$ und $n_0 = -1$.

★★ Abstand Punkt ↔ Hyperebene (Formelsammlung!)

$$d = \frac{|\vec{p}^T \vec{n} + n_0|}{|\vec{n}|}$$

Oder — wenn $\vec{n}$ die **Länge 1** hat (dann muss auch n_0 angepasst werden):

$$d = |\vec{p}^T \vec{n} + n_0|$$

🗝️ **Die drei Merksätze zur Hyperebenengleichung:**

1. Hyperebenengleichung = **Skalarprodukt + Konstante**
2. liefert den **vorzeichenbehafteten Abstand** eines Punktes zu E
3. Das **Vorzeichen** besagt, **auf welcher Seite** der Hyperebene ein Punkt liegt (bzw. ob Normalenvektor und Ortsvektor in dieselbe Richtung zeigen)

📝 Übungsaufgaben zu Hyperebenen (Kap. 4, Aufg. 10 & 11)

Aufgabe 10: Punkte $P = \{(3, 4), (-3, 2), (-2, 1) \mid (2, 0), (4, 2), (0, 0)\}$ (erste 3 vs. letzte 3 trennen) → **Lösung:** $\vec{n}^T \vec{x} + b = 0$ mit $b = -2$ und $\vec{n} = \begin{pmatrix} -1 \\ 2 \end{pmatrix}$, also $-x_1 + 2x_2 - 2 = 0$

Aufgabe 11: Punkte $P = \{(-1, -2), (4, -3), (2, -2) \mid (3, 3), (1, 0), (2, 0)\}$ → **Lösung:** $b = 1$, $\vec{n} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, also $x_2 + 1 = 0 \Rightarrow x_2 = -1$

🗝️ **Klausurtyp:** *, „Mehrere Lösungen sind möglich. Eine Zeichnung der Punkte hilft.“* → Immer zeichnen!

Aufgabe 6 (Kap. 9, mit Antwort „NEIN“): $b = 4$, $\vec{n} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, Punkte

$\{(-4, -4), (0, 3), (-4, 2) \mid (4, -1), (-1, 4), (-3, 0)\}$ → Einsetzen und Vorzeichen prüfen: **Die Ebene trennt die beiden Punktmengen NICHT.**

Rechenvorschrift zum Prüfen: Für jeden Punkt $\vec{p}$ den Wert $f(\vec{p}) = n_0 + \vec{n}^T \vec{p}$ berechnen. Wenn alle Punkte der einen Menge positiv und alle der anderen negativ sind → trennende Hyperebene.

C.3 ★★ Kann man lineare Regression für Klassifikation nutzen?

Der Ansatz

Codiere ein neues Label wie eine Dummy-Variable:

$$Y^* = \begin{cases} 0 & \text{Nein (kein Default)} \\ 1 & \text{Ja (Default)} \end{cases} \Rightarrow \hat{c}(x) = \begin{cases} 1 & \text{if } \hat{f}^*(x) > 0.5 \\ 0 & \text{else} \end{cases}$$

✓ Bei 2 Klassen: funktioniert grundsätzlich

⚠ ABER: zwei Probleme

Problem 1 — keine Wahrscheinlichkeit:

Eine lineare Regression kann $\hat{f}(x) > 1$ oder $\hat{f}(x) < 0$ ergeben. **Daraus folgt: $\hat{f}(x)$ ist KEINE Wahrscheinlichkeit.** Wir hätten aber gerne einen Wert, der uns die Wahrscheinlichkeit der Vorhersage liefert, da interpretierbar.

Problem 2 — mehr als 2 Klassen: 📝 Beispiel: Patient in der Notaufnahme mit 3 möglichen Symptomen:

$$\hat{Y} = \begin{cases} 1 & \text{if stroke} \\ 2 & \text{if overdose} \\ 3 & \text{if seizure} \end{cases}$$

⚠ Die (1,2,3)-Kodierung impliziert:

- Die Symptome sind **geordnet**
- overdose – stroke = seizure – overdose $\neq$ seizure – stroke — **macht wenig Sinn!**

🔑 **Folglich:** Lineare Regression mit **nur einem Modell** ist **nicht geeignet für >2 Klassen**. **Aber:** könnte in ein **mehrfaches Zweiklassenproblem** umgewandelt werden (**one-vs-one** oder **one-vs-all**).

C.4 ★★★ Logistische Regression (Klausuraufgabe 4!)

Motivation

Wir benötigen eine **Wahrscheinlichkeit** $p(X)$, dass ein Datenpunkt X zur Klasse $Y = 1$ gehört:

$$p(X) = \text{Pr}(Y = 1 \mid X)$$

★ Die logistische Funktion (Formelsammlung!)

$$p(x) = \frac{s}{1+s} \quad \text{mit} \quad s = e^a \quad \text{und} \quad a = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n$$

★★ Warum ist das wirklich eine Wahrscheinlichkeit? (Beweisführung des Dozenten)

- Der lineare Term kann theoretisch **jede** Zahl annehmen: $a \in \mathbb{R}$
- e^a liegt zwischen 0 und ∞
- Wenn $s = 0$: $p(X) = \frac{0}{1+0} = 0$
- Wenn $s = \infty$: $p(X) = \frac{\infty}{1+\infty} = 1$
- Daher: $p(x)$ ist immer zwischen 0 und 1! ■**

★ Log-Odds / Logit-Transformation (Formelsammlung!)

$$\ln\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n$$

⚠ ln ist der **natürliche Logarithmus** (Basis e).

📝★ **Der vollständige Äquivalenzbeweis (Übung 4.1 — kann als Klausuraufgabe kommen!)**

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

$$p(X) = \frac{1}{\frac{1}{e^{\beta_0 + \beta_1 X}} + 1} \quad (\text{Zähler und Nenner durch } e^{\beta_0 + \beta_1 X} \text{ teilen})$$

$$\frac{1}{e^{\beta_0 + \beta_1 X}} + 1 = \frac{1}{p(X)}$$

$$\frac{1}{e^{\beta_0 + \beta_1 X}} = \frac{1}{p(X)} - 1 = \frac{1-p(X)}{p(X)}$$

$$e^{\beta_0 + \beta_1 X} = \frac{p(X)}{1-p(X)}$$

$$\beta_0 + \beta_1 X = \ln\left(\frac{p(X)}{1-p(X)}\right) \quad \blacksquare$$

★★ Die Entscheidungsgrenze

Frage: Wo verläuft die Entscheidungsgrenze? **Antwort (wörtlich):** Dort, wo $\hat{p}(x) = 0.5$ erreicht wird — der Schnittpunkt der Kurve mit der Linie $p = 0.5$.

Aus der Logit-Form folgt: $\ln \frac{0.5}{0.5} = \ln 1 = 0$, also **Entscheidungsgrenze** $\Leftrightarrow \beta_0 + \beta_1 X_1 + \dots = 0$ — genau die **Hyperebenengleichung**!

🔑 **Merksatz:** „Logistische Regression erzeugt immer noch eine **Hyperebene** (linear im Feature-Raum!)“ — nur die Ausgabe wird durch die logistische Funktion in $[0, 1]$ gequetscht.

📝 Rechenbeispiele des Dozenten (Default-Daten)

Modell mit nur balance: $\hat{\beta}_0 = -10.5126$, $\hat{\beta}_{balance} = 0.0054475$

balance	$a = \beta_0 + \beta_1 \cdot \text{balance}$	$s = e^a$	$p = s / (1 + s)$
1000	$-10.5126 + 5.4475 = -5.0651$	0.00631	0.006274
2000	$-10.5126 + 10.8951 = 0.3825$	1.4659	0.594466

Modell mit nur student: $\hat{\beta}_0 = -3.37288$, $\hat{\beta}_{studentYes} = 0.197655$

- Studenten: $p = \frac{e^{-3.175}}{1 + e^{-3.175}} = \mathbf{0.04011}$
- Nicht-Studenten: $p = \frac{e^{-3.373}}{1 + e^{-3.373}} = \mathbf{0.03315}$

📝★★★ MUSTERAUFGABE (= Klausuraufgabe 4)

Modell: $p(X) = \frac{s}{1+s}$ mit $s = e^{0.6 + 0.5X_1 - 0.01X_2 + 10.5X_3}$ X_1 = fehlgeschlagene Versuche in 10 min, X_2 = Minuten seit letztem erfolgreichen Login, $X_3 = 0$ (Windows) / 1 (anderes OS) Klassen: 'Fehler' vs. 'Angriff' (p gibt die Wahrscheinlichkeit für 'Angriff' an)

(a) $X_1 = 100$, $X_2 = 6000$, $X_3 = 1$ (**Linux**). Wahrscheinlichkeit für 'Angriff'? (3 Nachkommastellen)

$$a = 0.6 + 0.5 \cdot 100 - 0.01 \cdot 6000 + 10.5 \cdot 1 = 0.6 + 50 - 60 + 10.5 = 1.1$$

$$s = e^{1.1} = 3.004$$

$$p = \frac{3.004}{1 + 3.004} = \frac{3.004}{4.004} = 0.750$$

(b) Was bedeutet der konstante Wert 0.6?

Er bestimmt die Ausgabe des Modells, **wenn alle Features Null sind** — also kein Loginversuch, keine verstrichene Zeit seit letztem Login, OS ist Windows. **Wenn der konstante Faktor Null wäre, ergäbe sich eine Wahrscheinlichkeit von 50 %**, dass ein Angriff vorliegt. Mit einem Wert von 0.6 ergibt sich eine entsprechend höhere Wahrscheinlichkeit von ca. **0.65**. (Nachrechnen: $e^{0.6}/(1 + e^{0.6}) = 1.822/2.822 = 0.646$)

(c) Wie viele Loginversuche für 100 % Sicherheit?

! FANGFRAGE! Eine **100 %-Wahrscheinlichkeit ist nur als Grenzwert bestimmbar**, wenn die Anzahl der Loginversuche **gegen Unendlich** läuft. **Somit praktisch nicht möglich.**

 ★ MUSTERAUFGABE 2 (Übung 4.7 — Statistikkurs)

X_1 = Lernstunden, X_2 = Notenschnitt, Y = Eins bekommen (ja/nein) $\beta_0 = -6$, $\beta_1 = 0.05$, $\beta_2 = 1$

(a) $X_1 = 40$, $X_2 = 3.5$ — Wahrscheinlichkeit für eine Eins?

$$a = -6 + 0.05 \cdot 40 + 1 \cdot 3.5 = -6 + 2 + 3.5 = -0.5$$

$$p = \frac{e^{-0.5}}{1 + e^{-0.5}} = \frac{0.6065}{1.6065} \approx 0.38$$

(b) Wie viele Stunden für 50 % Chance? → Logit-Gleichung nutzen!

$$\ln\left(\frac{0.5}{1 - 0.5}\right) = \ln(1) = 0 = -6 + 0.05X_1 + 1 \cdot 3.5$$

$$0.05X_1 = 6 - 3.5 = 2.5 \Rightarrow X_1 = \frac{2.5}{0.05} = 50 \text{ Stunden}$$

(c) Für 100 % Chance? → Nicht definiert (Division durch Null in der Logit-Gleichung). Könnte höchstens als **Grenzwert** bestimmt werden: **unendlich viele Stunden**.

🔑 KLAUSURTECHNIK: Für „**welcher X-Wert ergibt $p = q$?**“ immer die **Logit-Form** verwenden — nicht die logistische Funktion umstellen!

 ★ MUSTERAUFGABE 3 (Übung 4.12 — Entscheidungsgrenze zeichnen)

$$p(x) = \frac{s}{1+s} \text{ mit } s = e^{\beta_0 + \beta_1 X_1 + \beta_2 X_2}, \beta_0 = 2, \beta_1 = 4, \beta_2 = -1$$

$$\text{Entscheidungsgrenze: } \beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0 \Rightarrow 2 + 4x_1 - x_2 = 0 \Rightarrow x_2 = 4x_1 + 2$$

🔑 Allgemeine Umformung (auswendig!):

$$\text{Steigung} = -\frac{\beta_1}{\beta_2}, \quad \text{Y-Achsenabschnitt} = -\frac{\beta_0}{\beta_2}$$

(hier: Steigung = $-4/(-1) = 4$, Achsenabschnitt = $-2/(-1) = 2$ ✓)

Punkte klassifizieren:

- $X = (-2, 1): a = 2 + 4(-2) - 1(1) = -7 \rightarrow p = e^{-7}/(1 + e^{-7}) \approx 0.001 \rightarrow \text{Klasse B (da } p \leq 0.5)$
- $X = (0, 0): a = 2 \rightarrow p = e^2/(1 + e^2) \approx 0.881 \rightarrow \text{Klasse A (da } p > 0.5)$

C.5 ★★ Confounder / Störfaktoren (Klassiker!)

 **Das Phänomen:** Im Modell mit **nur student** ist der Koeffizient **positiv** (+0.198). Im Modell mit **allen Features** ist er **negativ** (-0.599)!

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-1.134e+01	6.937e-01	-16.346	<2e-16	***
studentYes	-5.992e-01	3.324e-01	-1.803	0.0715	. ← NEGATIV!
balance	5.767e-03	3.213e-04	17.947	<2e-16	***
income	1.686e-05	1.122e-05	1.502	0.1331	

★ Die Erklärung (auswendig!)

- Studenten haben tendenziell eine **höhere balance**
- Höhere balance bedeutet höhere Defaultrate → somit hat die Gruppe der Studenten eine höhere **marginale** Defaultrate als Nicht-Studenten
- **ABER:** Wenn die balance zweier Personen **gleich** ist, haben Studenten **seltener** Zahlungsausfall → **negativer Koeffizient**

 **student hängt sowohl mit default als auch mit balance zusammen. Wenn balance im Modell fehlt, ist student eine unbeobachtete Störgröße — ein CONFOUNDER.**

C.6 ★ Heart-Disease-Beispiel & Case-Control-Samples

Datensatz: 303 Patienten, Label AHD (Yes/No), **139 Fälle AHD=Yes**. Features: gemessene, potentielle Risikofaktoren. Ziel: **Identifizierung der Risikofaktoren** (einschließlich Wirkstärke).

Case-Control-Design

- Oft sind **Cases** (mit Krankheit) **selten** — der limitierende Faktor
- **Controls** (ohne Krankheit): idealerweise **bis zum Fünffachen** der Zahl der Cases
- Mehr Controls → **verringert die Varianz** (wünschenswert)
-  **Aber:** Kosten, **abnehmender Ertrag** — mehr als das Fünffache lohnt sich häufig nicht

★ Das Prävalenz-Problem und die Korrekturformel

Problem: Ungleiches Case/Control-Verhältnis zwischen erhobenen Daten und Bevölkerung.

- In den Herzdaten: $\tilde{a} = 139/303 \approx 46\%$
- Prävalenz in der Bevölkerung: wahrscheinlich viel kleiner, etwa $a = 5\%$
-  **Diese Diskrepanz verzerrt den Koeffizienten für den Achsenabschnitt β_0 !**

Einfache Korrektur:

$$\hat{\beta}_0^* = \hat{\beta}_0 + \ln\left(\frac{a}{1-a}\right) - \ln\left(\frac{\tilde{a}}{1-\tilde{a}}\right)$$

 **Annahme:** Wir kennen den wahren Wert a oder haben zumindest eine begründete Vermutung.

C.7 ★ Erweiterungen und Grenzen der logistischen Regression

$K > 2$ Klassen

- Leicht zu verallgemeinern (z. B. R-Paket **glmnet**): eine lineare Funktion **für jede Klasse**
- ⚠ **Hinweis:** Die Entscheidungsgrenzen sind dann **keine Hyperebenen mehr** — sie *sehen* aus wie Hyperebenen, aber **nur lokal**

Nichtlinearität

- Log. Reg. erzeugt eine Hyperebene, **aber die Entscheidungsgrenze kann nichtlinear werden**, wenn die Modellformel nichtlineare Terme enthält
- 🔑 **Gleichbedeutend mit:** Konstruktion **neuer Features** aus alten Features und anschließender Erzeugung einer Hyperebene auf **allen** Features (→ genau die Idee hinter Kernen!)

R-Beispiel:

```
glm(default ~ balance + income + balance:income + I(income^2) + I(balance^2),
  data=def_train, family="binomial")
```

C.8 ⚠⚠★ DAS PROBLEM DER EXAKT TRENNBAREN DATEN

Ein **sehr gern geprüftes** Konzept.

Die Argumentation Schritt für Schritt

Situation: 2 Klassen, exakt trennbar durch 1 Feature bei $x_1 \approx 1.5$.

Schritt 1 — Koeffizienten sind nicht eindeutig: Hyperebene $\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$. Eine mögliche Lösung:

- $\beta_1 = 1, \beta_2 = 0$ (Normalenvektor in Richtung X-Achse), $\beta_0 = -1.5$

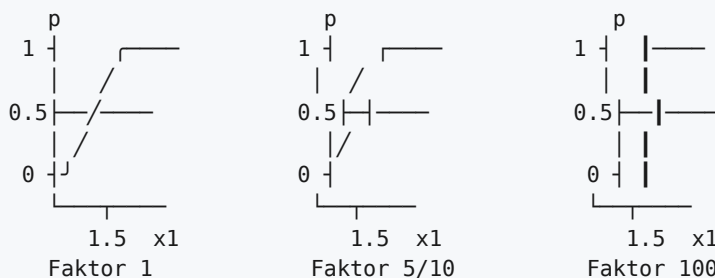
ABER: Dieselbe Hyperebene ergibt sich auch bei Multiplikation beider Seiten:

- $\beta_1 = 10, \beta_2 = 0, \beta_0 = -15$ ✓ (identische Gerade!)

Schritt 2 — aber die logistische Funktion ändert sich:

$$e^{-1.5+1 \cdot X_1} \neq e^{-15+10 \cdot X_1}$$

Die Steilheit der Sigmoid-Kurve wächst mit dem Faktor:



Schritt 3 — die Konsequenz:

Somit auch Wahrscheinlichkeiten $p(X)$, die **beliebig nahe an 1 oder 0** kommen. 🔑 **Die Optimierung liefert unendlich große oder unendlich kleine Koeffizienten!**

Schritt 4 — was R dazu sagt:

Warning: glm.fit: Algorithmus konvergierte nicht
 Warning: glm.fit: Angepasste Wahrscheinlichkeiten mit numerischem Wert 0 oder 1 aufgetreten

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-6e+01	7e+04	0	1	← absurde Werte,
x1	4e+01	4e+04	0	1	absurde SEs,
x2	6e-01	2e+04	0	1	p-Werte = 1

AIC: 6

Gegenbeispiel (nicht exakt trennbar) — funktioniert normal:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-5.8	1.3	-4.3	1e-05	***
x1	3.6	0.8	4.8	2e-06	***
x2	0.2	0.3	0.5	0.6	

AIC: 86.77

★ Lösungsansätze (auswendig!)

1. **Regularisierungsmethoden** (z. B. Bestrafung zu großer Koeffizientenwerte)
2. **Alternative Modelle** (z. B. **lineare Diskriminanzanalyse**)
3. (implizit auch: *Soft-Margin-Klassifikation / SVM*)

C.9 ★★ Diskriminanzanalyse (LDA, QDA, Naive Bayes)**Das Grundprinzip der Diskriminanzanalyse (DA)**

1. Bestimme, **wie jede Klasse bzgl. der Features X verteilt ist** — für jede Klasse **unabhängig** von den anderen
2. Bestimme die **Dichte** entsprechend dieser Verteilung
3. **Vorhersage** = die Klasse mit der **höchsten Dichte**

Mit **Normalverteilungsannahme** für jede Klasse → **lineare** oder **quadratische** Diskriminanzanalyse.
 (Aber: auch mit beliebigen anderen Verteilungen möglich.)

★ Warum Diskriminanzanalyse? (4 Gründe, auswendig!)

1. **LDA leidet nicht unter dem Problem mit exakt trennbaren Daten**
2. Wenn n **klein** ist und die Verteilung der Features in jeder Klasse **ungefähr normal** ist: **LDA ist stabiler als logistische Regression**
3. **LDA ist für > 2 Klassen beliebt**
4. **LDA kann niedrigdimensionale Visualisierungen** der Daten liefern

★★ Die drei Arten im Vergleich

Verfahren	Verteilungsannahme	Entscheidungsgrenze	Vergleichbar zu ...
LDA (linear)	Normalverteilung mit gleicher Kovarianz in jeder Klasse	linear	Log. Reg. mit linearen Termen
QDA (quadratisch)	Normalverteilung mit unterschiedlicher Kovarianz in jeder Klasse	quadratisch	Log. Reg. mit quadratischen Termen
Naive Bayes	Die Dichtefunktion ist das Produkt der individuellen Dichten — d. h. die Features werden als unabhängig angenommen	—	—

🔑 **Naive Bayes:** „Kann bei hochdimensionalen Daten sehr gut funktionieren.“

★★★★ ZUSAMMENFASSUNG: Welches Modell wann? (auswendig!)

Modell	Einsatz
Logistische Regression	nicht (exakt) trennbare Daten, oft mit wenigen Klassen
LDA	wenn n klein ist, oder die Klassen gut getrennt sind und die Gauß-Verteilungsannahmen passen. Auch für mehr als 2 Klassen .
QDA	wenn mehr Flexibilität / Nichtlinearität erforderlich ist. (Aber auch mit log. Reg. + <i>polynomialen Termen möglich</i> .)
Naive Bayes	wenn p sehr groß ist, oder bei gemischten Featuretypen (numerisch und kategorisch)

📝 Übungen 4.5 & 4.6 (LDA vs. QDA — die Standardfrage)

Wenn die Bayes-Entscheidungsgrenze LINEAR ist:

- **Auf Trainingsdaten** könnte **QDA besser** abschneiden, da durch **Rauschen** in den Daten die Verteilung leicht nicht-linear wirkt
- **Auf Testdaten** sollte (im Durchschnitt) **LDA besser** abschneiden, da es den tatsächlichen linearen Zusammenhang abbildet. **QDA würde möglicherweise overfitten.**

Wenn die Entscheidungsgrenze NICHT-LINEAR ist: meist **QDA in beiden Fällen besser.**

🔑 **Merksatz (Übung 4.6):** „Flexiblere Modelle sind **nicht immer** besser.“ → Bias-Variance-Tradeoff!

📝 **Übung 4.2:** $p = 1$ Feature, $K = 2$ Klassen, QDA ($\sigma_1 \neq \sigma_2$). Welche Form hat die Entscheidungsgrenze?

Lösung: Die Entscheidungsgrenze ist ein **Punkt** (da nur ein Feature — Hyperebene in $\mathbb{R}^1$ ist 0-dimensional).

C.10 ★★★★★ UNAUSGEWOGENE DATEN (Klausur-Dauerbrenner!)

Das Beispiel des Dozenten (LDA auf Default-Daten)

Konfusionsmatrix auf Trainingsdaten:

predicted	observed	
	0	1
0	4808	123
1	16	53

Fehlerrate = $(16 + 123)/5000 = 2.78\%$ — „Klingt toll.“

Konfusionsmatrix auf Testdaten:

predicted	observed	
	0	1
0	4774	157
1	69	0

Fehlerrate = $(69 + 157)/5000 = 4.52\%$ — „Klingt immer noch einigermaßen gut. Oder doch nicht?“

⚠️ Die Zerlegung — DAS Aha-Erlebnis

Betrachtung	Rechnung	Ergebnis
Nur die tatsächlichen [No]-Samples	$69/(4774 + 69)$	1.42 % Fehler
Nur die tatsächlichen [Yes]-Samples	$157/157$	100 % FEHLER!
Naives Modell: immer [No] vorhersagen	$157/5000$	3.14 % Fehler

🔑 Das naive „immer No“-Modell scheint besser zu sein, ist aber als Modell völlig unbrauchbar!

⚠️ **Problem (wörtlich):** „Unser Datensatz ist **unausgewogen (unbalanced)**. Sonst übliche Gütemaße sind **irreführend**, insbesondere die obig errechneten Fehlerraten.“*

★★ Bessere Maße für unausgewogene Daten

Maß	Definition
F1-Score	harmonisches Mittel aus Precision und Recall
Balanced Error / Balanced Accuracy	Mittelwertbildung des Fehlers pro Klasse, über alle Klassen

Balanced Error für LDA auf Default-Daten:

$$\frac{1.42\% + 100\%}{2} = 50.71\%$$

🔑 „Ergibt ein **realistischeres** Maß als die einfache Fehlerrate — **nicht durch die Mehrheitsklasse verzerrt.**“

📝 ★ Übung 4.9 (Musterantwort auswendig!)

Frage: 95 % Klasse 1, 5 % Klasse 2. Welches Problem, welche Lösung?

Lösung: Die schlechte Balance der Klassen sorgt dafür, dass bestimmte Maße wie **accuracy** oder **misclassification rate** zu optimistisch oder pessimistisch bewerten. So hätte ein Modell, das **immer Klasse 1 vorhersagt**, einen relativ kleinen Fehler von nur **5 %** (bzw. 95 % accuracy), obwohl es **tatsächlich keinerlei Nutzen** erbringt.

Mögliche Lösungen:

1. **Bessere Fehlermaße** (z. B. F1-Score oder balanced accuracy)
2. **Over-/Under-Sampling** der Daten
3. **Aufnahme neuer Daten der Minderheitsklasse**

C.11 Der Iris-Datensatz und Multiklassen-Praxis

Iris („Schwertlilie“): 4 Features, **3 Spezies** (Setosa, Versicolor, Virginica), **50 Datenpunkte je Klasse** = 150 gesamt. 🗝️ **LDA klassifiziert alle bis auf 3 der 150 Trainingsbeispiele richtig.**

⚠️ Warum `glm()` hier nicht funktioniert

1. **Zwei der Klassen sind exakt trennbar** (→ Problem aus C.8)
2. **`glm` ist nur für binäre Klassifikation** — hier aber 3 Klassen!

Idee: je ein Modell pro Klasse trainieren. ⚠️ Nachteile: **aufwändig** und liefert **Klassenwahrscheinlichkeiten, die in Summe nicht 1 ergeben**. ✅ **Besser:** `glmnet` (penalized / multinomial logistic regression).

Ergebnis mit `caret` + `glmnet`, 5-fold CV:

Confusion Matrix and Statistics

Prediction	Reference		
	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	10	2
virginica	0	0	8

Accuracy : 0.9333 95% CI : (0.7793, 0.9918)

No Information Rate : 0.3333 Kappa : 0.9

▼ ? Selbsttest Teil C (aufklappen)

1. Definieren Sie „Hyperebene“. Was ist eine Hyperebene in $\mathbb{R}^1$, $\mathbb{R}^2$, $\mathbb{R}^5$?
2. Nennen Sie zwei Gründe, warum lineare Regression für Klassifikation problematisch ist.
3. Beweisen Sie, dass $p(x) = s/(1+s)$ mit $s = e^a$ immer in $[0, 1]$ liegt.
4. Gegeben $s = e^{1+2X_1-0.5X_2}$. Berechnen Sie p für $X_1 = 2$, $X_2 = 4$. ($a=3$, $p=0.953$)
5. Bei welchem X_1 liegt in Aufgabe 4 (mit $X_2 = 4$) die Entscheidungsgrenze? ($1+2X_1-2=0 \rightarrow X_1=0.5$)
6. Erklären Sie den Confounder-Effekt am Beispiel `student` / `balance` / `default`.
7. Warum divergieren die Koeffizienten der logistischen Regression bei exakt trennbaren Daten?
8. Nennen Sie zwei Lösungen für dieses Divergenzproblem.
9. LDA vs. QDA: Welche Verteilungsannahme unterscheidet sie?
10. Wann ist Naive Bayes besonders geeignet?
11. Ein Modell hat 98 % Accuracy bei 98 % Klasse-A-Anteil. Bewerten Sie.
12. Berechnen Sie den Balanced Error bei Klassenfehlern von 2 % und 60 %.
13. Wie lautet die Case-Control-Korrekturformel und wann brauchen Sie sie?
14. Was ist das Verhältnis Controls:Cases, das laut Dozent maximal sinnvoll ist? (5:1)

TEIL D — SUPPORT VECTOR MACHINES (ISL Kap. 9)

D.1 Der Aufbau des Kapitels

Der Dozent behandelt **drei aufeinander aufbauende Verfahren**:

Hard Margin Classification
 ↓ (erlaubt Verletzungen)
 Soft Margin Classification = Support Vector Classification
 ↓ (erweitert Feature Raum implizit)
 Support Vector Machines (SVM)

⚠ **Einschränkung aller drei: Binary Classification** — nur genau 2 Klassen (Multiklassen später über OVO/OVA).

D.2 ★★ Trennende Hyperebene — Prüfkriterium

Koordinatengleichung: $f(\vec{x}) = n_0 + \vec{n}^T \vec{x}$

- $f(\vec{x}) > 0$ für Punkte auf **einer** Seite
- $f(\vec{x}) < 0$ für Punkte auf der **anderen** Seite

★ Das formale Kriterium (auswendig!)

Neukodierung der Klasse: $y_i = +1$ (Klasse ×) und $y_i = -1$ (Klasse ○)

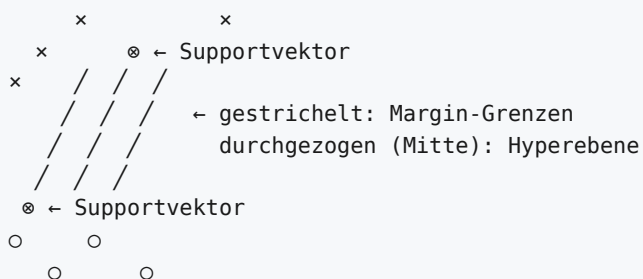
Wenn $(y_i \cdot f(\vec{x}_i) > 0) \forall i$ ODER $(y_i \cdot f(\vec{x}_i) < 0) \forall i$, dann ist $f(\vec{x}) = 0$ eine trennende Hyperebene

🔑 **Warum „ODER“?** Weil das Vorzeichen des Normalenvektors willkürlich ist — man kann $\vec{n}$ auch umdrehen. Entscheidend ist nur, dass **alle Produkte dasselbe Vorzeichen** haben.

D.3 ★★ Hard Margin Classifier

Definitionen

Begriff	Definition
Margin	minimaler Abstand zu den Datenpunkten
Hard Margin Classifier	die trennende Hyperebene mit dem größten Margin
Supportvektoren	die Punkte exakt auf den Margin-Grenzen



★ Warum „Supportvektoren“? (3 Aussagen, auswendig!)

1. Nur Supportvektoren „stützen“ oder „bestimmen“ die Hyperebene
2. Kleine Änderungen der Supportvektoren ändern die Lage der Ebene
3. Kleine Änderungen von Nicht-Supportvektoren haben **KEINEN Einfluss** auf die Lage der Ebene

★ Margin-Kriterium

Trennende Hyperebene **ohne Verletzung des Margins** M :

$$(y_i \cdot f(\vec{x}_i) \geq M) \quad \forall i \quad \text{ODER} \quad (y_i \cdot f(\vec{x}_i) \leq -M) \quad \forall i$$

⚠ **Voraussetzung: Der Normalenvektor $\vec{n}$ muss NORMIERT sein!** Sonst steht auf der linken Seite der Ungleichung **keine Distanz**.

★ Das Optimierungsproblem (Hard Margin)

Für Daten mit p Spalten (Features) und m Zeilen (Beobachtungen):

$$\begin{aligned} & \max_{n_0, n_1, \dots, n_p} M \\ \text{u.d.N.} \quad & \sum_{j=1}^p n_j^2 = 1 \quad (\text{Normierung}) \\ & y_i \cdot (n_0 + \vec{n}^T \vec{x}_i) \geq M \quad \forall i \in \{1, \dots, m\} \end{aligned}$$

Anmerkung des Dozenten: Kann in ein **quadratisches Problem** umformuliert werden. Entsprechend häufig mit Algorithmen für quadratische Probleme gelöst.

⚠ Grenzen des Hard Margins

Problem: Eine trennende Hyperebene **existiert nicht**. Das ist **üblich** — reale Daten sind häufig nicht linear trennbar (wenn $m \geq p$).

D.4 ★★ Soft Margin / Support Vector Classifier

Grundidee (wörtlich)

- Die Daten sind zwar **nicht exakt** linear trennbar
- **Aber vielleicht ungefähr** linear trennbar
- d. h.: ein **Großteil** der Daten (aber nicht alle) wird durch die Hyperebene getrennt

★ Supportvektoren beim Soft Margin (erweiterte Definition!)

Supportvektoren sind **alle Punkte, die den Margin verletzen**:

1. Punkte **exakt auf der Margin-Grenze**
2. Punkte **innerhalb** des Margins
3. Punkte sogar auf der **falschen Seite** der Hyperebene

★ Das Optimierungsproblem (Soft Margin) — vollständig

$$\max_{n_0, n_1, \dots, n_p; \epsilon_1, \dots, \epsilon_m} M \quad (1)$$

$$\text{u.d.N.} \quad \sum_{j=1}^p n_j^2 = 1 \quad (2)$$

$$y_i \cdot (n_0 + \vec{n}^T \vec{x}_i) \geq (M - \epsilon_i) \quad (3)$$

$$\epsilon_i \geq 0 \quad (4)$$

$$\forall i \in \{1, \dots, m\} \quad (5)$$

$$\sum_{i=1}^m \epsilon_i \leq C \quad (6)$$

🔑 C ist ein „Budget“, das die Summe der ϵ_i nicht überschreiten darf.

- ϵ_i = Slack-Variable = Ausmaß der Margin-Verletzung des i -ten Punkts
- Großes C → viele Verletzungen erlaubt → **breiterer Margin, mehr Bias, weniger Varianz**
- Kleines C → wenige Verletzungen → **schmalere Margin, weniger Bias, mehr Varianz**

📝 Übung 9.3 (Hard vs. Soft — Musterantwort)

Unterschied: Hard-Margin setzt voraus, dass alle Trainingsdatenpunkte außerhalb der Margins liegen. Soft-Margin erlaubt eine Verletzung der Margins.

Wann welche?

- Bei **linear trennbaren Daten mit wenig Rauschen** (geringer nicht-reduzierbarer Fehler) sind **Hard-Margins zu bevorzugen**
- Bei **nicht linear trennbaren Daten** erlauben **Soft-Margins** überhaupt erst den Einsatz einer linearen Trennfunktion
- Zudem sind Soft-Margins **weniger anfällig für extreme Ausreißer** im Datensatz

D.5 ★★ Erweiterung des Feature-Raumes

Ansatz 1: Polynome

Erweiterung des Feature-Raumes der Daten durch **Transformationen**, z. B. $X_1^2, X_2^2, X_1 \cdot X_2, \dots \rightarrow$ Trainieren des Support-Vector-Klassifikators **im erweiterten Raum** → **Ergebnis: nicht-lineare Entscheidungsgrenzen im ursprünglichen Raum**

📝 **Beispiel kubisches Polynom** — Erweiterung des **2D-Raums** zu einem **9D-Raum**:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 + \beta_6 X_1^3 + \beta_7 X_2^3 + \beta_8 X_1 X_2^2 + \beta_9 X_1^2 X_2 = 0$$

🔑 „Der **lineare** Support-Vector-Klassifikator im **erweiterten** Raum löst das **nicht-lineare** Problem im **niederdimensionalen** Raum.“

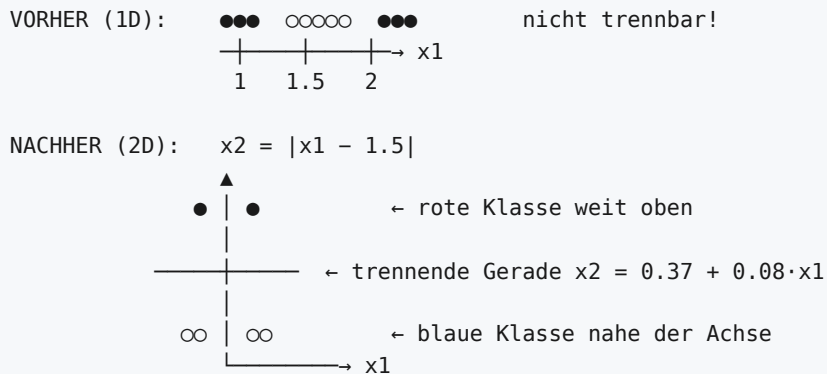
⚠️ Probleme mit polynomieller Erweiterung

1. Polynome höherer Ordnung werden **sehr schnell numerisch problematisch**
2. Es ist **unklar, welche Dimension / welchen Polynomgrad** wir brauchen; bei falscher Wahl kann das Modell **trotzdem scheitern**

☆☆ Ansatz 2: Distanzen (die Kernel-Idee!)

📝 Beispiel des Dozenten (1D, 2 Klassen, nicht trennbar):

1. Wähle einen Punkt in der Mitte des Raumes: $x'_1 = 1.5$
2. Berechne eine **neue, zweite Dimension**: $x_2 = |x_1 - x'_1|$ (Abstand jedes x_1 zu x'_1)
3. Berechne dieses neue Feature für **jeden Trainingsdatenpunkt**



Ergebnis: Entscheidungsgrenze $x_2 \leq 0.37 + 0.08x_1 \Rightarrow$ im ursprünglichen Raum: $|x_1 - 1.5| \leq 0.37 + 0.08x_1$
 $\Rightarrow$ mit Fallunterscheidung gelöst: **Rote Klasse wenn** $x_1 \in [1.046296, 2.032609]$

☆☆ Die zentrale Frage und ihre Antwort

? „Aber woher wissen wir, dass wir $x_1 = 1.5$ wählen sollten? Wie würden Sie einen Punkt auswählen?“

🔑 **Antwort:** „Wir wählen einfach **ALLE Trainingsdatenpunkte** aus (erzeugen neue Features, die den Abstand aller Punkte zu jedem anderen Punkt ausdrücken).“

Anstelle eines fest definierten Abstands wird normalerweise eine **parametrisierte Funktion** bzw. häufig eine **Kernel-Funktion** verwendet.

★ Der Radial-Kernel (RBF, Formelsammlung!)

$$K(\vec{x}, \vec{x}') = e^{-\gamma \sum_{j=1}^p (x_j - x'_j)^2}$$

D.6 ☆☆☆ Der Kernel-Trick

Das Problem

Berechnung der Margin-Maximierung in einem **extrem hochdimensionalen Raum** kann **rechenaufwändig** sein.

Die Lösung: Kernel-Trick

Voraussetzungen an die Kernel-Funktion:

- **Symmetrie**
- **Definitheit** (positiv semidefinit)
- ⚠ „nicht ganz einfach zu prüfen“

★ Schritt-für-Schritt-Herleitung

Schritt 1 — Hyperebene als Skalarprodukt:

$$f(\vec{x}) = n_0 + n_1 x_1 + \dots + n_p x_p = n_0 + \vec{n}^T \vec{x}$$

Schritt 2 — Skalarprodukt ist inneres Produkt:

$$f(\vec{x}) = n_0 + \vec{n}^T \vec{x} \quad \equiv \quad f(\vec{x}) = n_0 + \langle \vec{n}, \vec{x} \rangle$$

Schritt 3 — auch Kernelfunktionen sind innere Produkte:

$$K(\vec{x}_i, \vec{x}'_i) = \langle \phi(\vec{x}_i), \phi(\vec{x}'_i) \rangle$$

mit einer **Abbildungsfunktion** $\phi(X) : \mathbb{R}^p \rightarrow \mathcal{H}$

🔑 **Das ist der Kern der Sache:** „Wenn wir K berechnen, berechnen wir ein inneres Produkt — aber **nicht im ursprünglichen Feature-Raum** $\mathbb{R}^p$, sondern **implizit in einem hoch-dimensionalen Raum** $\mathcal{H}$, der durch den Kernel erzeugt wird.“

Schritt 4 — Hyperebene im Raum $\mathcal{H}$:

$$f(\vec{x}) = n_0 + \sum_{i=1}^n \alpha_i \langle \phi(\vec{x}_i), \phi(\vec{x}) \rangle = n_0 + \sum_{i=1}^n \alpha_i K(\vec{x}_i, \vec{x})$$

Schritt 5 — die Pointe:

🔑 „Wir müssen hierzu **nur** K **berechnen, nie** ϕ . Spart Aufwand!“ ★* „Insbesondere hilfreich, weil: ϕ_{radial} **bildet in einen UNENDLICH-dimensionalen Raum $\mathcal{H}$ ab!**“*

★ Die Gewichte α_i

Aussage
α_i sind Gewichte, die besagen, wie stark jeder Datenpunkt die Hyperebene in $\mathcal{H}$ beeinflusst
Werden beim Modell-Training bestimmt
$\alpha_i \neq 0 \iff$ Support-Vektoren
Alle Datenpunkte $\vec{x}_i$ mit $\alpha_i = 0$ werden für die Vorhersage NICHT benötigt

★★ Vorhersage des SVM-Modells (Formelsammlung!)

$$f(\vec{x}) = n_0 + \sum_{i=1}^n \alpha_i K(\vec{x}_i, \vec{x})$$

Klassenzuordnung (binäre Klassifikation) über das Vorzeichen:

$$\hat{c}(\vec{x}) = \text{sign}(f(\vec{x}))$$

📝★★★ MUSTERAUFGABE 1 (Übung 9.7 — Radial-Kernel) — TYPISCHE KLAUSURAUFGABE!

Gegeben: Radial-Kernel mit $\gamma = 0.5$, Bias $n_0 = 1$ **Trainingsdaten:** $\vec{x}_1 = (1, 1)$, $\vec{x}_2 = (1, 3)$, $\vec{x}_3 = (2, 2)$, $\vec{x}_4 = (3, 3)$, $\vec{x}_5 = (3, 3)$ $\vec{\alpha} = (0.2, 0.0, 0.0, 0.8, 0.0)^T$ **Testdatenpunkt:** $\vec{x} = (1, 2)$

Schritt 1 — Welche Kernel muss ich überhaupt berechnen? 🔑 **Nur für die Punkte mit $\alpha_i \neq 0$!** Hier: $i = 1$ **und** $i = 4$. Die anderen haben $\alpha_i = 0$ und **nehmen keinen Einfluss** auf die Vorhersage.

Schritt 2 — Kernelwerte berechnen:

$$K((1, 2), (1, 1)) = e^{-0.5[(1-1)^2 + (2-1)^2]} = e^{-0.5 \cdot 1} = e^{-0.5} \approx \mathbf{0.6065307}$$

$$K((1, 2), (3, 3)) = e^{-0.5[(1-3)^2 + (2-3)^2]} = e^{-0.5 \cdot 5} = e^{-2.5} \approx \mathbf{0.082085}$$

Schritt 3 — Vorhersage:

$$f(\vec{x}) = n_0 + \sum_i \alpha_i K(\vec{x}, \vec{x}_i) = 1 + (0.2 \cdot 0.6065307) + (0.8 \cdot 0.082085) = \mathbf{1.186974}$$

Schritt 4 — Klasse: $\text{sign}(1.187) = +1$ **★ MUSTERAUFGABE 2 (Übung 9.8 — linearer Kernel)**

Linearer Kernel: $K(\vec{x}, \vec{x}') = \vec{x}^T \vec{x}'$, Bias $n_0 = 2$ Dieselben Trainingsdaten, $\vec{\alpha} = (0.7, 0.2, 0.0, 0.1, 0.0)^T$, Testpunkt $\vec{x} = (1, 2)$

Nur $i = 1, 2, 4$ **relevant:**

$$K((1, 2), (1, 1)) = 1 \cdot 1 + 2 \cdot 1 = \mathbf{3}$$

$$K((1, 2), (1, 3)) = 1 \cdot 1 + 2 \cdot 3 = \mathbf{7}$$

$$K((1, 2), (3, 3)) = 1 \cdot 3 + 2 \cdot 3 = \mathbf{9}$$

$$f(\vec{x}) = 2 + (0.7 \cdot 3) + (0.2 \cdot 7) + (0.1 \cdot 9) = 2 + 2.1 + 1.4 + 0.9 = \mathbf{6.4}$$

D.7 ★ SVM für mehr als 2 Klassen

Verfahren	Beschreibung	Anzahl Modelle
OVA — One vs. All	Anpassung von K unterschiedlichen 2-Klassen-Klassifikatoren $\hat{f}_k(x)$ — jede Klasse gegen den Rest . Ordne x^* der Klasse zu, für die $\hat{f}_k(x^*)$ am größten ist.	K
OVO — One vs. One	Anpassung aller paarweisen Klassifikatoren. Ordne x^* der Klasse zu, die die meisten paarweisen Vergleiche gewinnt.	$\binom{K}{2} = \frac{K(K-1)}{2}$

★ Was soll man wählen? (auswendig!)

- **Wenn K nicht zu groß ist: OVO verwenden**
- **Andernfalls: OVA verwenden** (spart Rechenaufwand)

 **Übung 9.5:** „Wählen Sie die Möglichkeit, die für eine **sehr große Zahl von Klassen** besser geeignet ist.“

OVA — da hier für K Klassen **nur K Modelle** erzeugt werden müssen. OVA erzeugt jeweils ein Modell, das die Trennung **einer Klasse von allen anderen** modelliert.

D.8 ★★★★★ Modellwahl: Welches Modell wann? (Folie 38 — Klausurgold!)

Situation	Empfehlung
Klassen sind (fast) exakt trennbar	Soft-Margin-Klassifikator schneidet besser ab als Logistische Regression. <i>Das gilt auch für LDA.</i>
Klassen sind nicht trennbar	Log. Reg. und Soft-Margin-Klassifikator können sehr ähnlich sein
Sie brauchen Wahrscheinlichkeiten	Log. Reg. oder LDA verwenden. <i>(Aber: Es gibt SVM-Varianten, die auch Wahrscheinlichkeiten liefern!)</i>
Nichtlineare Daten/Probleme	SVM bzw. Kernel verwenden. <i>(Aber: Kernel können auch für LR und LDA verwendet werden — obwohl vielleicht rechnerisch weniger effizient.)</i>

⚠️ Schlussbemerkung des Dozenten zu Implementierungen

- Implementierungen und Definitionen können sich in **wichtigen Details unterscheiden**
- Typische Implementierungen (R und Python) haben Parameter wie C , **aber sie bedeuten oft nicht das Gleiche**, weil die Definitionen abweichen
- Die Modelle folgen den gleichen Konzepten, können sich aber bzgl. ihrer **Parameter unterschiedlich verhalten**

D.9 📝 Die Klausur-Musterantwort zu SVM (Klausuraufgabe 5, 5 Punkte)

Frage: Erläutern Sie den Zusammenhang von Support Vector Machines und Hyperebenen.

Musterlösung (wörtlich, auswendig!): Hyperebenen können für die **binäre Klassifikation** genutzt werden, da sie einen Raum **in zwei Hälften teilen**. Eine Methode, um für diesen Zweck Hyperebenen zu bestimmen, ist **Soft-Margin Classification**. Allerdings ist das **nicht in jedem Feature-Raum sinnvoll**, da eine Hyperebene zumindest eine **angenäherte lineare Trennbarkeit** voraussetzt. In bestimmten Fällen sind Daten **nicht (angenähert) linear trennbar**.

SVMs setzen daher **zwar ebenfalls Hyperebenen ein, konstruieren diese aber in einem erweiterten, hochdimensionalen Featureraum**, in dem die angenäherte lineare Trennbarkeit gegeben ist. Der **Kernel-Trick** erlaubt es dabei, die Hyperebene zu bestimmen, **ohne die Abbildungen in den hochdimensionalen Raum explizit berechnen zu müssen**.

D.10 📝 Weitere SVM-Übungsantworten

📝 Übung 9.1 (Warum helfen Kernel?):

Kernel ermöglichen die **Erstellung neuer Features (implizit oder explizit)**. Nach der Ergänzung dieser Features sind die Daten im **höherdimensionalen Featureraum möglicherweise linear trennbar**. Im ursprünglichen Featureraum ergibt sich so ein **nicht-lineares Modell** (z. B. nichtlineare Entscheidungsgrenze). *Beispiel: neues Feature = Abstand zum Punkt $X = 6$; im ursprünglichen Raum nicht trennbar, im erweiterten Raum mit $X_2 = 2.5$ trennbar.*

Übung 9.2 (kNN ↔ Kernelmethoden):

Ähnlich wie k-Nearest Neighbors arbeiten auch **Kernelmethoden (z. B. mit Gauß'schen Kernen) mit Nachbarschaften bzw. Distanzen**. Die Distanzberechnung in kNN könnte auch als Kernel interpretiert werden. Zudem gibt es Varianten von kNN, die **explizit Kernel einsetzen**, z. B. um Trainingsdatenpunkte mit unterschiedlicher Distanz zum Testdatenpunkt auch **unterschiedlich zu gewichten**.

D.11 R-Praxis (Lab 5)

```
library(e1071); library(ggplot2); set.seed(123)

# Künstlicher, nichtlinear separierbarer Datensatz (Kreis)
x1 <- runif(100, -2, 2); x2 <- runif(100, -2, 2)
y <- ifelse(x1^2 + x2^2 < 1.5, "Class_A", "Class_B")
dat <- data.frame(x1=x1, x2=x2, y=factor(y))

svm_model <- svm(y ~ ., data = dat,
                 kernel = "radial", # nichtlinearer Kernel
                 cost = 10,        # ~ 1/C
                 gamma = 1)        # Kernel-Breite

grid <- expand.grid(x1 = seq(min(dat$x1)-0.2, max(dat$x1)+0.2, length.out=300),
                  x2 = seq(min(dat$x2)-0.2, max(dat$x2)+0.2, length.out=300))
grid$pred <- predict(svm_model, newdata = grid)
```

Lab-Aufgabe: Den **Moons-Datensatz** (zwei ineinandergreifende Halbmonde) mit SVM modellieren und vergleichen, was mit `kernel="radial"` vs. linear passiert.

▼ ? Selbsttest Teil D (aufklappen)

1. Wie prüft man formal, ob $f(\vec{x}) = 0$ eine trennende Hyperebene ist?
2. Was ist der Margin? Was ist ein Supportvektor beim Hard Margin? Und beim Soft Margin?
3. Warum muss $\vec{n}$ normiert sein, damit $y_i f(x_i) \geq M$ Sinn ergibt?
4. Was bedeutet der Parameter C im Soft-Margin-Optimierungsproblem?
5. In welchen Raum bildet ϕ_{radial} ab? Warum ist der Kernel-Trick genau deshalb wertvoll?
6. Wieso reicht es, K zu berechnen und nie ϕ ?
7. Welche zwei Eigenschaften muss eine Kernelfunktion haben?
8. Was bedeutet $\alpha_i = 0$?
9. Berechnen Sie $K((0, 1), (2, 2))$ für den Radial-Kernel mit $\gamma = 1$. ($e^{-5} \approx 0.0067$)
10. Bei 100 Klassen: OVO oder OVA? Wie viele Modelle jeweils? (OVA: 100; OVO: 4950)
11. Sie brauchen Klassenwahrscheinlichkeiten. Welches Modell?
12. Nennen Sie zwei Probleme der polynomiellen Feature-Erweiterung.

TEIL E — VALIDATION UND RESAMPLING (ISL Kap. 5)

E.1 ★ Warum überhaupt Resampling?

Trainingsfehler vs. Testfehler

	Definition	Zweck
Trainingsfehler	Fehler bei der Vorhersage für Trainingsdaten	Misst den Fortschritt des Trainingsprozesses
Testfehler	Fehler bei der Vorhersage für Testdaten	Schätzt die Qualität für neue, ungesehene Daten

Häufiges Problem: Der Trainingsfehler unterscheidet sich **stark** vom Testfehler. Der Trainingsfehler kann den Testfehler **dramatisch unterschätzen**. ⚠ **Hinweis des Dozenten:** *Manchmal kann der Trainingsfehler den Testfehler auch **überschätzen**.*

★★ Die zwei Probleme des reinen Train/Test-Splits

1. Wir sollten keine Entscheidungen über Modelle auf der Grundlage von Testdaten treffen 2. Ein einziger Testdatensatz liefert eine einzige Qualitätsbeobachtung — also keine Informationen über Variation

Unser Ziel: zusätzliche Informationen über das Modell erhalten:

- **Vorabschätzung des Testfehlers** (ohne Testdaten zu verwenden!)
- **Varianz** unserer Schätzungen

Welche „Entscheidungen“ sind gemeint?

- **Modellauswahl**
- **Auswahl der Features** (Feature Selection)
- **Parametrisierung** des Modells (Hyperparameter-Tuning)

⚠ **Wenn mit dem Testdatensatz durchgeführt:**

- Kann zu **Overfit** führen
- **Testfehler** sagen nichts mehr über die Leistung bei völlig neuen Daten aus

E.2 ★★★ Die 3-Wege-Aufteilung (= Klausuraufgabe 6!)

Schlussfolgerung 1

Die Daten müssen in 3 Teile aufgeteilt werden, statt nur in 2!

Teil	Zweck
Trainingsdaten	ein Modell erstellen (Anpassung des Modells an die Daten)
Validierungsdaten	bewerten und Entscheidungen treffen (Konfiguration, Auswahl)
Testdaten	das endgültige Modell bewerten

🔑 Der Validierungsfehler dient zur Vorabschätzung des Testfehlers — ohne die eigentlichen Testdaten zu betrachten.

Schlussfolgerung 2

Wir brauchen nicht nur einen, sondern idealerweise MEHRERE Validierungsdatensätze — um abzuschätzen, wie sich das Modell ändert, wenn sich die Testdaten ändern.

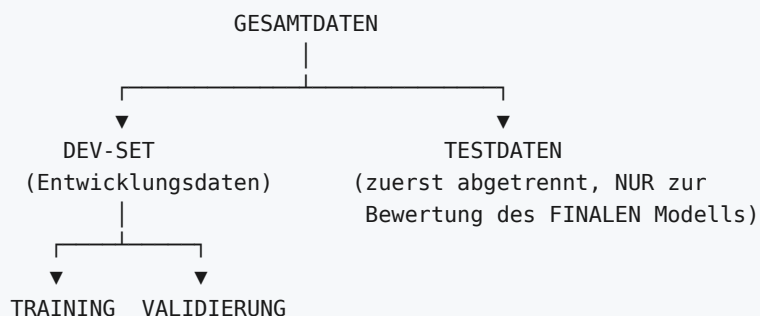
Begründung: Eine einzige Beobachtung ist selten ausreichend. Es ist wichtig, die Auswirkungen einer (kleinen) Änderung des Datensatzes zu messen → ermöglicht die Beurteilung der **Robustheit** des Modells.

!★ **Die Terminologie des Dozenten (Prüfungsrelevant, weil in der Literatur uneinheitlich!)**

„Die Terminologie ist in der Literatur leider **nicht immer einheitlich**:

- Dev Set / Entwicklungsdaten werden **manchmal** als Validierungsdaten verstanden
- Ebenso werden die Begriffe **Validierungsdaten und Testdaten manchmal synonym** verwendet“

UNSERE Terminologie (verbindlich für die Klausur):



1. Die **Testdaten** werden **zuerst** abgetrennt und **nur zur Bewertung des finalen Modells** verwendet
2. Die verbleibenden Daten sind das **Dev-Set**, weiter aufgeteilt in **Trainings-** und **Validierungsdaten**

📝☆☆ Die Klausur-Musterantwort (Aufgabe 6, 6 Punkte) — auswendig!

Frage: Begründen Sie, warum neben der Abspaltung von Test- und Trainingsdaten meist auch eine dritte Datenmenge (Validierungsdaten) von den Trainingsdaten abgespalten wird.

Musterlösung: Häufig werden neben dem Training des Modells auch **Entscheidungen über das Modell** getroffen (z. B. Hyperparameterwahl, Feature-Selection, etc.).

- Wenn diese Entscheidungen **auf den Trainingsdaten** getroffen werden, kann dies zu **Overfitting** führen (was dann auf Testdaten nur **erkannt**, aber nicht **verhindert** wird).
- Wenn stattdessen Entscheidungen **auf den Testdaten** getroffen werden, **fehlt eine unabhängige Datenmenge** — bzw. ein Overfitting könnte **nicht mal erkannt** werden.

Die **Validierungsdatenmenge** erlaubt, Entscheidungen zum Modell zu treffen und dabei schon eine **Vorabschätzung für den Testfehler** vorzunehmen, **ohne Testdaten zu verwenden**. Somit kann durch entsprechende Entscheidungen ein **Overfitting verhindert** werden.

E.3 ☆ Hold-Out Validation

Verfahren

1. Aufteilung des Dev-Sets **nach dem Zufallsprinzip** in zwei Teile: Trainings- und Validierungsdaten
2. Modell auf Trainingsdaten trainieren
3. Das trainierte Modell sagt die Labels für die Validierungsdaten vorher
4. **Wiederholen** des Vorgangs mit **verschiedenen Aufteilungen** des Dev-Sets
5. Anschließend geeignet zusammenfassen:
 - **Mittlerer Validierungs-Fehler** → Vorabschätzung des Testfehlers
 - **Varianz des Validierungsfehlers** → Wie sicher ist unsere Schätzung?

⚠️ Nachteile von Hold-Out (auswendig!)

1. Die **Validierungsschätzung des Testfehlers kann sehr stark variieren** — je nachdem, welche Beobachtungen in welcher Menge enthalten sind
2. Nur eine **kleinere Teilmenge** der Daten wird zur Anpassung des Modells verwendet — die Abtrennung der Validierungsdaten **reduziert die Mächtigkeit der Trainingsdatenmenge**
3. 🗝️ **Folglich: Der Validierungsfehler neigt zur ÜBERSCHÄTZUNG des Testfehlers**

E.4 ☆☆☆ K-fold Cross-Validation

Verfahren (auswendig!)

1. Dev-Set nach dem Zufallsprinzip in k **nicht überlappende, gleich große Teilmengen** $X_{dev.k} \subset X_{dev}$ aufteilen
2. Für **jede** dieser Teilmengen:
 - Teilmenge $X_{dev.k}$ wird **nicht** zum Training verwendet
 - Modell wird auf **allen anderen** Daten trainiert
 - Fehler des Modells wird **auf** $X_{dev.k}$ bestimmt
3. Abschließend: **Aggregieren aller erzeugten Fehlerschätzungen** (Mittelwert, Varianz usw.)

🔑 Die nicht-überlappenden Teilmengen heißen FOLDS.

5-fold CV visualisiert:

```

Runde 1: [VAL] [TRN] [TRN] [TRN] [TRN]
Runde 2: [TRN] [VAL] [TRN] [TRN] [TRN]
Runde 3: [TRN] [TRN] [VAL] [TRN] [TRN]
Runde 4: [TRN] [TRN] [TRN] [VAL] [TRN]
Runde 5: [TRN] [TRN] [TRN] [TRN] [VAL]
    
```

— jeder Punkt genau EINMAL Validierung —

★ Leave-One-Out CV (LOOCV)

Sonderfall $k = n$: exakt ein Datenpunkt in jedem Fold.

Aspekt	Bewertung
Kosten	⚠️ Kostspielig — erfordert n Modelle zu trainieren! (Aber: für einige Modelle gibt es Abkürzungen, die nur ein einziges Modelltraining erfordern)
Trainingsdaten-Variation	⚠️ Die Trainingsdaten werden in der Regel nicht ausreichend variiert (sind einander zu ähnlich)
Varianz	⚠️ Die Fehlerschätzungen für jedes Fold sind stärker korreliert , daher auch hohe Varianz
Bias	✅ Weniger Überschätzung des Testfehlers

★★★★ Der zweite Bias-Variance-Tradeoff (bei k !)

Jede Trainingsuntermenge ist nur $\frac{k-1}{k}$ so groß wie das Dev-Set → **Neigung zur Überschätzung des Testfehlers**

Wenn $k \uparrow$	Effekt
Trainingsdatensatz pro Fold wird größer	Bias sinkt ↓
Validierungsdatensatz wird kleiner	Varianz steigt ↑
Anzahl zu trainierender Modelle steigt	Rechenaufwand steigt ↑
Anzahl der Fehlerbeobachtungen steigt	✅ Vorteil

★ Praxisempfehlung (auswendig!)

Werte im Bereich $k = 5$ bis $k = 10$ bieten oft einen guten Kompromiss.

Aber: hängt von Daten und Modell ab:

- Bei **Millionen von Datenpunkten** und einem relativ **einfachen Modell** kann ein „gutes“ k wahrscheinlich **viel höher** angesetzt werden
- Bei **wenigen Datenpunkten** (< 100) und einem **komplexeren Modell** ist vielleicht sogar $k = 5$ bereits **zu groß**

📝 Übung 5.2 (Vollständige Musterantwort — Folgen einer k -Erhöhung):

- Mit der Zahl von Folds **steigt die Größe jedes einzelnen Trainingsdatensatzes**, die Größe jedes Validierungsdatensatzes **sinkt**
- Kleinere Trainingsdatensätze führen zu einer **Überschätzung des Testfehlers**; bei größerem k **sinkt diese Überschätzung**
- Gleichzeitig **steigt die Varianz**, da die Trainingsdatensätze zunehmend stark **korrelieren**, während die Validierungsdatensätze extrem klein werden
- Bei sehr großen k (bzw. LOOCV im Extremfall): **Gefahr, Overfitting nicht in der Auswertung zu erkennen**
- **Rechenaufwand steigt**
- **Vorteilhaft:** die größere Zahl von Folds erlaubt eine **größere Anzahl von Beobachtungen des Fehlers**

E.5 ★★ Bootstrap

Woher kommt der Name?

Idee: Sich selbst am Stiefelriemen „aus dem Sumpf ziehen“ — d. h. **Lösung eines Problems aus sich selbst heraus**. Ähnlich der Geschichte aus „*The Surprising Adventures of Baron Munchausen*“ von Rudolph Erich Raspe: **Baron Münchhausen zieht sich am eigenen Haar aus dem Sumpf**. ⚠ **Nicht dasselbe** wie der Begriff „Bootstrap“, der in der Informatik beim Systemstart („booten“) verwendet wird!

★ Das Verfahren (auswendig!)

1. Trainings-/Validierungsdatensatz zufällig ziehen (wie Hold-out)
2. **ABER MIT ZURÜCKLEGEN ziehen!**
3. So lange ziehen, bis man einen Datensatz hat, der **die gleiche Größe hat wie das komplette Dev-Set**
4. Wiederholen, so oft wie die Anzahl der benötigten Trainingsdatensätze

🔑 **Konsequenz:** Einige Beobachtungen können in einem gegebenen Bootstrap-Datensatz **mehrfach** vorkommen — und einige **überhaupt nicht**.

Eigenschaften

Eigenschaft	Bootstrap
Datenmengen	überschneidend
Trainingsdaten-Größe	genauso groß wie das gesamte Dev-Set
Bias	potentiell geringer

Andere Verwendungen von Bootstrap

- Häufig verwendet für die **Schätzung einfacher Statistiken** (nicht nur für ML-Modelle)
- **Ensemble-Modellierung** (Bagging, Random Forest) → siehe Teil F

Illustration mit $n = 3$

Jeder Bootstrap-Datensatz enthält $n = 3$ Punkte, **mit Zurücklegen** zufällig aus den Originaldaten gezogen. Jeder Bootstrap-Datensatz wird verwendet, um eine **Schätzung** α zu erhalten (z. B. einen Modellfehler).

 **Übung 5.6:** Warum enthält ein Bootstrap-Datensatz die gleiche Anzahl von Datenpunkten, aber nicht jeden Punkt?

Lösung: Bootstrap wählt n Punkte **zufällig, mit Zurücklegen**. Damit kann es passieren, dass einzelne Punkte **nicht** oder **mehrfach** im Bootstrap-Sample vorkommen.

 **Übung 5.7 (★ tiefergehend):** Was bedeutet ein mehrfach vorkommender Datenpunkt bei der linearen Regression?

Lösung: Taucht ein Datenpunkt mehrfach auf, so wird auch die zugehörige **quadratische Abweichung mehrfach beim Least-Squares-Verfahren berücksichtigt**. Ein mehrfach vorkommender Datenpunkt hat somit ein **höheres Gewicht** bzw. eine **größere Auswirkung** auf das lineare Regressionsmodell.

 **Übung 5.4 (Bootstrap zur Testfehlerschätzung):**

Auf jedem Bootstrap-Sample wird ein Modell trainiert. Um den Testfehler zu bestimmen, sollte jeweils ein **Validierungsdatensatz bestimmt werden, der alle Daten enthält, die NICHT im zugehörigen Bootstrap-Sample enthalten sind** (→ Out-of-Bag!). Der gesamte Testfehler kann dann durch **Mittelung der Fehler jedes Bootstrap-Samples** bestimmt werden.

 **Übung 5.8:** $E = (0.2, 0.3, 0.1, 0.4, 0.5) \rightarrow \hat{E}_{test} = \text{Mittelwert} = \frac{0.2+0.3+0.1+0.4+0.5}{5} = 0.3$

 **Übung 5.3 (Standardfehler der Vorhersage):**

Cross-Validation einsetzen: erst ein Modell **für jedes Fold** trainieren. Für einen neuen Punkt X^* die Vorhersage **jedes dieser k Modelle** bestimmen und dann die **Standardabweichung dieser Vorhersagen** bestimmen. Ähnlich kann auch Bootstrap zum Einsatz kommen.

E.6 ★★ CV vs. Bootstrap — Der Vergleich (Übung 5.5)

	k-fold CV	Bootstrap
Wie oft dient ein Punkt zur Fehlerbestimmung?	exakt einmal	mehrfach möglich
Größe des Trainingsdatensatzes	kleiner als der ursprüngliche Datensatz	genauso groß wie der ursprüngliche
Duplikate im Trainingsdatensatz	nein	ja, mehrfach möglich

E.7 !!! DIE GROSSE FALLE: Feature Selection außerhalb der Validierung

Das ist die wichtigste Warnung des ganzen Kapitels.

Das Szenario

1. Es liegen Daten mit **5000 Features**, **100 Datenpunkte** im Dev-Set vor
2. Suche nach den **100 Features**, die die **größte Korrelation** mit den Klassen-Labels aufweisen
3. Trainieren eines Klassifikators **nur mit diesen 100 Features**

? Können wir in Schritt 3 einfach Hold-Out, k-fold CV oder Bootstrap durchführen?

Das Experiment mit komplett zufälligen Daten

```
set.seed(1)
random_data <- as.data.frame(matrix(runif(5000*100),100)) # REINER ZUFALL
random_data$class <- sample(0:1, size=100, replace=T)      # ZUFÄLLIGE Klassen
cors <- apply(random_data[,1:5000], 2, cor, y=random_data$class)
ind <- order(abs(cors), decreasing=T)[1:100]                # ← FEHLER passiert HIER
random_data <- random_data[,c(ind,5001)]
train_control <- trainControl(method="cv", number=5)
model <- train(class ~ ., data=random_data, method="rf", trControl=train_control)
```

Ergebnis:

Random Forest – 100 samples, 100 predictors, 2 classes: '0','1'
Resampling: Cross-Validated (5 fold)

mtry	Accuracy	Kappa	
2	0.9704762	0.9405873	← 97 % Genauigkeit auf REINEM RAUSCHEN!
51	0.8483208	0.6936321	
100	0.8303258	0.6562379	

! Warum ist das schiefgegangen?

- Die Daten sind **rein zufällig erzeugt**, eine genaue Vorhersage ist also **unmöglich**¹
- Dennoch behauptet die Validierung, dass der Fehler **gering** ist!
- 🗝️ **URSACHE: Schritt 2 wird von der Validierungsmethode IGNORIERT — das Modell sieht aber bereits die Labels der Validierungsdaten!**
- **Die Entscheidung zur Feature Selection ist Teil des Trainings und muss in den Validierungsprozess einbezogen werden**
- Noch schlimmer, wenn nicht nur Validierungs-, sondern auch **Testdaten** in Schritt 2 „gesehen“ werden
- ⚠️ *Dieses Problem tritt häufig auf, z. B. auch in **hochkarätigen Papern zu Genomanalysen** (da sehr hochdimensional)*

¹ „Es sei denn, wir nehmen an, dass unser Modell komplex genug ist, um den Pseudo-Zufallszahlenstrom vorherzusagen. Extrem unwahrscheinlich.“

Realer Nachweis: Ambroise, C. & McLachlan, G. J. (2002): *Selection bias in gene extraction on the basis of microarray gene-expression data*. PNAS 99(10), 6562–6566. DOI: 10.1073/pnas.102102699

★ Warum tritt der Fehler so häufig auf? (4 Gründe)

1. **Fehlende Fachkenntnis** 🙄
2. **Flüchtigkeitsfehler** 😊
3. **Implementationen!** Softwareimplementationen **kombinieren häufig Modelltraining + Validierung in einem Schritt** — deutlich einfacher anzuwenden, **verleitet Nutzer aber dazu, ERST Feature Selection durchzuführen und DANACH Training + Validierung**
4. 🗝️ **„Ease-of-use kann zu Fehlern führen** 😞 — *saubere Validierung kann mehr Programmieraufwand bedeuten.*

★★ DIE REGEL (auswendig!)

🔴 **FALSCH:** k-fold CV **nur in Schritt 3** anwenden 🟢 **RICHTIG:** k-fold CV **gleichzeitig für Schritte 2 UND 3** anwenden

d. h.: **Features nur auf Trainingsdaten wählen** und diese Wahl auf Validierungsdaten (und später Testdaten) **anwenden**.

(Oder: Überspringe Schritt 2 ganz und verwende alle Features in Schritt 3, falls möglich.)

E.8 ★ Der Gesamtprozess — was am Ende passiert

Am Ende des Re-Sampling-/Validierungsprozesses:

- Wir haben **viele Modelle trainiert/validiert** → **Welches nehmen wir zum Testen?**
- 🗝️ **Soweit möglich: Training auf dem GESAMTEN Dev-Set! — Keine Daten an die Validierung verlieren!**
- **Erst dann:** Auswertung auf Testdaten!

Auch die Test-Evaluation ist vom Problem betroffen, dass wir nur **eine** Beobachtung des Fehlers erhalten.

- Streuung des Testfehlers bestimmen?
- Re-Sampling des Dev-Set/Test-Set-Splits? → **Unüblich. Kompliziert/aufwändig.**
- ✅ **Besser: Bootstrap des Testdatensatzes** (d. h. nur des Fehlers, nicht des Trainings)

E.9 ★★ TAKE-HOME-MESSAGE (Folie 37 — wörtlich!)

- Alle Resampling-Methoden „funktionieren“.
- Weniger wichtig ist, welche man wählt: Hold-out, CV, Bootstrap
- Wichtiger ist:
 1. Überhaupt eine der drei Methoden zu verwenden!
 2. Die Methoden RICHTIG anzuwenden!
- ⚠ **Achtung:** „Hilfreiche“ Methoden in Software-Implementationen können zu Fehlern verleiten!
- Bei komplexeren Datenstrukturen können Validierungsmethoden zusätzliche Überlegungen erfordern:
 - 📝 Z. B.: Wenn es sich bei den Daten um eine ZEITREIHE handelt, wird durch Samplen einzelner Beobachtungen die ZEITSTRUKTUR ZERSTÖRT.
 - 🔑 Lösung: Blockweise Daten ziehen.

E.10 ★ Das Auto-Daten-Experiment: Die drei Methoden im Vergleich

Setup: Welcher Grad d eignet sich am besten für ein lineares, polynomielles Modell der Auto-Daten?

- Laden, Trennung in Testdaten (60 %) und Dev-Set (40 %)
- $d \in \{1, 2, 3, 4, 5, 6, 7\}$
- 20–30 verschiedene Validierungs-/Trainingsdatenmengen aus dem Dev-Set
- Finales Modell auf gesamtem Dev-Set, Bewertung auf Testdaten

★★ DIE ERGEBNISTABELLE (auswendig!)

Methode	Validierungsfehler vs. Testfehler	Varianz des Validierungsfehlers
Hold-Out	überschätzt den Testfehler (insbesondere bei großen d)	extrem hoch
k-fold CV	unterschätzt den Testfehler	viel geringer als Hold-Out
Bootstrap	überschätzt, aber nicht ganz so extrem wie Hold-Out	geringer als Hold-Out

🔑 **Fazit des Dozenten:** „Zusammenfassend lässt sich sagen, dass **Bootstrap hier Ergebnisse liefert, die zwischen Hold-out und CV liegen.**“

R-Code-Kern der drei Varianten:

```
# HOLD-OUT
random_sel <- sample(1:nrow(nottest), nrow(nottest)*0.5)
train <- nottest[random_sel,]; validation <- nottest[-random_sel,]

# k-fold CV (k = 20)
sel <- seq(from=1, by=1, to=nrow(nottest)/k) + (i-1)*nrow(nottest)/k
train <- nottest[-sel,]; validation <- nottest[sel,]

# BOOTSTRAP
random_sel <- sample(1:nrow(nottest), nrow(nottest), replace=T) # <--- !!!
train <- nottest[random_sel,]; validation <- nottest[-random_sel,]
```

▼ ? Selbsttest Teil E (aufklappen)

1. Wozu dienen Trainings-, Validierungs- und Testdaten jeweils? (jeweils ein Satz)
2. Warum darf man Hyperparameter nicht auf den Testdaten wählen?
3. Was ist das „Dev-Set“ in der Terminologie des Dozenten?
4. Beschreiben Sie k-fold CV in 4 Schritten.
5. Was passiert mit Bias und Varianz, wenn k steigt? Warum?
6. Welchen Bereich für k empfiehlt der Dozent, und wann weicht man davon ab?
7. Was ist LOOCV? Nennen Sie 2 Nachteile.
8. Wie zieht man ein Bootstrap-Sample? Welche zwei Eigenschaften hat es?
9. Nennen Sie zwei Unterschiede zwischen CV und Bootstrap.
10. Beschreiben Sie das 5000-Features-Experiment und erklären Sie, warum 97 % Accuracy dort ein Artefakt ist.
11. Wie muss die Feature Selection korrekt validiert werden?
12. Welche Methode überschätzt, welche unterschätzt den Testfehler?
13. Womit trainiert man das FINALE Modell?
14. Wie validiert man Zeitreihendaten korrekt?

TEIL F — ENTSCHEIDUNGSBÄUME UND ENSEMBLES (ISL Kap. 8)

F.1 ★ Grundidee von Entscheidungsbäumen

- Baum-basierte Methoden für **Regression UND Klassifikation**
- **Aufteilung oder Segmentierung des Featureraums** in mehrere **rechteckige** Bereiche
- Die Segmentierung kann als **Baum** dargestellt werden

★★ Pros & Cons (Kurzfassung)

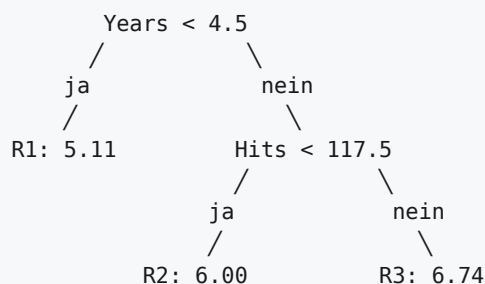
✓ Vorteile	✗ Nachteile
Einfach und gut zu erklären/interpretieren	Nicht sehr genau , im Vergleich zu anderen Modellen

🔑 * „Die Genauigkeit kann durch **Ensemble-Methoden** (Kombination vieler Bäume) erhöht werden. **Ensemble-Methoden verlieren an Interpretierbarkeit.**“*

F.2 ★★ Fachbegriffe (auswendig!)

Begriff	Definition
Blätter (leaves)	die Regionen $R_1, R_2, \dots, R_J$ — typischerweise unten gezeichnet (Baum steht auf dem Kopf!)
Interne Knoten / Verzweigungsknoten	Stellen, an denen der Feature Raum aufgeteilt wird
Wurzel (root)	der Startknoten des Baumes
Split	eine einzelne Aufteilung an einem internen Knoten

Baseball-Beispiel des Dozenten:



- Regressionsbaum, der das **Gehalt (Salary)** eines Baseballspielers vorhersagt
- basierend auf **Years** (gespielte Jahre) und **Hits** (Treffer im Vorjahr)
- **Zwei interne Knoten** ($\text{Years} < 4.5$, $\text{Hits} < 117.5$), **drei Blätter**
- Die **Zahl in jedem Blatt** ist der **Mittelwert des Labels** für die Beobachtungen, die dort landen

Interpretation:

- **Years** ist der wichtigste Faktor bei der Bestimmung von **Salary** — weniger Erfahrung bedeutet weniger Gehalt
- Für **weniger erfahrene** Spieler: Anzahl der Hits im Vorjahr **unwichtig** für Salary
- Für **erfahrene** Spieler (5+ Years): Hits im Vorjahr beeinflussen das Gehalt — **mehr Hits → mehr Gehalt**
-  „**Starke Vereinfachung** der Zusammenhänge, aber einfach darzustellen, zu interpretieren und zu erklären.“

F.3 ★★ Vorhersage und Baumbildung (Regression)

Vorhersage

1. Prüfen, in welche Region R_j ein neuer Datenpunkt fällt
2. **Vorhersage = Mittelwert der Trainingsdaten-Labels in R_j**

Form der Regionen

Theoretisch könnten Regionen jede Form haben. **Hier aber: immer mehrdimensionale Rechtecke, oder „Boxen“** — einfach und leicht zu interpretieren. **Ziel:** Boxen finden, die den Fehler minimieren — bei Regression: **RSS**.

★★ Recursive Binary Splitting (auswendig!)

Es ist **rechnerisch zu aufwändig**, jede mögliche Kombination von Splits zu prüfen bzw. die ideale Aufteilung zu bestimmen. **Kompromiss: top-down, „greedy“ Ansatz**, auch bekannt als **recursive binary splitting**.

Eigenschaften:

- Beginnt am Start des Baumes (**Wurzel / root**)
- Splittet den Feature Raum **schrittweise** auf
- Jeder Split ist eine weitere Verästelung nach unten
-  **Greedy-Split**: Auswahl des Splits, der den **RSS NUR IN DIESEM SCHRITT am meisten reduziert** — d. h. **keine Vorausschau auf folgende Splits**

Algorithmus:

1. Wähle ein Feature X_j und den zugehörigen Splitpunkt s , sodass der RSS durch den Split bei $X_j = s$ am stärksten reduziert wird
2. Wiederhole 1 für jede neu erzeugte Region, REKURSIV
3. Abbruch – z. B. wenn jedes Blatt nur noch 5 Trainingsdatenpunkte enthält

F.4 ★ Pruning (Zurückschneiden)

Warum?

 **Zu große Bäume können problematisch sein:**

1. **Overfitting**
2. **Schwerer zu interpretieren**

Zwei Lösungsmöglichkeiten

Lösung	Beschreibung
Baumgröße begrenzen	durch einen Parameter (Maximale Tiefe, Anzahl Datenpunkte pro Blatt, ...)
Pruning	1. Zuerst einen vollständigen Baum erstellen (ein Datenpunkt pro Blatt, maximale Baumgröße) 2. Dann gezielt Zweige abschneiden (Blätter entfernen), auf eine Weise, die einen guten Kompromiss zwischen Komplexität (Größe) und Qualität (Fehler) ergibt

„Guter Ansatz, kann aber **zeitaufwendig** sein. Wir werden dies nicht im Detail behandeln. Siehe ISL-Buch bei Interesse.“

F.5 ★★★★★ Klassifikationsbäume und Split-Kriterien

Unterschiede zu Regressionsbäumen

- Diskrete/kategorische Antwort (qualitativ)
- **Vorhersage: häufigste beobachtete Klasse** der Trainingsdaten in einer Region — auch bekannt als der „**Modus**“ der Klasse im Blatt
- Gleicher Erstellungsprozess: **recursive binary splitting**
- ⚠ **ABER: RSS kann nicht als Split-Kriterium verwendet werden!**

Die drei Maße

1. Fehlerrate (Classification Error Rate)

$$E = 1 - \max_k (\hat{p}_{mk})$$

mit $\hat{p}_{mk}$ = Anteil der Trainingsbeobachtungen in R_m , die der k -ten Klasse angehören.

⚠ **Problem: E reagiert NICHT AUSREICHEND SENSITIV auf Änderungen der Splits!**

2. ★ Gini-Index (Formelsammlung!)

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk}) = 1 - \sum_{k=1}^K \hat{p}_{mk}^2$$

Eigenschaft

Maß für die **Gesamtvarianz** über die K Klassen

G ist **niedrig**, wenn alle $\hat{p}_{mk}$ -Werte **nahe bei Null oder Eins** liegen

🔑 G ist ein Maß für die **REINHEIT eines Knotens** — kleines G bedeutet, dass ein Knoten **hauptsächlich Beobachtungen einer einzigen Klasse** enthält

Verwendung: Um einen Split zu bewerten, wird G **für jeden der beiden neuen Knoten** berechnet und dann ein **nach Anzahl der Beobachtungen gewichtetes Mittel** gebildet.

3. Cross-Entropy D

Alternative zu G . **Hinweis: G und die Cross-Entropy liefern sehr ähnliche Ergebnisse.**

📝★★★★ Das Sensitivitäts-Beispiel (Folie 20) — Klausurklassiker!

10 Datenpunkte: **6 mit Klasse A, 4 mit Klasse B**, ein Feature x . Zwei mögliche Splits. **Die Fehlerrate bewertet beide Splits GLEICH (jeweils 4 Fehler). G bewertet den linken Split BESSER als den rechten!**

Rechenbeispiel zum Nachvollziehen:

Split	Knoten links	Knoten rechts	E	Gewichteter Gini
A	3A, 0B (n=3)	3A, 4B (n=7)	$(0 + 3)/10 = 0.3$	$\frac{3}{10} \cdot 0 + \frac{7}{10} \cdot \left(1 - \left(\frac{3}{7}\right)^2 - \left(\frac{4}{7}\right)^2\right) = 0.7 \cdot 0.4898 = \mathbf{0.343}$
B	4A, 2B (n=6)	2A, 2B (n=4)	$(2 + 2)/10 = 0.4$	$\frac{6}{10} \cdot 0.4444 + \frac{4}{10} \cdot 0.5 = 0.2667 + 0.2 = \mathbf{0.467}$

🔑 **Merksatz:** G und D sind zu bevorzugen, da E nicht sensitiv genug ist (z. B. nicht sensitiv auf kleine Änderungen des Split-Punktes).

F.6 ★★ Vorteile und Nachteile von Bäumen (auswendig!)

	Aussage
✓	Bäume sind für Menschen sehr einfach zu erklären/interpretieren — sogar noch einfacher als die lineare Regression!
✓	Bäume können sehr gut grafisch dargestellt werden
✓	Bäume können sehr einfach auch kategorische und gemischte Features verarbeiten — OHNE Dummy-Variablen
✓	<i>Behauptung:</i> Entscheidungsbäume spiegeln die menschliche Entscheidungsfindung wider
✗	Leider meist eine vergleichsweise geringe Vorhersagegenauigkeit

🔑 „Durch **Aggregation vieler Entscheidungsbäume** kann die Vorhersagegenauigkeit erheblich verbessert werden. **Hinweis:** Wir gewinnen an Vorhersagegenauigkeit, verlieren aber an Interpretierbarkeit. 😊“

Bäume vs. lineare Modelle

Wahre Abgrenzung	Lineares Modell	Entscheidungsbaum
linear	✓ passt gut	✗ treppenförmige Approximation
nicht-linear	✗ schlecht	✓ passt gut

F.7 ★ Ensemble Models — Überblick

Methoden, die einzelne Modelle zu „größeren“ Modellen zusammenfassen. Zweck: die Leistung der einzelnen Modelle verbessern, indem sie **die Schwächen der einzelnen Modelle ausgleichen**.

Beliebte Ensemble-Methoden:

1. **Bagging**
2. **Random Forest**
3. **Boosting**
4. **Stacking**

F.8 ★★ Bagging (Bootstrap Aggregating)

Definition

Bagging ist eine Ensemble-Methode — ein **allgemein anwendbares Verfahren zur Verringerung der VARIANZ** einer statistischen Lernmethode. Häufig für Entscheidungsbäume verwendet (v. a. Random Forests), aber **nicht ausschließlich**.

★ Die Motivation: Mittelwertbildung verringert Varianz

Allgemeine Beobachtung: Mittelwertbildung über Gruppen von Beobachtungen **verringert Varianz!**

📝 **Das Zahlenexperiment des Dozenten:**


```
means_single_sample <- rnorm(100, 42, 1) # 1 Beobachtung je Schätzung
means_multiple_sample <- simulate_multiple_samples(100, 100) # 100 Beobachtungen je Schätzung
```

	Varianz	Mittelwert
Eine Beobachtung	1.084424	42.03251
Mehrere Beobachtungen (n=100)	0.01268246	41.9903

🔑 Die Varianz sinkt um Faktor ~85!

Das Problem und seine Lösung

Wir könnten einen **Durchschnitt über eine Reihe von Modellen** bilden, die auf **mehreren Datensätzen** basieren → verringert die Varianz der Modellleistung → **erhöht die Genauigkeit / reduziert den Fehler**.

⚠️ Das ist nicht praktikabel: normalerweise sind nicht mehrere Trainingsdatensätze verfügbar.

✅ Wie lösen wir das? → **BOOTSTRAPPING!**

★★ Der Bagging-Algorithmus

1. Erzeugen von B **verschiedenen Bootstrap-Trainingsdatensätzen**
2. Modelle mit **jedem** der B Bootstrap-Trainingsdatensätze trainieren
3. Vorhersage am Punkt x für jedes Modell: $\hat{f}^{*i}(x)$
4. **Durchschnitt aller Vorhersagen der Bäume:**

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{i=1}^B \hat{f}^{*i}(x)$$

★ Bagging von KLASSIFIKATIONSbäumen

⚠️ Die Berechnung des Mittelwerts funktioniert **nur bei Regression** — der **Mittelwert von Klassen-Labels ist nicht definiert**.

Stattdessen:

1. **Mehrheitsabstimmung** — die Gesamtvorhersage ist die **häufigste Vorhersage** unter den B Klassifikatoren
2. **Alternative:** Berechnung des **Mittelwerts der Klassenhäufigkeiten** im jeweiligen Blatt jedes Baums

✍️ ★★★ Übung 8.8 — Mehrheit vs. Durchschnitt (KLASSIKER!)

10 Bootstrap-Bäume liefern für X^* die Wahrscheinlichkeiten $P(\text{rot} \mid X^*)$: **0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, 0.75**

Ansatz 1 — Mehrheitsentscheidung: Zähle, wie viele Bäume „rot“ vorhersagen ($p > 0.5$): **0.55, 0.6, 0.6, 0.65, 0.7, 0.75** → **6 Bäume** vs. 4 Bäume „grün“ → $C(x) = \text{ROT}$

Ansatz 2 — durchschnittliche Wahrscheinlichkeit:

$$\bar{p} = \frac{0.1 + 0.15 + 0.2 + 0.2 + 0.55 + 0.6 + 0.6 + 0.65 + 0.7 + 0.75}{10} = \frac{4.5}{10} = 0.45$$

→ $0.45 \leq 0.5 \rightarrow C(x) = \text{GRÜN}$

🚨 Die beiden Ansätze können zu **UNTERSCHIEDLICHEN** Ergebnissen kommen — das ist genau der Punkt der Aufgabe!

☆☆ Out-of-Bag (OOB) Schätzung

Unkomplizierter Weg zur Schätzung des Testfehlers eines Bagging-Modells:

- Jeder Baum verwendet etwa 2/3 der Trainingsdaten
- Das verbleibende 1/3 wird nicht für das Training des Baums verwendet → Out-of-Bag (OOB) Beobachtungen
- Vorhersage für jede Beobachtung erzeugen, **unter Verwendung nur der Bäume, bei denen diese Beobachtung OOB war**
- Dies ergibt **etwa $B/3$ Vorhersagen für jede Beobachtung**, die wir mitteln

(Zur Erinnerung: B ist die Anzahl der Bäume. Mathematisch: $\lim_{n \rightarrow \infty} (1 - 1/n)^n = e^{-1} \approx 0.368$, daher ~37 % OOB.)

📝 R-Beispiel Baseball:

```
bag_of_trees <- bagging(formula = Salary ~ ., data = data_train,
                        nbagg = 200, # Anzahl Bootstrap-Samples
                        coob = TRUE) # OOB-Fehlerschätzung
# Out-of-bag estimate of root mean squared error: 300.6547
```

F.9 ☆☆☆ Random Forest (Klausuraufgabe 9!)

Definition

Random Forests bedeutet Bagging mit Bäumen — mit einer kleinen, aber wichtigen Verbesserung.

🚨 **Ziel: DEKORRELATION der Bäume (und ihrer Vorhersagen)**

- Die Dekorrelation **verringert die Varianz**, wenn wir die Bäume mitteln
- **Geringere Varianz bedeutet bessere Vorhersagegenauigkeit**

☆ Demonstration: Korrelation erhöht die Varianz

„Vielleicht nicht ganz intuitiv, dass Korrelation die Varianz erhöht, aber an einem Beispiel demonstrierbar.“

Fall	Varianz	Mittelwert	mittlere Korrelation
Eine Beobachtung	1.084424	42.03251	—
100 unabhängige Beobachtungen ($\rho = 0$)	0.009708757	41.98939	-0.00028
100 korrelierte Beobachtungen ($\rho = 0.5$)	0.4780572	42.01183	0.48294

🚨 Die Varianz ist bei korrelierten Beobachtungen ~49× höher!

★★ Das Random-Forest-Verfahren (auswendig!)

Wie beim Bagging werden Entscheidungsbäume auf Basis von Bootstrap-Datensätzen erstellt. ABER, beim Aufbau jedes Baumes:

- Jedes Mal, wenn ein Split in einem Baum betrachtet wird:
- Zufällige Auswahl einer Teilmenge von m Features aus der gesamten Menge von p Features
 - Der Split darf NUR die gewählten m Features verwenden
 - Bei JEDEM WEITEREN Split wird eine NEUE Auswahl von m Features getroffen

Typischerweise: $m \approx \sqrt{p}$ (kann aber „getuned“ werden)

„Es gibt aber auch andere Möglichkeiten, Modelle zu dekorrelieren, z. B. durch **Manipulation der Trainingsdaten**. Die obige Methode funktioniert jedoch sehr gut für Bäume.“

📝★★ DIE KLAUSUR-MUSTERANTWORT (Aufgabe 9, 4 Punkte) — auswendig!

Frage: Welchem Zweck dient diese zufällige, reduzierte Auswahl von Features?

Musterlösung: Ohne diesen Trick werden sich die erzeugten Bäume des Ensembles alle sehr ähnlich sein bzw. korrelieren. Das erhöht die Varianz bzw. den Fehler des Modells. Mit der reduzierten Auswahl werden die Bäume gezwungen, die Trainingsdaten mit anderen Features abzubilden, somit dekorreliert, was den Fehler senken soll.

📝 Vergleich der drei Modelle (Baseball-Daten, RMSE auf Testdaten)

Modell	Testfehler (RMSE)	OOB-Fehler
Einzelner Baum (<code>rpart</code>)	341.6251	—
Bagging (<code>ipred::bagging</code> , 200 Bäume)	308.0267	300.6547
Random Forest (<code>ranger</code> , 400 Bäume)	297.9347	294.684

🔑 **Klare Reihenfolge: Baum > Bagging > Random Forest** (kleinerer Fehler = besser).

★ Feature-Wichtigkeit (Variable Importance)

Wichtigkeit: Summierung der Fehler-Abnahme, verursacht durch Splits über ein bestimmtes Feature, gemittelt über alle B Bäume. Großer Wert → wichtiges Feature.

```
forest <- ranger::ranger(Salary ~ ., data=data_train,
                        num.trees=300, importance="impurity")
barplot(sort(forest$variable.importance), horiz=T, las=1)
```

★★★ Parameter des RF-Modells

Parameter	Bedeutung	Empfehlung
B	Anzahl der Bäume im Ensemble	Mehr ist in der Regel besser (bei Standard-RF). Aber: größeres B erhöht die Rechenkosten. B liegt oft im Bereich von Hundertern oder Tausenden .
m	Anzahl der bei jedem Split berücksichtigten Features	Normalerweise $m = \sqrt{p}$ oder $m = \lfloor \sqrt{p} \rfloor$. Steuert die Dekorrelation von Bäumen.
Baumgrößen-Parameter	Anzahl der Splits, Anzahl der Blätter, Anzahl der Proben je Blatt	—
Bootstrap-Details	Anteil gezogener Datenpunkte; mit oder ohne Zurücklegen	—

🔑 **Wichtige Merkaussage:** „Wenn keine extrem ungewöhnlichen Werte gewählt werden, ist **Random Forest in der Regel SEHR UNEMPFINDLICH gegenüber Parameteränderungen**“ — d. h. kleine Änderungen der Parameter haben kaum Auswirkungen auf die Leistung.

📝 **Experiment (Folie 44):** m von 1 bis 10 und 19 durchgetestet. „**19**“ ist gleichbedeutend mit **einfachem Bagging** (keine Einschränkung der Split-Features, da die Daten nur 19 Features enthalten).

F.10 ★★★ Boosting (Klausuraufgabe 8!)

Grundprinzip

	Bagging	Boosting
Baumerzeugung	Jeder Baum wird unabhängig von den anderen erstellt, parallel	sequenziell und NICHT unabhängig — jeder neue Baum wird unter Verwendung der Informationen des vorherigen Baums erstellt
Parallelisierbar?	✅ ja	❌ nein (beim Training)

📝 **Übung 8.10 (Musterantwort):** Welches Modell profitiert eher von Parallelisierung?

Random Forest, da **jeder Baum unabhängig** von allen anderen trainiert werden kann. Bei Boosting erfolgt das Training **sequenziell**, jeder Baum ist abhängig von den Ergebnissen der vorherigen Bäume. ⚠️ **Bei der VORHERSAGE ergibt sich allerdings KEIN Unterschied**, da beide Ensembles eine **Summe einzelner Modelle** erzeugen.

★★★ Der Boosting-Algorithmus für Regressionsbäume (AUSWENDIG!)

- Setze $\hat{f}(x) = 0$ und $r_i = y_i$ für alle i in den Trainingsdaten
- Für $b = 1, 2, \dots, B$ wiederhole:
 - Trainiere Baum $\hat{f}^b$ mit d Splits auf den Trainingsdaten (X, r)
 - Aktualisiere $\hat{f}$ durch Hinzufügen einer REDUZIERTEN Version des Baums:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \cdot \hat{f}^b(x)$$
 - Aktualisiere die Residuen:

$$r_i \leftarrow r_i - \lambda \cdot \hat{f}^b(x_i)$$
- Ausgabe des finalen Modells:

$$\hat{f}(x) = \sum_{b=1}^B \lambda \cdot \hat{f}^b(x)$$

★★ Die Idee dahinter (auswendig!)

- **Vermeiden von Overfitting durch LANGSAMES LERNEN** (Lernen impliziert hier eine schrittweise Reduzierung des Fehlers)
- Basierend auf dem aktuellen Modell: einen neuen Entscheidungsbaum **an die RESIDUEN anpassen**
- Wird dieser Baum auf das Modell addiert, **reduziert das die Residuen**
- **Jeder einzelne Baum kann recht flach sein** (wenige Knoten), da er durch folgende Bäume **korrigiert** wird
- Die Baumtiefe wird gesteuert durch den Parameter d
- Anpassung **kleiner Bäume an die Residuen** → **langsame Reduzierung der Fehler**
- Der Lernparameter λ **verlangsamt den Prozess**

Boosting für Klassifikation

Ähnelt prinzipiell dem Boosting für Regression, ist aber **komplexer**. Wird weder hier noch im **ISL-Buch näher erläutert**. Siehe „*Elements of Statistical Learning*“, Kapitel 10 (dieselben Autoren).

Implementierungen:

- R-Paket **gbm** (gradient boosted models) — Regression und Klassifikation
- **xgboost** — häufig verwendet, um anspruchsvolle ML-Aufgaben zu lösen; 🔑 **immer noch state-of-the-art bei klassischen, tabellarischen Daten**; auch in Python verfügbar

★★★★ Tuning-Parameter für Boosting (KLAUSURKERN!)

Parameter	Bedeutung	⚠️ Wichtige Aussagen
B	Anzahl der Bäume im Ensemble	⚠️⚠️ Im Gegensatz zu Bagging und Random Forests kann Boosting bei ZU GROSSEM B deutlich OVERFITTEN!
λ (eta)	Lernparameter / shrinking parameter — kleine positive Zahl, steuert die Schrittgröße	Typische Werte: 0.01 oder 0.001 , problemabhängig. ⚠️ Der Standardwert in xgboost (R) ist 0.3 — VIEL ZU GROSS. 🔑 Sehr kleines λ erfordert viel größeres B!
d (max_depth)	Tiefe / Anzahl der Splits in jedem Baum	Steuert die Komplexität der Einzelmodelle . Oft funktioniert ein kleiner Wert (≤ 10) erstaunlich gut. 🔑 d steuert die INTERAKTIONSTIEFE des Modells, da d Splits höchstens d Variablen beinhalten.

⚠️ **Warnung des Dozenten:** „Die **Standardparameter** dieser Implementierung [xgboost] sind **sehr, sehr schlecht**. Insbesondere: `nrounds`, `max_depth` und `eta`.“ „xgboost ist eines der am stärksten **tune-baren** Modelle, zum Teil weil es manchmal schlechte Defaultparameter verwendet.“

★★★★ DIE KLAUSUR-MUSTERANTWORT (Aufgabe 8, 2 Punkte) — auswendig!

Frage: Ihr Boosting-Modell zeigt starkes Overfitting. Würden Sie λ erhöhen oder verringern?

Musterlösung: **VERRINGERN**. Kleinere λ -Werte führen dazu, dass das Modell in **jedem Boosting-Schritt weniger stark an die Daten angepasst** wird, also **langsamer lernt** — wirkt somit **Overfitting entgegen**.

✎ ★ Übung 8.14 — Boosting-Vorhersage berechnen

Zwei Bäume eines 2-Baum-Boosting-Ensembles, $\lambda = 0.5$. Beobachtung $(1, 3)$. Baumvorhersagen:
 $f_1((1, 3)) = 0$ und $f_2((1, 3)) = -2$

$$f_{\text{boosted}}(\vec{x}) = \sum_{i=1}^B \lambda f_i(\vec{x}) = 0.5 \cdot 0 + 0.5 \cdot (-2) = -1$$

🔑 **Rezept:** (1) Jeden Baum einzeln ablaufen, (2) alle Vorhersagen **mit λ multiplizieren**, (3) **aufsummieren**.

F.11 ★ Zusammenfassung Bäume & Ensembles (Folie 56)

- **Entscheidungsbäume** sind einfache und interpretierbare Modelle für Regression und Klassifikation — oft aber mit **relativ schlechter Vorhersagegenauigkeit**
- **Bagging, Random Forests und Boosting** sind gute Methoden zur Verbesserung der Genauigkeit — durch das geschickte Verknüpfen vieler Bäume (Ensembles), die Vorhersagen kombinieren
- ⚠️ **ABER: Nicht mehr so gut interpretierbar!**
- 🔑 **Random Forest und Boosting gelten für viele ML-Probleme immer noch als STAND DER TECHNIK**

F.12 ✎ ★ Übungsaufgaben zu Bäumen (mit vollständigen Lösungen)

✎ Übung 8.1 — Feature Raum-Aufteilung zeichnen

- (a) Graph, der den 2D-Feature Raum in **6 Bereiche** aufteilt. (b) Der entsprechende Entscheidungsbaum, inkl. **Wurzel, Blätter mit Werten, Entscheidungsungleichungen mit Werten**.
 (c) *Wie viele Datenpunkte müssten mindestens vorliegen?* → **6 Datenpunkte** (mindestens 1 je Blatt)

✎ Übung 8.2 — Supervised oder Unsupervised?

Entscheidungsbäume sind **üblicherweise supervised learning**, da zur Findung der Entscheidungsgrenzen die **Zielvariable (Label)** genutzt wird. Allerdings könnten theoretisch auch Entscheidungen anhand eines anderen Maßes getroffen werden, welches **nur die Verteilung der Featurewerte X** in jeder Region bewertet.

⚠️ ✎ Übung 8.7 — DIE FANGFRAGE

„Zeichnen Sie den zugehörigen Entscheidungsbaum zu folgender Aufteilung im Feature-Raum.“

Lösung: NICHT MÖGLICH, da die Aufteilung **keinem Entscheidungsbaum entspricht**. Durch **Aufteilung anhand einzelner Features** können die Regionen in der Grafik **nicht erzeugt** werden.

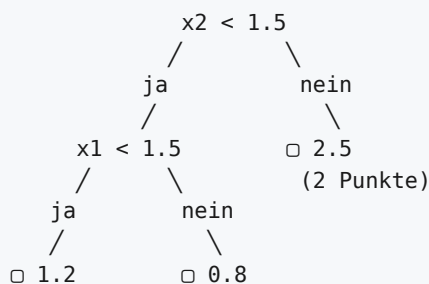
🔑 **Merkregel:** Eine Aufteilung ist nur dann baumfähig, wenn sie durch **rekursive achsenparallele Zweiteilungen** entstehen kann. Ein „Kreuz“-Muster oder ein T-Stück in der Mitte, das nicht durch einen Schnitt über die volle Breite/Höhe der Elternregion entsteht, ist **nicht** darstellbar.

★ ★ ★ Übung 8.13 — Baum von Hand konstruieren (SEHR klausurrelevant!)

Daten (x_1, x_2, y) : $(1.0, 1.0, 1.2)$, $(1.0, 2.0, 2.4)$, $(2.0, 1.0, 0.8)$, $(2.0, 2.0, 2.6)$ Wie würde ein Regressionsbaum diese Punkte in genau 3 Regionen aufteilen? Warum?

Lösungsgang:

- Erster Split:** Die größtmögliche Reduzierung des RSS ergibt sich durch $x_2 < 1.5$ (Prüfung: bei $x_2 < 1.5$ sind die y -Werte 1.2 und 0.8 (nah beieinander), bei $x_2 \geq 1.5$ die Werte 2.4 und 2.6 (nah beieinander) → sehr reine Gruppen)
- Zweiter Split:** Nur noch eine weitere Region erlaubt. Die **stärkste Reduzierung ergibt sich in der Region mit der HÖHEREN VARIANZ** — also der unteren Region ($x_2 < 1.5$; Werte 1.2 und 0.8, Spannweite 0.4 vs. 2.4/2.6, Spannweite 0.2). Also $x_1 < 1.5$ in der unteren Region.
- Vorhersagen:** jeweils die y_i -Werte der einzelnen Datenpunkte (wo nur eine Beobachtung im Blatt landet), bzw. der **Mittelwert** in der größten Region: $(2.4 + 2.6)/2 = 2.5$



Übung 8.9 — Einzelbaum vs. Random Forest

(a) Einzelner Baum: Einfach zu erklären/verstehen/Regeln erstellen, grafische Darstellung, einfache/schnelle Berechnung **(b) Random Forest:** Genauere Vorhersagen möglich (dafür: keine Interpretierbarkeit, höherer Rechenaufwand)

Übung 8.11 — Zusammenhang Bagging ↔ Random Forest

Bagging beschreibt, wie mittels **Bootstrap Aggregation** ein Modell-Ensemble erzeugt werden kann. **Random Forest baut auf Bagging auf, speziell für Entscheidungsbäume.** Dabei wird für jeden Entscheidungsbaum die Zahl der Features auf eine **Untermenge** reduziert, um so die Bäume zu **dekorrelieren**.

F.13 Klausuraufgabe 7 — Einen Baum ablaufen

Aufgabe: Bestimmen Sie die Vorhersage für $dis = 1$, $nox = 0.6$, $crim = 1$, $tax = 300$. **Dokumentieren Sie Ihr Vorgehen.**

Musterlösung (Format der Dokumentation genau so übernehmen!):

```
nox ist NICHT >= 0.67      → 13.6 (rechts)
tax ist >= 268             → (links)
nox ist >= 0.514           → (links)
dis ist NICHT >= 1.38      → (rechts)
Blatt: Vorhersage 35.3
```

 **Klausurtechnik:** Jeden Knoten einzeln aufschreiben, die Bedingung prüfen und explizit „ja/nein → links/rechts“ notieren. Die Punkte gibt es für die **Dokumentation**, nicht nur für die Zahl!

F.14 R-Praxis: xgboost (Lab 7)

```
require(xgboost)
data <- read.csv("https://www.statlearning.com/s/Auto.csv")[,1:7]
set.seed(42)
random_sel <- sample(1:nrow(data), nrow(data)*0.4)
label <- which(names(data)=="mpg")

data_trainp <- data.matrix(data[random_sel,-label])
data_testp <- data.matrix(data[-random_sel,-label])
y_trainp <- data[random_sel,label]; y_testp <- data[-random_sel,label]

boosted_trees <- xgboost(data=data_trainp, label=y_trainp,
                        objective="reg:squarederror",
                        nrounds = 2,           # B – VIEL ZU KLEIN!
                        verbose = 0)

pred_boost <- predict(boosted_trees, newdata=data_testp)
sqrt(mean((pred_boost - y_testp)^2)) # → 4.892089
```

 **Preprocessing-Hinweis:** „*xgb needs preprocessing: converting categorical to numerical features*“ — kategoriale Features müssen mit `model.matrix()` zu **Dummies** umgewandelt werden (im Gegensatz zu einfachen Bäumen!).

▼ ? Selbsttest Teil F (aufklappen)

1. Was sind Blätter, interne Knoten und die Wurzel eines Baumes?
2. Wie lautet die Vorhersage in einem Regressionsblatt? Und in einem Klassifikationsblatt?
3. Was bedeutet „greedy“ bei recursive binary splitting?
4. Warum kann RSS bei Klassifikationsbäumen nicht als Split-Kriterium verwendet werden?
5. Berechnen Sie G für einen Knoten mit 5A und 5B. Und für 9A und 1B. (0.5 bzw. 0.18)
6. Warum ist E als Split-Kriterium schlechter als G ?
7. Nennen Sie 4 Vorteile und 1 Nachteil von Entscheidungsbäumen.
8. Wie funktioniert Bagging in 4 Schritten?
9. Was ist eine OOB-Beobachtung? Welcher Anteil ist typischerweise OOB?
10. Wie unterscheidet sich Random Forest von Bagging? Wozu dient der Unterschied?
11. Wie groß wählt man m typischerweise?
12. Schreiben Sie den Boosting-Algorithmus komplett auf.
13. Boosting overfittet. Was tun Sie mit λ ? Was mit B ?
14. Wie hängen λ und B zusammen?
15. Was steuert d beim Boosting neben der Komplexität noch?
16. Warum ist Random Forest parallelisierbar, Boosting aber nicht?
17. 10 Bäume liefern $P(\text{rot}) = 0.4, 0.45, 0.48, 0.49, 0.9, 0.9, 0.9, 0.9, 0.9, 0.9$. Mehrheit vs. Durchschnitt?

TEIL G — NEURONALE NETZE / DEEP LEARNING (ISL Kap. 10.1, 10.2, 10.7)

Quellen des Dozenten für dieses Kapitel: teilweise basierend auf dem Deep-Learning-Kurs von **Charles Ollion und Olivier Grisel**, einzelne Materialien aus dem **NVIDIA Teaching Kit**, sowie eigene Ergänzungen. Relevante ISL-Kapitel: **10.1, 10.2, 10.7**.

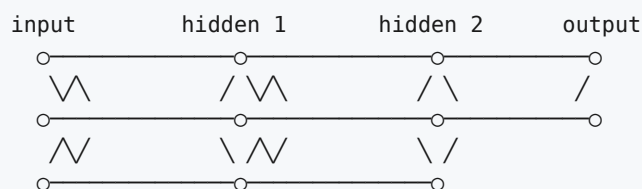
G.1 ★ Was sind künstliche Neuronen und Netze?

Neuron in einem Satz

Ein Vektor $\vec{x}$ geht rein → irgendwas wird berechnet → eine Zahl y kommt raus. Es ist das **grundlegende Rechenmodul** im Neuronalen Netz (NN).

Neuronale Netze

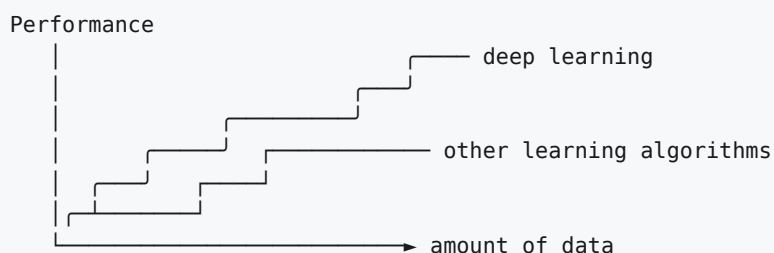
- **Graphen bzw. Netze aus vielen Neuronen**
- Häufig (aber nicht immer) in „**Schichten**“ (**Layers**) angeordnet
- Auch genannt: **Multi-Layer Perceptron (MLP)**
- Alle Neuronen sind mit **jedem** Neuron der nächsten Schicht verbunden
- Solche Schichten heißen **dicht / dense** („vollständig verbunden“)



★ Warum aktuell so erfolgreich? (4 Gründe, auswendig!)

1. **Besseres Verständnis von Algorithmen/Modellen** 🧑🧠🧑
2. **Rechenkapazität** (GPUs, TPUs, ...)
3. **Verfügbarkeit von (gelabelten) Daten**
4. **Open-Source Tools und Modelle**

🔑 **Die klassische Kurve (nach Andrew Ng):** Klassische Lernalgorithmen erreichen ab einer gewissen Datenmenge ein **Plateau**, während **Deep Learning weiter skaliert**.



G.2 Moderne Anwendungen (aus den Folien)

Bereich	Beispiele
Sprache	Sprache → Sprache (Übersetzung, TTS/STT)
Large Language Models	ChatGPT, Gemini, Claude · Suchmaschinen → Antwortmaschinen · perplexity.ai, elicitor.com · Agentensysteme
Bildverarbeitung	Sprache/Bild → Label/Sprache
Bildgenerierung	Stable Diffusion (GUIs: comfyUI, SD-webui), DALL·E 3
Video	WAN, Qwen, Flux, LTX, Gen-2/runwayml — „inzwischen ziemlich gut, auch zunehmend open source“
Genomik	AlphaFold (DeepMind) — 3D-Proteinstrukturen vorhersagen
Chemie	Gómez-Bombarelli et al. (2018): Automatic Chemical Design — Eigenschaften unbekannter Moleküle
Physik	Tompson et al. (2016): Accelerating Eulerian Fluid Simulation With Convolutional Networks

G.3 ★★ Grenzen und Herausforderungen (Übungsaufgabe 10.3!)

Grenzen: Qualität

KI macht Fehler, z. B.:

- Bilder zeigen **andere Inhalte** als gewünscht/behauptet
- Elemente in Bildern sind **nicht realistisch** (verzerrte Architektur, verformte Hände/Augen)
- **Wichtige Fakten** werden in Texten nicht erwähnt
- **Fakten werden behauptet, sind aber fiktiv** (Halluzination)
- Die KI **versteht die Eingaben nicht vollständig**
- Erstellter Code ist **nicht ausführbar**

🔑 → Die KI übernimmt (noch?) nicht das gesamte Denken für uns.

Weitere Grenzen

Kategorie	Problematic
Datenschutz	Anbieter (z. B. OpenAI/ChatGPT) verwenden Benutzereingaben/Daten wieder → potenziell ungeeignet für vertrauliche Daten
Demokratisierung / Open Source	Freier Zugang für alle? Generative KI nicht nur für einige wenige Großkonzerne
Herkunft der Daten	Woher stammen die Trainingsdaten? Urheberrechte?
Missbrauch	Bias bei generierten Objekten; Betrug, Fake News, Zensur
Ressourcen	Rechenaufwand, Hardware-Anforderungen, Stromverbrauch

🔑 → Konflikte zwischen Problembereichen. Keine einfachen Lösungen!

📝 Übung 10.3 — Musterantwort (3 Herausforderungen)

- **Ethisch/Juristisch: Urheberrecht** — welchen Beschränkungen unterliegen die verwendeten Trainingsdaten (z. B. Bilder für generative txt2img-Modelle)? Wem gehören die Daten? Treffen Fair-Use-Bedingungen zu? Problematische Trennung zwischen **non-profit und for-profit** Startups, nationale Unterschiede.
- **Technisch: Hohe Hardwareanforderungen**, Demokratisierung der AI — kann jeder selbst ein Modell laufen lassen, zumindest für Inference? **Welche Ressourcen verbraucht KI?**
- **Praktisch: Schwachstellen** z. B. von Textmodellen (ChatGPT generiert manchmal fehlerhafte Aussagen).

📝 Übung 10.4 — Anwendungsbereiche in der Wissenschaft:

- **Chemie:** Eigenschaften von unbekannten Molekülen bestimmen
- **Physik:** Simulation von physikalischen Systemen beschleunigen
- **Biologie:** 3D-Proteinstrukturen vorhersagen

G.4 ★★★★★ Das künstliche Neuron (Formel auswendig!)

$$z(\vec{x}) = \vec{w}^T \vec{x} + b \quad f(\vec{x}) = g(z(\vec{x})) = g(\vec{w}^T \vec{x} + b)$$

Symbol	Bedeutung	Biologische Analogie
$\vec{x}$	Eingabe	~ Dendriten
$f(\vec{x})$	Ausgabe	~ Axon
$z(\vec{x})$	Vor-Aktivierung (linearer Operator)	—
$\vec{w}, b$	Gewichte und Bias	Synapsenstärke
$g(\cdot)$	Aktivierungsfunktion (nichtlinearer Operator)	Feuerschwelle

⚠ **Hinweis:** Es gibt auch **andere Arten** künstlicher Neuronen (z. B. **Spiking Neurons, Quantum Neurons**), aber in der Regel wird die oben beschriebene Art verwendet.

📝 Übung 10.5 (Musterantwort — biologisch vs. künstlich):

Natürliche Neuronen übertragen Informationen entlang von **Axonen** hin zu weiteren Neuronen, die diese Informationen als **elektrische Signale an den Dendriten** aufnehmen. Künstliche Neuronen bilden diese Prozesse **abstrahiert** ab: es werden **nicht-lineare Ausgabewerte** aus jedem Neuron ermittelt und über eine **gewichtete Summenbildung** an weitere Neuronen übertragen.

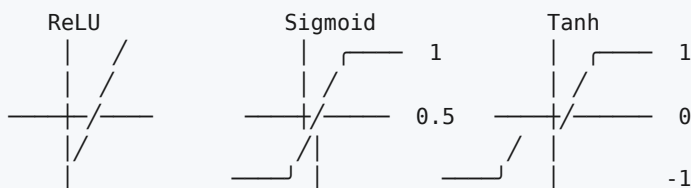
G.5 ★★★★★ Aktivierungsfunktionen (Formelsammlung!)

Die wichtigste Eigenschaft

Nichtlinearer Teil jedes Neurons. ★ **Wichtige Eigenschaft: DIFFERENZIERBAR** — die Differenzierbarkeit wird für die übliche Methode des Modelltrainings (Backpropagation) benötigt.

Die drei Hauptfunktionen

Funktion	$h(x)$	Ableitung $h'(x)$	Wertebereich
ReLU (Rectified Linear Unit)	$\begin{cases} x & x > 0 \\ 0 & \text{else} \end{cases}$	$\begin{cases} 1 & x > 0 \\ 0 & \text{else} \end{cases}$	$[0, \infty)$
Sigmoid	$\frac{1}{1 + e^{-x}}$	$h(x)(1 - h(x))$	$(0, 1)$
Tanh	$\tanh(x)$	$1 - \tanh^2(x)$	$(-1, 1)$



Weitere Aktivierungsfunktionen

- **Softmax** — für den **Output Layer** bei Klassifikation mit **mehr als 3 exklusiven Klassen** (macht die Ausgaben zu Wahrscheinlichkeiten, die sich zu 1 summieren)
- **ELU, Leaky-ReLU, Swish, Cos/Sin, Soft-Sign, ...**
- Beliebige andere Funktionen, die durch Anwender/Experten erstellt werden

🔑 **Wichtig: Differenzierbarkeit.**

G.6 ★★★★★ Der Anknüpfungspunkt zu klassischer Statistik/ML (Prüfungsklassiker!)

Einzelnes Neuron = lineares Regressionsmodell, gefolgt von einer (nicht-linearen) Transformation

? Wenn Sigmoid-Aktivierung: ... ? 🔑 → **LOGISTISCHE REGRESSION!**

Neuronales Netz = Menge miteinander verbundener Neuronen, also viele Module bestehend aus **linearem Modell + nichtlinearer Aktivierung**.

📝 ★ Übung 10.8 (a, b, c) — die vollständige Brücke

(a) LinReg ↔ NN-Regression:

Klassische lineare Regressionsmodelle sind im Prinzip **neuronale Netze, die nur aus einem einzigen Neuron bestehen**. Damit lässt sich lineare Regression durch NN abbilden, **NNs können aber auch beliebige nicht-lineare Zusammenhänge abbilden**.

(b) Klassifikation:

Logistische Regression entspricht einem Neuron mit logistischer Funktion (Sigmoid) als Aktivierungsfunktion.

(c) ★★ SVM ↔ NN (überraschender Zusammenhang!):

Ja, es gibt Ähnlichkeiten. Das **Training** ist zwar recht unterschiedlich, die **Inferenz aber tatsächlich extrem ähnlich**. In beiden Modellen wird die Ausgabe über **gewichtete Summen und eine nichtlineare Transformation (Kernel)** gebildet.

SVM-Vorhersage: $n_0 + \sum_{i=1}^n \alpha_i K(\vec{x}_i, \vec{x}_i')$

- **Gewichtete Summe mit Bias n_0** entspricht einer **linearen Ausgabeschicht**
- Die **Kernelfunktion K** kann als **verdeckte Schicht mit einer speziellen Aktivierungsfunktion** interpretiert werden

📝 Übung 10.7 — Grundelemente eines NN:

1. **Eingabe(-Schicht):** Aufnahme der Eingabedaten
2. **Neuronen / Layers:** mehrfache gewichtete Summenbildung + nicht-lineare Funktion (activation functions)
3. **Ausgabeschichten** (auch Neuronen, aber meist mit anderen activation functions)
4. Auch die **Loss-Function** kann man als Teil des Netzwerkes sehen (Zielfunktion für die Gewichtsoptimierung)
5. Schließlich ist auch ein **Trainingsalgorithmus (Optimierungsalgorithmus)** notwendig, der für die gegebenen Daten, Netzstruktur und Loss-Function die Gewichte bestimmt

G.7 ★★★★★ INFERENCE / FORWARD PASS (= Klausuraufgabe 10!)

📝★★★★ Das vollständige Beispiel des Dozenten (3-3-2-Netz, Sigmoid)

Eingabe: $\vec{x} = (0.5, 0.9, -0.3)^T$ **Gewichte hidden layer 1** (zeilenweise je Neuron):

$$W_h = \begin{pmatrix} 1 & -2 & 2 \\ 2 & 1 & -4 \\ 1 & -1 & 0 \end{pmatrix} \begin{matrix} \leftarrow \text{Neuron 1} \\ \leftarrow \text{Neuron 2} \\ \leftarrow \text{Neuron 3} \end{matrix}$$

Gewichte output layer:

$$W_o = \begin{pmatrix} -3 & 1 & -3 \\ 0 & 1 & 2 \end{pmatrix} \begin{matrix} \leftarrow \text{Neuron 1} \\ \leftarrow \text{Neuron 2} \end{matrix}$$

Aktivierung: Sigmoid $g(x) = \frac{1}{1+e^{-x}}$ (kein Bias in diesem Beispiel)

SCHICHT 1:

$$h_1 = g(0.5 \cdot 1 + 0.9 \cdot (-2) + (-0.3) \cdot 2) = g(0.5 - 1.8 - 0.6) = g(-1.9) \approx \mathbf{0.13}$$

$$h_2 = g(0.5 \cdot 2 + 0.9 \cdot 1 + (-0.3) \cdot (-4)) = g(1.0 + 0.9 + 1.2) = g(3.1) \approx \mathbf{0.96}$$

$$h_3 = g(0.5 \cdot 1 + 0.9 \cdot (-1) + (-0.3) \cdot 0) = g(0.5 - 0.9 + 0) = g(-0.4) \approx \mathbf{0.40}$$

SCHICHT 2 (Output):

$$y_1 = g(0.13 \cdot (-3) + 0.96 \cdot 1 + 0.40 \cdot (-3)) = g(-0.39 + 0.96 - 1.20) = g(-0.63) \approx \mathbf{0.35}$$

$$y_2 = g(0.13 \cdot 0 + 0.96 \cdot 1 + 0.40 \cdot 2) = g(0 + 0.96 + 0.80) = g(1.76) \approx \mathbf{0.85}$$

⚠ **Achtung des Dozenten:** „Wir runden hier stark, kann zu Abweichungen führen.“ → In der Klausur möglichst spät runden!

★ Matrixmultiplikation statt Einzelberechnung

Statt jedes Neuron einzeln zu berechnen: alle Neuronen eines Layers gleichzeitig!

- **Matrixmultiplikation** der Eingabedaten mit den Gewichten
- Eingabedaten: 1 Datenpunkt (**Vektor**) oder mehrere Datenpunkte (**Matrix**)
- Gewichte: Gewichte jedes Neurons (z. B. **zeilenweise**)
- **Danach:** auf jedes Element die Aktivierungsfunktion anwenden

🔑 Erlaubt effiziente Ausführung auf hoch-paralleler Hardware (GPU/TPU).

$$\vec{h} = g(W_h \vec{x} + \vec{b})$$

✍️☆☆ MUSTERAUFGABE: Klausuraufgabe 10 (5 Punkte)

Netz: 2 Schichten. Schicht 1 hat **2 Neuronen**, Schicht 2 hat **1 Neuron**. Eingabe hat **2 Features**.

- Datenpunkt: $x_1 = 3, x_2 = 4$
- Neuron 1 (Schicht 1): $w_{11} = -1, w_{12} = 1, b_1 = 1$
- Neuron 2 (Schicht 1): $w_{21} = -1, w_{22} = 0, b_2 = -1$
- Aktivierung: **ReLU** in beiden Schichten
- **Frage:** Mit welchen Parameterwerten ($\vec{w}^*$, Bias b^*) der zweiten Schicht erzeugt das Netz eine Ausgabe von $y(\vec{x}) = 0$?

Schritt 1 — Vor-Aktivierungen Schicht 1:

$$s_1 = 3 \cdot (-1) + 4 \cdot 1 + 1 = -3 + 4 + 1 = 2$$

$$s_2 = 3 \cdot (-1) + 4 \cdot 0 - 1 = -3 + 0 - 1 = -4$$

Schritt 2 — ReLU anwenden:

$$a_1 = \text{ReLU}(2) = 2, \quad a_2 = \text{ReLU}(-4) = 0$$

Schritt 3 — Bedingung für die Ausgabe aufstellen:

$$w_1^* \cdot a_1 + w_2^* \cdot a_2 + b^* = 0 \Rightarrow w_1^* \cdot 2 + w_2^* \cdot 0 + b^* = 0 \Rightarrow \boxed{2w_1^* + b^* = 0}$$

Schritt 4 — Antwort:

Es ergeben sich **unendlich viele Lösungen** mit $2w_1^* = -b^*$. w_2^* **ist beliebig** (da $a_2 = 0$). Konkrete Beispiellösung: $w_1^* = 1, w_2^* = \text{beliebig}, b^* = -2$.

🔑 **Der Trick der Aufgabe:** Weil $a_2 = 0$ ist, fällt w_2^* komplett raus. Und weil das Ergebnis 0 sein soll, greift ReLU nicht einschränkend ($\text{ReLU}(0)=0$).

✍️☆☆ MUSTERAUFGABE 2: Übung 10.9 (2-2-2-1-Netz, ReLU) — vollständig gerechnet

Netz: 2 Eingabewerte, **2 hidden layers mit je 2 Neuronen**, output layer mit 1 Neuron. ReLU überall, **kein Bias**.

- **Layer 1:** N1: $w_{11} = 0.5, w_{12} = -0.1$ · N2: $w_{21} = 0.1, w_{22} = 0.3$
- **Layer 2:** N1: $w_{11} = 1.5, w_{12} = -0.2$ · N2: $w_{21} = 0.8, w_{22} = -0.2$
- **Output:** $w_{o,1} = -0.2, w_{o,2} = 0.7$

P1 = (1, 1):

Schicht	Rechnung	Ergebnis
L1-N1	$\text{ReLU}(0.5 \cdot 1 - 0.1 \cdot 1) = \text{ReLU}(0.4)$	0.4
L1-N2	$\text{ReLU}(0.1 \cdot 1 + 0.3 \cdot 1) = \text{ReLU}(0.4)$	0.4
L2-N1	$\text{ReLU}(1.5 \cdot 0.4 - 0.2 \cdot 0.4) = \text{ReLU}(0.6 - 0.08)$	0.52
L2-N2	$\text{ReLU}(0.8 \cdot 0.4 - 0.2 \cdot 0.4) = \text{ReLU}(0.32 - 0.08)$	0.24
Output	$\text{ReLU}(-0.2 \cdot 0.52 + 0.7 \cdot 0.24) = \text{ReLU}(-0.104 + 0.168)$	0.064

P2 = (1, 0):

Schicht	Rechnung	Ergebnis
L1-N1	$\text{ReLU}(0.5 \cdot 1 - 0.1 \cdot 0)$	0.5
L1-N2	$\text{ReLU}(0.1 \cdot 1 + 0.3 \cdot 0)$	0.1
L2-N1	$\text{ReLU}(1.5 \cdot 0.5 - 0.2 \cdot 0.1) = \text{ReLU}(0.75 - 0.02)$	0.73
L2-N2	$\text{ReLU}(0.8 \cdot 0.5 - 0.2 \cdot 0.1) = \text{ReLU}(0.4 - 0.02)$	0.38
Output	$\text{ReLU}(-0.2 \cdot 0.73 + 0.7 \cdot 0.38) = \text{ReLU}(-0.146 + 0.266)$	0.120

P3 = (0, 0): alle Vor-Aktivierungen 0 → **0.000**

Musterlösung des Dozenten: $\begin{pmatrix} 0.064 \\ 0.120 \\ 0.000 \end{pmatrix} \checkmark$

(c) Punkt mit $y \approx 1$?

 **Trick:** ReLU ist **positiv homogen** — ohne Bias skaliert die Ausgabe **linear** mit der Eingabe!

$$P = \left(\frac{25}{3}, 0\right) \quad \text{denn} \quad 0.120 \cdot \frac{25}{3} = \frac{3.0}{3} = 1.000$$

(allgemein: Faktor = $1/0.12 = 25/3 \approx 8.333$ auf P2 anwenden)

Einzelne-Neuronen-Aufgaben (Übungen 10.1 und 10.2)

Übung 10.1 — Neuron **ohne Bias**, Eingaben (1, -2, 3, 4), Gewichte (-0.5, 0.2, -0.5, 1), **ReLU**:

$$z = 1 \cdot (-0.5) + (-2) \cdot 0.2 + 3 \cdot (-0.5) + 4 \cdot 1 = -0.5 - 0.4 - 1.5 + 4 = \mathbf{1.6}$$

$$\text{Da } z > 0: \quad y = \text{ReLU}(1.6) = \mathbf{1.6}$$

Übung 10.2 — Neuron **mit Bias** $b = 1$, Eingaben (1, -2, 3, 3), Gewichte (-1, 0.3, -0.5, 1), **Sigmoid**:

$$z = 1 \cdot (-1) + (-2) \cdot 0.3 + 3 \cdot (-0.5) + 3 \cdot 1 + 1 = -1 - 0.6 - 1.5 + 3 + 1 = \mathbf{0.9}$$

$$y = \frac{1}{1 + e^{-0.9}} = \frac{1}{1 + 0.4066} \approx \mathbf{0.71}$$

 **Achtung Vorzeichenfalle:** In der Musterlösung des Dozenten steht $-1 \cdot 1 - 2 \cdot 0.3 - 3 \cdot 0.5 + 3 \cdot 1 + 1 = 0.9$ — das ist dasselbe, nur anders geschrieben.

G.8 ★ Computation Graph (die andere Sichtweise)

- **Neuronales Netz = parametrisierte, nichtlineare Funktion $f(\vec{x})$**
- **Gerichteter Graph von Funktionen**, abhängig von Parametern (Neuronengewichte)
- **Kombination von linearen (parametrisierten) und nichtlinearen Funktionen**
-  **Die Anwendung von Funktionsblöcken muss NICHT strikt sequentiell erfolgen** (→ Skip-Connections, Verzweigungen)

G.9 ★★ Training: Loss, Backpropagation, Gradient Descent

Was wird gelernt?

- **Jedes Gewicht an einer Kante des Netzes, und der Bias:** unbekannte Parameter
- Außerdem: **zusätzliche Parameter**, z. B. von Aktivierungsfunktionen (siehe Leaky-ReLU)
- 🗝️ **Große Netze können MILLIARDEN von Parametern haben**
- „Lernen“/Training erfolgt über die **Anpassung dieser Parameter**

★ Loss-Funktion

Aussage

Loss-Funktion ist ein Maß für den Modellfehler

Der Loss wird **während des Trainings reduziert**, um das Modell an die Daten anzupassen

D. h.: wir suchen **Gewichte (Parameter), die geringen Loss verursachen**

Beispiel-Maße: log-Loss, Cross-Entropy, MSE, ...

★ **Genau wie Aktivierungsfunktionen: sollte DIFFERENZIERBAR sein** (zumindest für Standardansätze)

Kann auch **Maße für die Modellkomplexität** einbeziehen, um „einfachere“ Modelle zu finden → **Regularisierung**

★★ Verkettung differenzierbarer Knoten

- **Jeder Knoten in einem NN ist (normalerweise) differenzierbar, einschließlich Loss**
- So können **partielle Ableitungen für jeden Knoten** berechnet werden (Ausgabe abgeleitet nach Parametern und Eingaben)
- **Über die KETTENREGEL der Differentialrechnung:** Verknüpfen der Ableitungen verbundener Knoten

★★★★ Backpropagation + Gradient Descent (auswendig!)

1. FORWARD PASS
→ Berechnen der Ausgabe des Netzes für einen (Teil-)Datensatz
2. Berechnen des LOSS und seiner partiellen Ableitungen
3. BACKWARD PASS
→ Partielle Ableitungen RÜCKWÄRTS verketteten mit denen der vorigen Knoten
→ bis wir partielle Ableitungen des Loss nach ALLEN Parametern erhalten
→ daraus ergibt sich der GRADIENT für alle Parameter
4. Aktualisierung der Parameter mit dem
GRADIENTENABSTIEGSVERFAHREN (gradient descent)

📝 Übung 10.6 (Inference-Musterantwort):

Bei der Inference (auch: **Forward-Pass**) werden Datenpunkte **schrittweise durch das Netzwerk von Neuronen verarbeitet**. Dabei werden in jedem Verarbeitungsschritt erst **gewichtete Summen** an jedem Neuron gebildet, dann diese durch eine **nicht-lineare activation function** verarbeitet und an folgende Neuronen weitergegeben. **Die Ausgabe der letzten Neuronenschicht bildet die Ausgabe des Netzwerkes ab.**

G.10 ★ Modellarchitektur

Die bisherigen Beispiele: „**vollständig verbundene**“ oder **dichte Schichten** — jedes Neuron in einer Schicht ist mit jedem Neuron der folgenden Schicht verbunden.

Viele andere Netz-Architekturen möglich:

Architektur	Typische Anwendung
Convolutional Layers	häufig für Bilder / Video / Zeitreihen
Skip- und Residual-Connections	wichtig zur Stabilisierung des Trainings u. v. m.
Rekurrente Schichten	zur Einbeziehung von Sequenz- oder Zeitstruktur
LSTM, GRU, Transformer-Blöcke	insbesondere für Sprache und andere Sequenzen

! ★ Wichtige Einschränkung

Die Architektur selbst kann NICHT direkt durch Backpropagation optimiert werden. Stattdessen häufig:

- **copy-paste und Expertenwissen**
- oder: **Tuning / Ausprobieren**
- Siehe auch: **Neural Architecture Search (NAS)**

G.11 Libraries & Frameworks

📝 Übung 10.10 (Musterantwort):

TensorFlow und Torch bzw. PyTorch. ⚠️ **Anmerkung: Keras ist eigentlich KEINE eigene Implementation, sondern eine SCHNITTSTELLE für TensorFlow u. a.**

Einfache Keras-Implementierung eines Computation Graph:

```
model = Sequential()
model.add(Dense(H, input_dim=N))  ## definiert Linearität + W0
model.add(Activation("tanh"))      ## definiert 1. Nichtlinearität
model.add(Dense(K))               ## definiert Linearität + W1
model.add(Activation("softmax"))   ## definiert 2. Nichtlinearität
```

G.12 Lab: Palmer Penguins (Python/Keras)

Datensatz: 344 Pinguine, **3 Spezies**, 3 Inseln, Jahre 2007–2009. Features: Schnabel- (bill), Flossen- (flipper) Maße, Gewicht, Geschlecht.

 **Anmerkung des Dozenten:** „Wir benutzen **Python**, da für NNs die bei weitem üblichere Programmiersprache.“

```
import pandas, numpy
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import to_categorical, set_random_seed
from sklearn.model_selection import cross_val_score, KFold
from sklearn.preprocessing import LabelEncoder

set_random_seed(42)
dataset = pandas.read_csv("../penguins.csv").dropna() # → 333 Zeilen, 7 Spalten

X = dataset.values[:,2:6]; X = numpy.asarray(X).astype('float32')
Y = dataset.values[:,0]
encoder = LabelEncoder(); encoder.fit(Y)
dummy_y = to_categorical(encoder.transform(Y)) # One-Hot für 3 Klassen

def my_model():
    model = Sequential()
    model.add(Dense(10, input_dim=X.shape[1], activation='relu'))
    model.add(Dense(8, activation='relu'))
    model.add(Dense(3, activation='softmax')) # 3 Klassen → softmax
    model.compile(loss='categorical_crossentropy',
                  optimizer='adam', metrics=['accuracy'])
    return model
```

Die didaktische Pointe des Labs

Nach 5 Epochen: Accuracy nur **0.2042** — das Modell ist schlecht.

 ***„Aber ist dieses Modell gut? — Schwer zu sagen. Möglicherweise nicht, Genauigkeit ist gering. Aber wir haben auch keine saubere Validierung vorgenommen. Wir sollten zumindest eine k-fold CV durchführen!“***

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = []
for train_index, test_index in kf.split(X):
    X_train, X_val = X[train_index], X[test_index]
    y_train, y_val = dummy_y[train_index], dummy_y[test_index]
    model = my_model()
    model.fit(X_train, y_train, epochs=5, batch_size=5, verbose=0)
    loss, acc = model.evaluate(X_val, y_val, verbose=0)
    scores.append(acc)

print("Cross-val mean of accuracy:", numpy.mean(scores)) # 0.4478
print("Cross-val st.dev. of accuracy:", numpy.std(scores)) # 0.2422
```

🔑 **Lehre:** Selbst mit CV ist das Modell schlecht ($44.8\% \pm 24.2\%$ bei 3 Klassen = kaum besser als Raten). **Die hohe Standardabweichung von 0.24 zeigt außerdem, wie instabil das Modell ist** — genau das, wofür man Resampling braucht (Teil E)! **Lab-Aufgabe: Verbessern Sie das Modell** (z. B. Feature-Skalierung, mehr Epochen, andere Architektur).

▼ ? Selbsttest Teil G (aufklappen)

1. Schreiben Sie die Formel eines künstlichen Neurons auf und benennen Sie alle Symbole.
2. Warum müssen Aktivierungsfunktionen differenzierbar sein?
3. Nennen Sie ReLU, Sigmoid, Tanh mit Formel und Ableitung.
4. Wann verwendet man Softmax?
5. Ein Neuron mit Sigmoid-Aktivierung entspricht welchem klassischen Modell?
6. Ein Neuron mit Identitäts-Aktivierung entspricht welchem klassischen Modell?
7. Erklären Sie die Analogie zwischen SVM-Inferenz und einem NN.
8. Neuron: Eingaben (2, -1, 3), Gewichte (0.5, 2, -1), Bias 0.5, ReLU. Ausgabe? ($z = 1 \cdot 2 - 3 + 0.5 = -3.5 \rightarrow 0$)
9. Dasselbe mit Sigmoid. ($1/(1+e^{\{3.5\}}) \approx 0.029$)
10. Beschreiben Sie Backpropagation in 4 Schritten.
11. Warum kann die Architektur nicht durch Backpropagation optimiert werden? Was macht man stattdessen?
12. Nennen Sie 4 Gründe für den aktuellen Erfolg von Deep Learning.
13. Nennen Sie drei Herausforderungen (ethisch/juristisch, technisch, praktisch).
14. Ist Keras eine eigene Implementierung? Begründen Sie.
15. Warum ist eine Matrixmultiplikation je Layer effizienter als Einzelberechnung?

TEIL H — DIE ÜBUNGSKLAUSUR KOMPLETT (mit Musterlösungen)

Diese Klausur ist die **beste Vorbereitung**. Rechnen Sie sie **zuerst ohne Lösung** durch!

Aufgabe 1 (6 Punkte) — Grundbegriffe

(a) (4 P.) Erläutern Sie den Unterschied von Klassifikation und Regression, und nennen Sie jeweils ein Anwendungsbeispiel.

Musterlösung: Sowohl Klassifikation als auch Regression sind mögliche **Problemstellungen beim Supervised Learning**. In beiden Fällen sind somit bei Trainingsdaten **Labels bekannt** und sollen für neue Daten vorhergesagt werden.

- Bei der **Klassifikation** sind die Labels **ungeordnete Kategorien**, wie z. B. die Unterscheidung zwischen **Spam und Ham**, oder **krank/gesund**.
- Bei der **Regression** hingegen sind es üblicherweise **reelle Zahlen**, wie z. B. ein **Blutdruck** oder eine **Niederschlagsmenge**.

(b) (2 P.) Beispiele für ein kategorisches Feature mit Ausprägungen — eines mit 2, eines mit mehr als 2 Ausprägungen.

Musterlösung:

- Ob eine Mail von einem **bekannten Absender** kommt (Ausprägungen: **ja/nein**)
- **Materialtyp** eines Bauteils (Ausprägungen: **Metall, Glas, Plastik, ...**)

Aufgabe 2 (6 Punkte) — Underfitting

(a) (1 P.) Was bedeutet Underfitting für die Komplexität des Modells?

Das Modell ist **NICHT KOMPLEX GENUG**, um die Zusammenhänge in den Daten abbilden zu können.

(b) (2 P.) Was schließen Sie über Varianz und Bias?

Aufgrund der **geringen Komplexität** wird das Modell eine relativ **geringe VARIANZ** und einen relativ **hohen BIAS** aufweisen.

(c) (3 P.) Wie lösen Sie das Underfit-Problem OHNE andere Modelltypen?

Man könnte entweder

1. **neue Features hinzufügen**, beispielsweise durch Ergänzung aus **anderen Datenquellen**, oder
2. das lineare Modell durch **polynomielle Terme und Interaktionsterme** komplexer gestalten.

Aufgabe 3 (6 Punkte) — Interaktionsterm (Blutdruck)

$y = 50 - 2.5x_1 + 8x_2 - 1.2x_1x_2$; y = diastolischer Blutdruck (mmHg), x_1 = Alter (Jahre), $x_2 = 1$ (Raucher) / 0 (Nichtraucher). Alle p-Werte < 0.05.

(a) (3 P.) Steigt der Blutdruck stärker bei Rauchern oder Nichtrauchern?

Durch **Ableiten und Einsetzen**:

$$\frac{\partial y}{\partial x_1} = -2.5 - 1.2 x_2$$

- Nichtraucher ($x_2 = 0$): -2.5
- Raucher ($x_2 = 1$): $-2.5 - 1.2 = -3.7$

Interpretation: Nach diesem Modell **nimmt der Blutdruck mit dem Alter ab** (negatives Vorzeichen) — und zwar **stärker bei Rauchern** ($-3,7$ mmHg/Jahr) als bei Nichtrauchern ($-2,5$ mmHg/Jahr).

(b) (3 P.) In welchem Alter haben Raucher und Nichtraucher denselben Blutdruck?

- Raucher: $y_R(x_1) = 50 - 2.5x_1 + 8 - 1.2x_1 = 58 - 3.7x_1$
- Nichtraucher: $y_N(x_1) = 50 - 2.5x_1$

Gleichsetzen: $58 - 3.7x_1 = 50 - 2.5x_1 \Rightarrow 8 = 1.2x_1 \Rightarrow x_1 = \frac{8}{1.2} \approx 6,67$

Bei einem Alter von ca. 6,67 Jahren sind die vorhergesagten Blutdruckwerte gleich.

Aufgabe 4 (6 Punkte) — Logistische Regression (Login-Angriff)

$$p(X) = \frac{s}{1+s} \text{ mit } s = e^{0.6+0.5X_1-0.01X_2+10.5X_3}$$

(a) (2 P.) $X_1 = 100$, $X_2 = 6000$, $X_3 = 1$ (**Linux**). p mit 3 Nachkommastellen?

$$s = \exp(0.6 + 50 - 60 + 10.5) = e^{1.1} = 3.004 \text{ und } p = \frac{3.004}{1+3.004} = 0.750$$

(b) (2 P.) Was bedeutet der konstante Wert 0.6?

Der Faktor bestimmt die Ausgabe des Modells, **wenn alle Features Null sind** — also kein Loginversuch, keine verstrichene Zeit seit letztem Login, OS ist Windows. **Wenn der konstante Faktor Null wäre, ergäbe sich eine Wahrscheinlichkeit von 50 %**, dass ein Angriff vorliegt. Mit einem Wert von 0.6 ergibt sich eine entsprechend **höhere Wahrscheinlichkeit von ca. 0.65**.

(c) (2 P.) Wie viele Loginversuche für 100 % Sicherheit?

Eine **100 %-Wahrscheinlichkeit ist nur als GRENZWERT bestimmbar**, wenn die Anzahl der Loginversuche **gegen Unendlich** läuft. **Somit praktisch nicht möglich**.

Aufgabe 5 (5 Punkte) — SVM und Hyperebenen

(Musterlösung siehe Teil D.9 — bitte dort wörtlich lernen.)

Aufgabe 6 (6 Punkte) — Warum Validierungsdaten?

(Musterlösung siehe Teil E.2 — bitte dort wörtlich lernen.)

Aufgabe 7 (4 Punkte) — Baum ablaufen

Musterlösung (Dokumentationsformat!):

```
nox ist NICHT >= 0.67    → 13.6 (rechts)
tax ist >= 268           → (links)
nox ist >= 0.514         → (links)
dis ist NICHT >= 1.38    → (rechts)
Blatt: Vorhersage 35.3
```

Aufgabe 8 (2 Punkte) — Boosting-Overfitting

VERRINGERN. Kleinere λ -Werte führen dazu, dass das Modell in **jedem Boosting-Schritt weniger stark an die Daten angepasst** wird, also **langsamer lernt**, und wirkt somit **Overfitting entgegen**.

Aufgabe 9 (4 Punkte) — Random Forest, zufällige Feature-Auswahl

Ohne diesen Trick werden sich die erzeugten Bäume des Ensembles alle sehr ähnlich sein bzw. **korrelieren**. Das **erhöht die Varianz bzw. den Fehler des Modells**. Mit der reduzierten Auswahl werden die Bäume gezwungen, die Trainingsdaten **mit anderen Features abzubilden**, somit **dekorreliert**, was den Fehler senken soll.

Aufgabe 10 (5 Punkte) — Neuronales Netz

$s_1 = 3 \cdot (-1) + 4 \cdot 1 + 1 = 2$, $s_2 = 3 \cdot (-1) + 0 - 1 = -4$ Nach ReLU: $a_1 = 2$, $a_2 = 0$ Bedingung $y = 0$: $w_1^* \cdot 2 + w_2^* \cdot 0 + b^* = 0$ bzw. $2w_1^* + b^* = 0$ Es ergeben sich unendlich viele Lösungen mit $2w^* = -b^*$.

TEIL I — ALLE ÜBUNGSAUFGABEN ZU KAPITEL 2 (Grundlagen)

Diese Aufgaben sind bisher noch nicht vollständig eingearbeitet. Sie sind **stark prüfungsähnlich**.

Übung 2.1 ★★ — Flexibel oder unflexibel? (KLASSIKER!)

Geben Sie für jeden Teil an, ob die Leistung eines flexiblen Modells besser oder schlechter wäre als die eines unflexiblen. Begründen Sie.

Szenario	Antwort	Begründung
(a) n extrem groß, p klein	FLEXIBEL	Große n und kleine p verringern jeweils das Risiko von Overfitting → ein flexibles Modell könnte relativ gut funktionieren
(b) p extrem groß, n klein	UNFLEXIBEL	Kleine n und große p erhöhen das Overfitting-Risiko → ein unflexibles Modell könnte relativ gut funktionieren
(c) Beziehung stark nicht-linear	FLEXIBEL	Ein flexibles Modell kann die starken Nichtlinearitäten besser abbilden
(d) $\sigma^2 = \text{var}(\epsilon)$ extrem hoch	UNFLEXIBEL	Aufgrund der hohen nicht-reduzierbaren Fehler besteht ein großes Risiko, dass ein flexibles Modell sich nur an diese Fehler anpasst und somit overfittet

🔑 **Merkregel:** flexibel = viele Daten + wenige Features + Nichtlinearität; unflexibel = wenige Daten + viele Features + viel Rauschen.

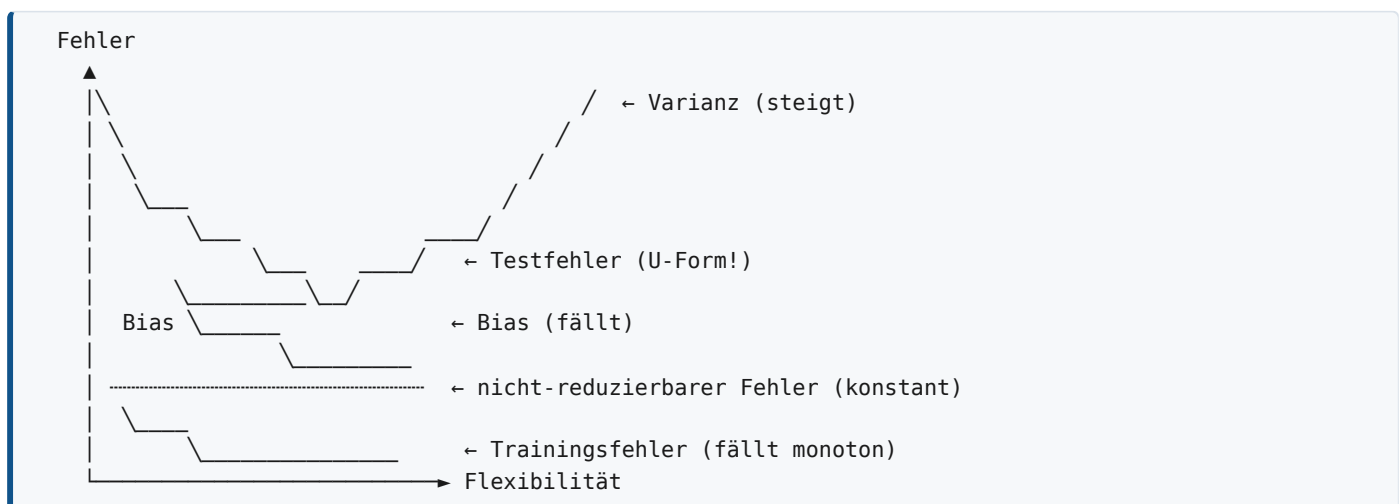
Übung 2.2 ★ — Klassifikation/Regression, Inferenz/Vorhersage, n und p

Szenario	Typ	Zielvariable	Interesse	n	p
(a) 500 größte US-Unternehmen: Gewinn, Beschäftigtenzahl, Branche, CEO-Gehalt. Welche Faktoren beeinflussen das CEO-Gehalt?	Regression	CEO-Gehalt	Schlussfolgerungen (Inferenz)	500	3
(b) 20 ähnliche Produkte: Erfolg/Misserfolg, Preis, Marketingbudget, Konkurrenzpreis + 10 weitere Variablen. Wird das neue Produkt Erfolg haben?	Klassifikation	Produkt-erfolg	Vorhersage	20	13
(c) Wöchentliche Daten 2012: %-Änderung USD/EUR, US-Markt, UK-Markt, DE-Markt. Wechselkurs vorhersagen.	Regression	Wechselkurs	Vorhersage	52	3

⚠ **Zählfalle bei p :** Die Zielvariable zählt **NICHT** als Feature! (a): 4 Variablen – 1 Zielvariable = 3. (b): 3 genannte + 10 weitere = 13. (c): 4 – 1 = 3.

Übung 2.3 ★★ — Die Kurvenskizze (SEHR beliebt!)

(a) Skizzieren Sie typische Kurven für Bias, Varianz, Trainingsfehler, Testfehler und den nicht-reduzierbaren Fehler. x-Achse = Flexibilität.



(b) Erklären Sie die Form jeder Kurve — die Musterlösung:

Kurve	Erklärung (wörtlich)
Bias	Der Bias beschreibt, wie stark die mittlere Vorhersage der Zielvariable von der Regressionsfunktion abweicht (verursacht durch die Modellstruktur selbst). Der Bias sinkt mit steigender Flexibilität, da das Modell zunehmend an die Daten angepasst werden kann.
Varianz	Die Varianz meint die Varianz der Vorhersage der Zielvariable (verursacht durch leichte Änderungen der Trainingsdaten). Die Varianz steigt , da mit höherer Flexibilität auch die Auswirkung von geänderten Trainingsdaten wächst.
Nicht-reduzierbarer Fehler	Verläuft konstant , da er nicht durch das Modell zu beeinflussen ist.
Testfehler	Ergibt sich als Summe von Bias, Varianz des Modells und Varianz des nichtreduzierbaren Fehlers . Bei steigender Flexibilität sinkt der Testfehler erst (Underfitting wird zu ‚gutem‘ Fit) und steigt dann (Overfitting).
Trainingsfehler	Vernachlässigt die Varianz des Modells und sinkt somit bei steigender Flexibilität (oder bleibt ab einem gewissen Punkt konstant). Das Modell passt sich zunehmend den Trainingsdaten an und kann bei sehr hoher Flexibilität selbst den nicht-reduzierbaren Fehler unterschreiten (Overfitting).

Übung 2.4 — Vor-/Nachteile flexibler Ansätze

Vorteile eines flexiblen Ansatzes: die Möglichkeit, **genauere Modelle** zu erzeugen, bzw. auch bei **stärker nichtlinearen Zusammenhängen** zwischen Features und Zielvariable ein gutes Modell zu erzeugen. (Nachteile: Overfitting-Risiko, schlechtere Interpretierbarkeit, mehr Daten nötig.) Siehe auch Aufgabe 1.

Übung 2.5 ★ — Parametrisch vs. nicht-parametrisch

	Parametrisch	Nicht-parametrisch
Definition	Die Form des Modells ist weitgehend vorgegeben (z. B. lineares Regressionsmodell). Über Parameter passt sich diese Form den Daten an.	Die Form ist nicht direkt vorgegeben und hängt somit stärker von den vorliegenden Daten ab.
Vorteile	Bei wenigen Daten möglicherweise vorzuziehen	Flexibler . Besser, insbesondere wenn viele Daten vorliegen.
Nachteile	Unflexibel — funktioniert weniger gut, wenn die Form nicht zu den Daten passt	Benötigt mehr Daten

(Beispiele: parametrisch = lineare/logistische Regression; nicht-parametrisch = kNN, Entscheidungsbäume)

Übung 2.6 ★★ — kNN von Hand rechnen (SEHR klausurrelevant!)

Nr.	X_1	X_2	X_3	Y
1	0	3	0	Red
2	2	0	0	Red
3	0	1	3	Red
4	0	1	2	Green
5	-1	0	1	Green
6	1	1	1	Red

Testpunkt: $X_1 = X_2 = X_3 = 0$

(a) Euklidische Distanzen berechnen:

Nr.	Rechnung	d
1	$\sqrt{0^2 + 3^2 + 0^2} = \sqrt{9}$	3
2	$\sqrt{2^2 + 0 + 0} = \sqrt{4}$	2
3	$\sqrt{0 + 1^2 + 3^2} = \sqrt{10}$	≈ 3.2
4	$\sqrt{0 + 1^2 + 2^2} = \sqrt{5}$	≈ 2.2
5	$\sqrt{1^2 + 0 + 1^2} = \sqrt{2}$	≈ 1.4
6	$\sqrt{1^2 + 1^2 + 1^2} = \sqrt{3}$	≈ 1.7

(b) Vorhersage mit $k = 1$?

GREEN — die Distanz zwischen Testpunkt und **Beobachtung 5** ist am niedrigsten ($\sqrt{2} \approx 1.4$).

(c) Vorhersage mit $k = 3$?

RED — die drei nächsten sind **Beobachtungen 5 (1.4), 6 (1.7), 2 (2.0)**. Klassen: Green, Red, Red → **Red** ist die häufigste Klasse.

(d) Wenn die wahre Bayes-Entscheidungsgrenze stark nichtlinear ist — großes oder kleines k ?

KLEINES k . Bei **wenigen Nachbarn** kann die Entscheidungsgrenze des Modells selbst **stark nichtlinear** sein. Das liegt daran, dass bei wenigen Nachbarn **kleine Änderungen des Testpunktes schneller zu einer Änderung der gesamten Nachbarschaft** führen können.

(e) Was, wenn X_1 die Zielvariable und X_2, X_3, Y die Features wären?

In diesem Fall ist die Zielvariable X_1 **quantitativ/numerisch** statt qualitativ → **Regression** statt Klassifikation. Dies **ändert nichts am Modell selbst**, üblicherweise wird dann aber der **Mittelwert der k nächsten Nachbarn** statt der häufigsten Beobachtung vorhergesagt. Zudem liegt nun ein **qualitatives Feature (Y)** vor → **Dummy-Coding**, z. B. Green = 1, Red = 2.

Übung 2.7 — $f(x)$ vs. $\hat{f}(x)$

Die Funktion $f(x)$ beschreibt den **tatsächlichen bedingten Erwartungswert** der Zielvariable Y , der üblicherweise **nicht zu ermitteln** ist. Die Funktion $\hat{f}(x)$ versucht diesen Wert **basierend auf Daten und einem entsprechenden ML-Modell zu schätzen**.

Übung 2.8 — Ideale Regressionsfunktion und Mittelwert

Die ideale Regressionsfunktion $f(x)$ ist grob gesagt der **Mittelwert der Zielvariable Y an einem Punkt x** . Gemittelt wird über verschiedene Beobachtungen, deren Unterschied sich **nur aus dem nichtreduzierbaren Fehler** ergibt („Rauschen“ von Y).

Übung 2.9 ★ — Kann ϵ genau Null werden?

Ja, er kann Null werden, wenn das beobachtete System selbst **DETERMINISTISCH** ist. Das kann z. B. eintreten, wenn die beobachteten Daten aus einer **diskreten Computersimulation** stammen. Daten aus der echten Welt (Sensoren, Umfragen, etc.) weisen hingegen üblicherweise **inhärent Fehler** auf.

⚠️ **Zudem ist es möglich, dass der nichtreduzierbare Fehler mit Null geschätzt wird, weil OVERFITTING stattfindet.** Der tatsächliche nichtreduzierbare Fehler wird allerdings (auf neuen/ungesehenen Daten) in diesem Fall **nicht Null** sein.

Übung 2.10 — Ideale Regressionsfunktion ↔ Nearest Neighbor

Die echte Regressionsfunktion sowie die Verteilung des nicht-reduzierbaren Fehlers sind grundsätzlich **unbekannt**. Zudem ist es nicht möglich, den Mittelwert über **unendlich viele Beobachtungen** zu bestimmen — es liegt meist eine **begrenzte Zahl von Beobachtungen** vor. Um mit nur wenigen Datenpunkten (bzw. auch an Stellen, wo keine Datenpunkte vorliegen) eine Abschätzung des Mittels vornehmen zu können, wird über das Nearest-Neighbor-Verfahren die **Definition aufgeweicht: Gemittelt wird nicht mehr exakt am Punkt X , sondern in seiner NACHBARSCHAFT**.

Übung 2.11 ★ — Wann Nearest Neighbor anwenden?

Nearest-Neighbor-Verfahren basieren auf der Annahme, dass **lokale Nachbarschaften** von Punkten ausgenutzt werden können. Dies ist insbesondere bei **größerer Anzahl von Trainingsdaten n** sinnvoll.

Aufgrund des ‚**Curse of Dimensionality**‘ werden die Nachbarschaften von Punkten bei gleichbleibender Zahl von Nachbarn **zunehmend größer** (z. B. steigt der Radius stark an). Somit **verlieren Nachbarschaften zunehmend die Eigenschaft, ‚lokal‘ zu sein**. Vorhersagen basieren dann auf Nachbarn, die **sehr weit vom Testpunkt entfernt** liegen und somit **weniger Aussagekraft** haben.

🔑 **Neben großen n sind also auch kleine p (Anzahl Features, Dimension) sinnvoll.**

Übung 2.12 & 2.13 ★ — Grafik-Diagnose

2.12 — Systematische Muster in den Residuen:

Es scheinen **sowohl reduzierbare wie auch nichtreduzierbare Fehler** vorzuliegen. Einerseits scheinen die Beobachtungen selbst für festgelegte Punkte $X = x$ eine gewisse **Streuung** zu haben (nicht-reduzierbarer Fehler). Andererseits ist zu beobachten, dass für **kleine und große Werte von x** die **Abweichungen von der Regressionslinie eher nach OBEN** verschoben sind, während **in der Mitte eher eine Verschiebung nach UNTEN** zu beobachten ist. **Dies deutet auf eine NICHTLINEARE Komponente hin.** → Möglicherweise hilft die Hinzunahme eines **quadratischen Terms**.

2.13 — Zu wenige Datenpunkte:

Problematisch ist hier vor allem die **geringe Anzahl von Datenpunkten**, die zwar so eben noch die Schätzung des Modells zulässt, aber **kaum Rückschluss über die Qualität** erlaubt. Somit ist **unklar, ob reduzierbarer Fehler, Varianz oder Bias problematisch** sind. **Eine Hinzunahme weiterer Datenpunkte wäre hilfreich.**

🔑 **Diagnose-Merkregel:** Systematisches Muster in den Residuen (oben-unten-oben) → **Nichtlinearität**. Zufällige Streuung ohne Muster → **nicht-reduzierbarer Fehler**. Zu wenige Punkte → **keine Diagnose möglich**.

Übung 2.14 ★ — Wie prüft man auf Overfitting?

Grundlage muss **zumindest ein Datensplit** sein: nur auf einem Trainingsdatensatz trainieren, auf einem weiteren **unabhängigen Validierungsdatensatz** die Generalisierbarkeit prüfen. (Komplexere Verfahren wie Bootstrapping oder k-Fold CV können auch Verwendung finden.)

🔑 **Wichtig für den Nachweis:** zu beobachten, **wie sich der Trainings- und Validierungsfehler verhalten, wenn die MODELLKOMPLEXITÄT ERHÖHT wird**. Overfitting liegt dann vor, wenn bei steigender Komplexität (Flexibilität) der Trainingsfehler **SINKT** und der Validierungsfehler **STEIGT**.

Übung 2.15 & 2.16 ★ — Over-/Underfit aus einer Grafik beurteilen

2.15 (stark geschlängelte Kurve durch lineare Daten):

Ohne genauere Analyse **nicht eindeutig zu beantworten**. Allerdings deutet sich an, dass das Modell versucht, **das Rauschen in den Daten zu lernen**. Die Daten scheinen einem **linearen Trend** zu folgen, das Modell ist allerdings **stark nicht-linear**. **Eine Tendenz zum OVERFIT ist hier zumindest zu vermuten.**

2.16 (Kurve folgt dem Trend):

Ohne genauere Analyse nicht eindeutig zu beantworten. Allerdings deutet sich an, dass das Modell **den Trend der Daten gut abbildet und nicht das Rauschen erlernt**. Das spricht für einen **GUTEN MODELLFIT**.

⚠️ **Klausurtechnik:** Beginnen Sie solche Antworten immer mit „*Ohne eine genauere Analyse ist dies nicht eindeutig zu beantworten, allerdings deutet sich an, dass...*“ — das ist die vom Dozenten erwartete wissenschaftliche Vorsicht!

Übung 2.17 ★ — kNN-Classification vs. kNN-Regression

Beide Varianten bestimmen für die Vorhersage eines neuen Testdatenpunkts die **Distanz zu den nächsten Nachbarn** und bestimmen dann über eine **Aggregation der Y -Werte** dieser Nachbarn die Vorhersage.

Der wichtigste Unterschied ist die AGGREGIERUNG selbst:

- **Regression:** der **Mittelwert** der Nachbarn
- **Klassifikation:** der **Modus** (die häufigste beobachtete Klasse)
- Zusätzlich kann für Klassifikation auch die **Wahrscheinlichkeit bzw. Häufigkeit jeder Klasse** bestimmt werden

TEIL J — QUERSCHNITT: VERGLEICHE ÜBER ALLE KAPITEL

J.1 ★★ Die große Modellübersicht

Modell	Typ	Kap.	Flexibilität	Interpretierbarkeit	Liefert Wahrsch. ?	Stärke	Schwäche
kNN	beides	2	steuerbar über k	mittel	ja (Klassenhäufigkeit)	nichtparametrisch, einfach	Curse of Dimensionality
Lineare Regression	Regression	3	gering (ohne Polynome)	sehr hoch	—	analytisch lösbar, interpretierbar	nur linear (ohne Erweiterung)
Logistische Regression	Klassifikation	4	gering (ohne Polynome)	hoch	ja	echte Wahrscheinlichkeiten	⚠ divergiert bei exakt trennbaren Daten
LDA	Klassifikation	4	gering (linear)	mittel	ja	stabil bei kleinem n , gut für $K > 2$, Visualisierung	Normalverteilungsannahme
QDA	Klassifikation	4	mittel (quadratisch)	mittel	ja	nichtlineare Grenzen	Overfit-Gefahr, mehr Parameter
Naive Bayes	Klassifikation	4	mittel	mittel	ja	sehr gut bei großem p , gemischte Typen	Unabhängigkeitsannahme
Soft-Margin Classifier	Klassifikation	9	gering (linear)	mittel	nein	robust bei fast trennbaren Daten	nur linear
SVM (Kernel)	Klassifikation	9	hoch	sehr gering	nein (Standard)	nichtlinear, Kernel-Trick	schwer interpretierbar, nur 2 Klassen (nativ)
Entscheidungsbaum	beides	8	mittel	am höchsten	ja	kategorische Features ohne Dummies, grafisch	geringe Genauigkeit
Bagging	beides	8	hoch	gering	ja	Varianz ↓, OOB-Fehler gratis	Bäume korreliert
Random Forest	beides	8	hoch	gering	ja	state of the art , robust ggü. Parametern, parallelisierbar	nicht interpretierbar
Boosting/xgboost	beides	8	sehr hoch	gering	ja	state of the art bei Tabellendaten	⚠ overfittet bei zu großem B , schlechte Defaults, nicht parallelisierbar
Neuronales Netz	beides	10	maximal	am geringsten	ja (softmax)	beliebige Nichtlinearität, skaliert mit Daten	viele Parameter, Rechenaufwand, Blackbox

J.2 ★★ Was tun bei Underfit / Overfit? (Die Handlungstabelle)

Symptom	Ursache	Maßnahmen
UNDERFIT (Train- und Testfehler hoch)	Modell zu einfach , Bias zu hoch , Varianz zu niedrig	• Neue Features hinzufügen (andere Datenquellen) • Polynomterme + Interaktionsterme • Flexibleres Modell (Baum → RF, LinReg → SVM/NN) • kNN: k verkleinern • Boosting: B oder d erhöhen
OVERFIT (Trainingsfehler ↓, Testfehler ↑)	Modell zu komplex , Varianz zu hoch , Bias zu niedrig	• Regularisierung (Lasso, Penalty auf Gewichte) • Feature Selection (korrekt validiert!) • Pruning / Baumtiefe begrenzen • Boosting: λ verringern, B verringern, d verringern • kNN: k vergrößern • Mehr Trainingsdaten • Bagging / RF (Varianz ↓)

J.3 ★ Wo taucht der Bias-Variance-Tradeoff überall auf?

Ort	Kleinerer Wert →	Größerer Wert →
Modellflexibilität allgemein	Bias ↑, Varianz ↓	Bias ↓, Varianz ↑
k bei kNN	flexibler: Bias ↓, Varianz ↑	glatter: Bias ↑, Varianz ↓
Polynomgrad d (LinReg)	Bias ↑, Varianz ↓	Bias ↓, Varianz ↑
C bei Soft-Margin SVM	schmäler Margin: Bias ↓, Varianz ↑	breiter Margin: Bias ↑, Varianz ↓
k bei k-fold CV ⚠	größere Val.-Sets: Bias ↑ (überschätzt), Varianz ↓	größere Train.-Sets: Bias ↓, Varianz ↑
m bei Random Forest	stärkere Dekorrelation: Varianz ↓	mehr Korrelation: Varianz ↑
λ bei Boosting	langsames Lernen: weniger Overfit	schnelleres Lernen: mehr Overfit
d bei Boosting	einfachere Einzelbäume, geringere Interaktionstiefe	komplexere Bäume, mehr Overfit
LDA vs. QDA	LDA: Bias ↑, Varianz ↓	QDA: Bias ↓, Varianz ↑

J.4 ★ Alle „Hyperebenen-Modelle“ im Vergleich

Modell	Entscheidungsgrenze	Wie bestimmt?
Lineare Regression + Schwellwert 0.5	Hyperebene	Least Squares
Logistische Regression	Hyperebene ($p = 0.5$)	Maximum Likelihood (Logit)
LDA	Hyperebene	Normalverteilung mit gleicher Kovarianz
QDA	quadratische Fläche	Normalverteilung mit unterschiedlicher Kovarianz
Hard Margin Classifier	Hyperebene mit maximalem Margin	Optimierung mit harten Nebenbedingungen
Soft Margin Classifier	Hyperebene mit Budget C	Optimierung mit Slack-Variablen
SVM	Hyperebene im hochdimensionalen Raum $\mathcal{H} \rightarrow$ nichtlinear im Originalraum	Kernel-Trick
NN (Output-Neuron)	Hyperebene im Raum der letzten Hidden-Layer-Aktivierungen	Backpropagation

🔑 **Das ist die große Klammer des Moduls:** Fast alle behandelten Klassifikatoren sind im Kern „eine Hyperebene in irgendeinem Raum“.

J.5 ★ Die Kernel-Idee an drei Stellen

Kapitel	Wo?
Kap. 3	Polynomterme + Interaktionsterme erweitern den Featureraum explizit
Kap. 4	Nichtlineare Terme in der log. Regression = „Konstruktion neuer Features aus alten Features und anschließende Erzeugung einer Hyperebene auf allen Features“
Kap. 9	Kernel erweitern den Featureraum implizit (Kernel-Trick), sogar unendlich-dimensional
Kap. 10	Ein Hidden Layer ist eine gelernte, adaptive Feature-Transformation — der Output-Layer ist wieder linear

J.6 ★ Alle Fehlermaße auf einen Blick

Maß	Formel	Typ	Anmerkung
RSS	$\sum (y_i - \hat{y}_i)^2$	Regression	Split-Kriterium bei Regressionsbäumen
TSS	$\sum (y_i - \bar{y})^2$	Regression	Referenz für R^2
MSE	$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$	Regression	= RSS/n
RMSE	$\sqrt{MSE}$	Regression	in Einheiten von y
RSE	$\sqrt{\frac{1}{n-2} RSS}$	Regression	nicht direkt interpretierbar
R^2	$(TSS - RSS)/TSS$	Regression	Anteil erklärter Varianz; kann negativ werden
Fehlklassifizierungsrate	falsche/alle	Klassifikation	⚠ irreführend bei unbalancierten Daten
E (Classification Error Rate)	$1 - \max_k \hat{p}_{mk}$	Klassifikation	zu unsensibel als Split-Kriterium
Gini G	$1 - \sum_k \hat{p}_{mk}^2$	Klassifikation	Reinheitsmaß , Split-Kriterium
Cross-Entropy D	—	Klassifikation	$\approx$ Gini, auch Loss bei NN
Balanced Error/Accuracy	Mittel des Fehlers pro Klasse	Klassifikation	✅ für unbalancierte Daten
F1-Score	harmon. Mittel Precision/Recall	Klassifikation	✅ für unbalancierte Daten
OOB-Fehler	Fehler auf Out-of-Bag-Punkten	Ensembles	„gratis“ bei Bagging/RF

J.7 ★ Die Zitate und Merksätze des Dozenten

Zitat / Merksatz	Kontext
„Essentially, all models are wrong, but some are useful“ (George Box)	Lineare Regression
„Statistik ist eine wichtige Grundlage für ML“	Intro
„Hinweis $\neq$ Beweis“	p-Werte
„Korrelation ist nicht Kausalität“	p-Werte
„Algorithmen allein sind nicht genug“	Prozessmodelle
„Ease-of-use kann zu Fehlern führen“ ☹	Resampling
„Alle Resampling-Methoden funktionieren. Wichtiger ist, überhaupt eine zu verwenden — und sie richtig anzuwenden!“	Resampling
„Wir gewinnen an Vorhersagegenauigkeit, verlieren aber an Interpretierbarkeit“ 😊	Ensembles
„Die Standardparameter [von xgboost] sind sehr, sehr schlecht“	Boosting
„Die KI übernimmt (noch?) nicht das gesamte Denken für uns“	Deep Learning
„Random Forest ist in der Regel sehr unempfindlich gegenüber Parameteränderungen“	Random Forest
„Wir müssen nur K berechnen, nie ϕ “	Kernel-Trick

TEIL K — SELBSTTESTFRAGEN (150 Fragen mit Kurzantworten)

Nutzung: Antwort abdecken, Frage beantworten, dann vergleichen. Fragen mit ★ sind besonders klausurnah.

K.1 Grundlagen (Kap. 2) — Fragen 1–30

#	Frage	Antwort
1	Synonym für Machine Learning?	Statistical Learning
2 ★	5 Synonyme für „Label“?	Abhängige Variable, Output, Antwort, Zielvariable, Y
3 ★	5 Synonyme für „Feature“?	Prädiktor, Input, unabhängige Variable, Kovariate, Merkmal
4	Was steht für die Anzahl der Features? Für die Beobachtungen?	p bzw. n / N
5 ★	Regression vs. Klassifikation?	Y quantitativ vs. Y qualitativ/kategorisch
6	Drei Ziele des Supervised Learning?	Vorhersage, Zusammenhang verstehen, Unsicherheit bestimmen
7	Was fehlt beim Unsupervised Learning?	Das Label Y
8	5 Ziele des Unsupervised Learning?	Gruppen erkennen, Korrelationen/Muster, Dimensionsreduktion, Normalverhalten lernen, Vorverarbeitung
9	Warum ist Unsupervised Learning schwerer zu bewerten?	Kein Abgleich mit Y möglich
10	Grundmodell des Supervised Learning?	$Y = f(X) + \epsilon$
11 ★	Was ist die ideale Regressionsfunktion?	$f(x) = E(Y \mid X = x)$ — der bedingte Erwartungswert
12	Welche Eigenschaft hat sie?	Minimaler quadratischer Fehler
13 ★	Warum ist ϵ nicht Null?	Y ist eine Verteilung bei gegebenem X (Gehaltsbeispiel)
14	Wann kann $\epsilon = 0$ sein?	Bei einem deterministischen System, z. B. diskrete Computersimulation
15 ★★	Bias-Variance-Zerlegung?	Error = Varianz + Bias + ϵ
16 ★	Ursache von Bias?	Die Modellstruktur
17 ★	Ursache von Varianz?	Kleine Änderungen in den Trainingsdaten
18 ★★	Flexibilität $\uparrow \rightarrow$ was passiert?	Varianz $\uparrow$, Bias $\downarrow$, ϵ konstant
19 ★	Underfit: Bias/Varianz?	hoher Bias, geringe Varianz
20 ★	Overfit: Bias/Varianz?	geringer Bias, hohe Varianz
21 ★★	Warum ist „Trainingsfehler < Testfehler“ kein Overfit-Beweis?	Nur schwacher Indikator; kann an Ausreißern, Skalen, Stichprobengrößen liegen
22 ★	Was ist der bessere Overfit-Indikator?	Trainingsfehler sinkt, während Testfehler steigt (bei steigender Komplexität)
23	Welche Bedingung müssen Test-/Trainingsdaten erfüllen?	Einigermaßen unkorreliert, keine Redundanz
24 ★	Was ist der Curse of Dimensionality?	In hohen Dimensionen ist selbst der nächste Nachbar tendenziell weit entfernt
25	Wann funktioniert kNN gut?	Kleines p (einstellig), großes N (Hunderte)
26 ★	kNN-Aggregation bei Regression? Bei Klassifikation?	Mittelwert bzw. Modus (häufigste Klasse)
27 ★	Bayes-optimaler Klassifikator?	Klasse mit größtem $p_k(x)$; hat die minimale Fehlklassifizierungsrate
28	Formel Fehlklassifizierungsrate?	falsche Vorhersagen / alle Vorhersagen
29 ★	6 Phasen von CRISP-DM?	Business Understanding, Datenverstehen, Datenaufbereitung, Modellierung, Auswertung, Einsatz

#	Frage	Antwort
30	Welche Phase ist der „Kern des Prozesses“?	Modellierung

K.2 Lineare Regression (Kap. 3) — Fragen 31–60

#	Frage	Antwort
31	Zitat von George Box?	„All models are wrong, but some are useful“
32	Was ist ein Residuum?	$e_i = y_i - \hat{y}_i$
33 ★	Was minimiert Least Squares?	$RSS = \sum e_i^2$
34 ★	Warum ist LS besonders praktisch?	RSS kann analytisch optimiert werden
35	Alternative zu LS? Ergebnis?	MLE — bei klassischer LinReg dasselbe Ergebnis
36 ★	Normalengleichung?	$\vec{\beta} = (X^T X)^{-1} X^T \vec{y}$ bzw. LGS $(X^T X) \vec{\beta} = X^T \vec{y}$
37	Was enthält die Designmatrix zusätzlich?	Eine Spalte aus Einsen (für β_0)
38 ★	95 %-Konfidenzintervall?	$[\hat{\beta} - 2SE(\hat{\beta}), \hat{\beta} + 2SE(\hat{\beta})]$
39	Welches Intervall ist breiter: 80 % oder 90 %?	90 %
40 ★	Nullhypothese eines Koeffizienten?	$H_0 : \beta_1 = 0$ (kein Zusammenhang)
41 ★★	p-Wert klein → ?	H_0 ablehnbar, Hinweis auf Zusammenhang
42 ★★	p-Wert groß → ?	H_0 nicht ablehnbar; kein Hinweis dafür UND kein Hinweis dagegen
43 ★★	Ist der p-Wert die Wahrscheinlichkeit, dass H_0 wahr ist?	NEIN
44 ★	Was sagt der p-Wert über Kausalität?	Nichts — „Korrelation ist nicht Kausalität“
45	KI enthält 0 → was folgt?	H_0 ist nicht ablehnbar
46 ★	R^2 -Formel?	$(TSS - RSS)/TSS = 1 - RSS/TSS$
47 ★	Wann wird R^2 negativ?	Wenn das Modell schlechter ist als die Mittelwertvorhersage
48	RSE-Formel? Interpretierbar?	$\sqrt{RSS/(n-2)}$ — nicht direkt, braucht Vergleich
49 ★★	Interpretation von β_j ?	Durchschn. Auswirkung auf Y pro Einheitsanstieg in X_j , wenn alle anderen Features gleich bleiben
50 ★	Was ist Kollinearität? 3 Probleme?	Korrelation von Features; Varianz der Koeffizienten $\uparrow$, numerische Instabilität, Interpretation schwierig
51 ★	Mögliche Lösung für Kollinearität? Einschränkung?	PCR — Interpretation bleibt schwierig
52 ★	Münzbeispiel: Vorzeichen von β_1 mit/ohne X_2 ?	ohne: positiv; mit (Gesamtzahl Münzen): negativ
53	Wie viele Modelle bei Brute-Force-Feature-Selection?	2^p
54 ★	Stepwise Forward Selection in 4 Schritten?	Leeres Modell → je ein Feature hinzufügen → bestes wählen → wiederholen bis Abbruch
55	4 Auswahlkriterien?	Mallow's C_p , AIC, BIC, adjusted R^2
56 ★	Wie viele Dummies bei k Ausprägungen?	$k - 1$
57 ★	Was ist die Baseline?	Die Ausprägung ohne eigene Dummy-Variable; alle Koeffizienten sind Differenzen dazu

#	Frage	Antwort
58 ★	Warum ist One-Hot bei LinReg problematisch?	Lineare Abhängigkeit zwischen Intercept und Summe der One-Hot-Spalten → ein Koeffizient nicht schätzbar
59 ★★	Hierarchieprinzip?	Bei Interaktionen immer auch die Haupteffekte einbeziehen — sonst nicht interpretierbar
60 ★	Interaktion vs. Kollinearität?	Verschiedene Konzepte: Interaktion = Effekt hängt vom anderen Feature ab; Kollinearität = Features korrelieren

K.3 Klassifikation (Kap. 4) — Fragen 61–90

#	Frage	Antwort
61 ★	Definition Hyperebene?	Affiner Unterraum der Dimension $d - 1$ in $\mathbb{R}^d$
62	Hyperebene in $\mathbb{R}^1/\mathbb{R}^2/\mathbb{R}^3$?	Punkt / Gerade / Ebene
63	Was ist $\vec{n}$?	Normalenvektor, orthogonal zur Hyperebene
64	Wann geht die Hyperebene durch den Ursprung?	Wenn $n_0 = 0$
65 ★	Abstandsformel Punkt–Ebene?	$d = \vec{p}^T \vec{n} + n_0 / \vec{n} $
66 ★	Was sagt das Vorzeichen von $f(\vec{p})$?	Auf welcher Seite der Hyperebene der Punkt liegt
67 ★	2 Probleme der LinReg für Klassifikation?	$\hat{f}$ ist keine Wahrscheinlichkeit; bei > 2 Klassen impliziert die Kodierung eine Ordnung
68	Wie kann man LinReg dennoch für > 2 Klassen nutzen?	One-vs-One oder One-vs-All
69 ★★	Logistische Funktion?	$p(x) = s/(1 + s)$ mit $s = e^a$, $a = \beta_0 + \beta_1 X_1 + \dots$
70 ★★	Log-Odds/Logit?	$\ln(p/(1 - p)) = \beta_0 + \beta_1 X_1 + \dots$
71 ★	Warum liegt p immer in $[0, 1]$?	$e^a \in (0, \infty)$; $s = 0 \Rightarrow p = 0$, $s = \infty \Rightarrow p = 1$
72 ★★	Wo liegt die Entscheidungsgrenze?	Bei $\hat{p}(x) = 0.5$, d. h. $\beta_0 + \beta_1 X_1 + \dots = 0$
73 ★	Steigung & Achsenabschnitt der Entscheidungsgrenze in 2D?	Steigung $-\beta_1/\beta_2$, Achsenabschnitt $-\beta_0/\beta_2$
74 ★	Welche Form nutzt man für „welches X ergibt $p = q$ “?	Die Logit-Form
75 ★★	Was ist ein Confounder? Beispiel?	Unbeobachtete Störgröße; <code>student</code> hängt mit <code>balance</code> UND <code>default</code> zusammen
76	Warum wechselt der student-Koeffizient das Vorzeichen?	Bei gleicher <code>balance</code> haben Studenten seltener Default
77	Verhältnis Controls:Cases maximal sinnvoll?	5:1
78 ★	Case-Control-Korrekturformel?	$\hat{\beta}_0^* = \hat{\beta}_0 + \ln \frac{a}{1-a} - \ln \frac{\bar{a}}{1-\bar{a}}$
79 ★★	Warum divergieren Koeffizienten bei exakt trennbaren Daten?	Dieselbe Hyperebene erlaubt beliebig skalierte Koeffizienten $\rightarrow p$ nähert sich 0/1 $\rightarrow$ Optimum bei ∞
80 ★	2 Lösungen dafür?	Regularisierung; alternative Modelle (LDA)
81 ★	Prinzip der Diskriminanzanalyse?	Verteilung je Klasse bestimmen, Dichte berechnen, Klasse mit höchster Dichte vorhersagen
82 ★	LDA vs. QDA?	Gleiche vs. unterschiedliche Kovarianz je Klasse
83 ★	Naive-Bayes-Annahme?	Features sind unabhängig (Dichte = Produkt der Einzeldichten)
84 ★	4 Gründe für LDA?	Kein Trennbarkeitsproblem; stabiler bei kleinem n ; beliebt für $K > 2$; Visualisierung
85 ★★	Lineare Bayes-Grenze: LDA oder QDA besser?	Auf Trainingsdaten evtl. QDA, auf Testdaten LDA (QDA overfittet)
86 ★	Wann Naive Bayes?	Sehr großes p oder gemischte Featuretypen
87 ★★	Problem unbalancierter Daten?	Accuracy/Fehlerrate irreführend — „immer Mehrheitsklasse“ wirkt gut, ist aber nutzlos

#	Frage	Antwort
88 ★	2 bessere Maße?	F1-Score, Balanced Error/Accuracy
89 ★	3 Lösungsansätze für unbalancierte Daten?	Bessere Maße, Over-/Under-Sampling, neue Daten der Minderheitsklasse
90	Warum funktioniert <code>glm</code> bei Iris nicht?	Nur binär + zwei Klassen sind exakt trennbar → besser <code>glmnet</code>

K.4 SVM (Kap. 9) — Fragen 91–110

#	Frage	Antwort
91 ★	Kriterium für eine trennende Hyperebene?	$y_i \cdot f(\vec{x}_i) > 0 \forall i$ ODER $< 0 \forall i$ (mit $y_i = \pm 1$)
92 ★	Was ist der Margin?	Der minimale Abstand zu den Datenpunkten
93 ★	Hard Margin Classifier?	Die trennende Hyperebene mit dem größten Margin
94 ★	Supportvektoren beim Hard Margin?	Punkte exakt auf den Margin-Grenzen
95 ★	Supportvektoren beim Soft Margin?	Alle Punkte, die den Margin verletzen (auf der Grenze, innerhalb, oder falsche Seite)
96 ★	3 Aussagen zur Rolle der Supportvektoren?	Nur sie stützen die Ebene; ihre Änderung ändert die Ebene; Änderung anderer Punkte nicht
97	Warum muss $\vec{n}$ normiert sein?	Sonst steht links keine Distanz
98 ★	Was ist C ?	Budget für die Summe der Slack-Variablen ϵ_i
99	Wann existiert keine trennende Hyperebene?	Häufig bei realen Daten, insbesondere wenn $m \geq p$
100 ★	2 Probleme der polynomiellen Feature-Erweiterung?	Numerisch problematisch; unklar welcher Grad nötig
101 ★	Alternative Erweiterungs-idee?	Distanzen zu (allen) Trainingspunkten
102 ★★	Radial-Kernel?	$K(\vec{x}, \vec{x}') = e^{-\gamma \sum_j (x_j - x'_j)^2}$
103 ★	2 Eigenschaften einer Kernelfunktion?	Symmetrie, Definitheit
104 ★★	Kernel-Trick in einem Satz?	K berechnet ein inneres Produkt implizit im hochdimensionalen Raum $\mathcal{H}$ — man muss ϕ nie berechnen
105 ★	Wohin bildet ϕ_{radial} ab?	In einen unendlich-dimensionalen Raum
106 ★★	SVM-Vorhersageformel?	$f(\vec{x}) = n_0 + \sum_i \alpha_i K(\vec{x}_i, \vec{x})$
107 ★	Bedeutung von $\alpha_i = 0$?	Der Punkt ist kein Supportvektor und wird für die Vorhersage nicht benötigt
108	Klassenzuordnung?	$\hat{c}(\vec{x}) = \text{sign}(f(\vec{x}))$
109 ★	OVA vs. OVO — Modellanzahl?	OVA: K ; OVO: $K(K-1)/2$
110 ★	Wann OVO, wann OVA?	OVO bei kleinem K ; OVA sonst (spart Rechenaufwand)

K.5 Resampling (Kap. 5) — Fragen 111–130

#	Frage	Antwort
111 ★	Zweck Trainingsfehler?	Fortschritt des Trainingsprozesses messen
112 ★	Zweck Testfehler?	Qualität für neue, ungesehene Daten schätzen
113 ★★	Die 2 Probleme des Train/Test-Splits?	Keine Entscheidungen auf Testdaten treffen; nur eine einzige Fehlerbeobachtung (keine Varianz)
114 ★	3 Beispiele für „Entscheidungen“?	Modellauswahl, Feature Selection, Hyperparameter
115 ★★	Zweck der 3 Datenmengen?	Training: Modell anpassen · Validierung: Entscheidungen + Vorabschätzung · Test: finales Modell bewerten
116 ★	Was ist das Dev-Set?	Alles außer Testdaten = Training + Validierung
117 ★	3 Nachteile von Hold-Out?	Starke Varianz; kleinere Trainingsmenge; Überschätzung des Testfehlers
118 ★★	k-fold CV in 3 Schritten?	Dev-Set in k Folds; je Fold auf Rest trainieren + auf Fold testen; Fehler aggregieren
119	Was ist ein Fold?	Eine der k nicht-überlappenden Teilmengen
120 ★	Was ist LOOCV?	$k = n$ — ein Punkt pro Fold
121 ★	2 Nachteile von LOOCV?	Kostspielig (n Modelle); Trainingsdaten zu ähnlich → korrelierte Fehler → hohe Varianz
122 ★★	$k \uparrow \rightarrow$ Bias und Varianz?	Bias $\downarrow$ (größere Trainingsmengen), Varianz $\uparrow$ (kleinere Validierungsmengen, korrelierte Trainingssets)
123 ★	Empfohlener k -Bereich? Ausnahmen?	5–10; höher bei sehr vielen Daten + einfachem Modell; evtl. sogar 5 zu groß bei $n < 100$ + komplexem Modell
124 ★	Bootstrap in einem Satz?	Ziehen mit Zurücklegen , bis der Datensatz die Größe des Dev-Sets hat
125 ★	Namensherkunft?	Baron Münchhausen zieht sich am eigenen Haar aus dem Sumpf — Lösung aus sich selbst heraus
126 ★	2 Unterschiede CV ↔ Bootstrap?	Punkt genau einmal vs. mehrfach zur Fehlerbestimmung; Trainingsset kleiner vs. gleich groß (mit Duplikaten)
127 ★★	Was war der Fehler im 5000-Features-Experiment?	Feature Selection vor der CV → das Modell sieht bereits die Labels der Validierungsdaten
128 ★★	Die richtige Vorgehensweise?	CV gleichzeitig für Feature Selection UND Training — Features nur auf Trainingsdaten wählen
129 ★	Wer über/unterschätzt den Testfehler?	Hold-Out: überschätzt (stark) · CV: unterschätzt · Bootstrap: überschätzt (moderat)
130 ★	Womit trainiert man das finale Modell?	Mit dem gesamten Dev-Set — keine Daten an die Validierung verlieren

K.6 Bäume & Ensembles (Kap. 8) — Fragen 131–150

#	Frage	Antwort
131 ★	Wie sagt ein Regressionsbaum vorher?	Mittelwert der Trainings-Labels in der Region R_j
132 ★	Wie ein Klassifikationsbaum?	Häufigste Klasse (Modus) im Blatt
133 ★	Was ist recursive binary splitting?	Top-down, greedy: je Schritt der Split, der den RSS in diesem Schritt am stärksten reduziert
134	Typisches Abbruchkriterium?	z. B. nur noch 5 Trainingspunkte pro Blatt
135 ★	Was ist Pruning?	Vollen Baum bauen, dann Zweige abschneiden für guten Komplexität/Fehler-Kompromiss
136 ★★	Gini-Index? Was misst er?	$G = 1 - \sum_k \hat{p}_{mk}^2$ — die Reinheit eines Knotens
137 ★	Wie wird G für einen Split bewertet?	G für beide neuen Knoten, dann nach Beobachtungszahl gewichtetes Mittel
138 ★★	Warum ist E als Split-Kriterium schlecht?	Nicht sensitiv genug für kleine Änderungen des Split-Punkts
139 ★	4 Vorteile von Bäumen?	Leicht erklärbar (leichter als LinReg), grafisch darstellbar, kategorische Features ohne Dummies , spiegeln menschliche Entscheidungsfindung
140 ★	4 Ensemble-Methoden?	Bagging, Random Forest, Boosting, Stacking
141 ★★	Bagging-Formel?	$\hat{f}_{bag}(x) = \frac{1}{B} \sum_i \hat{f}^{*i}(x)$
142 ★	Bagging bei Klassifikation?	Mehrheitsabstimmung ODER Mittel der Klassenhäufigkeiten in den Blättern
143 ★★	Was ist OOB? Wie groß ist der Anteil?	Nicht im Bootstrap-Sample enthaltene Punkte; $\sim 1/3$ (je Baum $\sim 2/3$ Training)
144 ★★	Was macht Random Forest anders als Bagging?	Vor jedem einzelnen Split zufällige Auswahl von m aus p Features
145 ★★	Wozu dient das?	Dekorrelation der Bäume $\rightarrow$ geringere Varianz $\rightarrow$ bessere Genauigkeit
146 ★	Typische Wahl von m ?	$m \approx \sqrt{p}$
147 ★	Wie misst man Feature-Wichtigkeit?	Summierte Fehlerabnahme durch Splits über dieses Feature, gemittelt über alle B Bäume
148 ★★	Boosting-Algorithmus in 3 Schritten?	$\hat{f} = 0, r_i = y_i \rightarrow$ je Runde: Baum auf (X, r) , $\hat{f} += \lambda \hat{f}^b, r_i -= \lambda \hat{f}^b(x_i) \rightarrow$ Summe ausgeben
149 ★★	Was macht λ ? Was passiert bei zu großem B ?	Verlangsamt das Lernen; Boosting overfittet bei zu großem B (anders als Bagging/RF)
150 ★	Was steuert d außer der Komplexität?	Die Interaktionstiefe — d Splits beinhalten höchstens d Variablen

TEIL L — KLAUSUR-SPICKZETTEL

⚠ Die Formelsammlung wird gestellt! Dieser Spickzettel enthält daher vor allem das, was Sie **zusätzlich** im Kopf haben müssen: Interpretationen, Definitionen, Algorithmen und Rechenwege.

L.1 Die 10 Sätze, die immer passen

1. „Regression = quantitatives Y , Klassifikation = qualitatives Y .“
2. „ β_j ist die durchschnittliche Auswirkung auf Y eines Einheitsanstiegs in X_j , wenn alle anderen Features gleich bleiben.“
3. „Ein kleiner p-Wert ist ein Hinweis auf einen Zusammenhang — Hinweis $\neq$ Beweis, und Korrelation ist nicht Kausalität.“
4. „Ein großer p-Wert ist kein Hinweis dafür UND kein Hinweis dagegen.“
5. „Flexibilität $\uparrow \rightarrow$ Varianz $\uparrow$, Bias $\downarrow$, nicht-reduzierbarer Fehler konstant.“
6. „Overfit liegt vor, wenn bei steigender Komplexität der Trainingsfehler sinkt und der Validierungsfehler steigt.“
7. „Bei k Ausprägungen braucht man $k - 1$ Dummy-Variablen; die Ausprägung ohne Dummy ist die Baseline.“
8. „Die Entscheidungsgrenze der logistischen Regression liegt bei $p = 0.5$, also bei $\beta_0 + \beta_1 X_1 + \dots = 0$ — eine Hyperebene.“
9. „Random Forest dekorreliert die Bäume; Dekorrelation senkt die Varianz und damit den Fehler.“
10. „Bei unbalancierten Daten sind Accuracy und Fehlerrate irreführend — nehmen Sie F1 oder Balanced Accuracy.“

L.2 Rechenwege in je 3 Zeilen

Logistische Regression, p berechnen:

1. $a = \beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \dots$ (alle Werte einsetzen)
2. $s = e^a$ (Taschenrechner)
3. $p = s / (1 + s)$ (auf verlangte Stellenzahl runden)

Logistische Regression, „welches X ergibt $p = q$?“:

1. Logit-Form nutzen: $\ln(q/(1-q)) = \beta_0 + \beta_1 \cdot X_1 + \dots$
2. Bei $q=0.5$: $\ln(1) = 0 \rightarrow$ Gleichung = 0
3. Nach X auflösen. ($q=1 \rightarrow$ nicht definiert, nur als Grenzwert!)

Entscheidungsgrenze in 2D zeichnen:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0 \rightarrow x_2 = -(\beta_1/\beta_2) \cdot x_1 - (\beta_0/\beta_2)$$

Steigung = $-\beta_1/\beta_2$, Y-Achsenabschnitt = $-\beta_0/\beta_2$

Interaktionsmodell, „bei wem stärker?“:

1. $\partial y / \partial x_1$ bilden (Interaktionsterm liefert $+\beta_3 \cdot x_2$)
2. Dummy-Werte 0 und 1 einsetzen
3. In Worten interpretieren (Vorzeichen beachten: negativ = ABNAHME!)

Interaktionsmodell, „wann gleich?“:

1. Beide Geraden separat aufstellen ($x_2=0$ und $x_2=1$ einsetzen, zusammenfassen)
2. Gleichsetzen
3. Nach x_1 auflösen

SVM-Vorhersage:

1. Nur die i mit $\alpha_i \neq 0$ betrachten!
2. $K(x, x_i)$ für diese berechnen
3. $f(x) = n_0 + \sum \alpha_i \cdot K(x, x_i)$; Klasse = $\text{sign}(f(x))$

Neuronales Netz, Forward Pass:

Je Neuron: $z = \sum w_j \cdot x_j + b \rightarrow a = g(z)$
 Schicht für Schicht durchgehen, Ergebnisse der Schicht als Eingabe der nächsten.
 Spät runden!

Gini-Index eines Splits:

1. G je Knoten: $G = 1 - \sum p_k^2$
2. Gewichten mit $(n_{\text{Knoten}} / n_{\text{gesamt}})$
3. Beide addieren $\rightarrow$ gewichteter Gini des Splits (kleiner = besser)

Boosting-Ensemble-Vorhersage:

$f(x) = \sum_b \lambda \cdot f^b(x)$ (jeden Baum ablaufen, mit λ multiplizieren, summieren)

Baum ablaufen (Aufgabe 7-Stil):

Für jeden Knoten explizit notieren:
 <Feature> ist (NICHT) $\geq$ <Schwelle> $\rightarrow$ (links/rechts)
 Am Ende: Blatt: Vorhersage <Wert>

 R^2 und RSS von Hand:

$RSS = \sum (y_i - \hat{y}_i)^2$ $TSS = \sum (y_i - \bar{y})^2$ $R^2 = (TSS - RSS) / TSS$
 (R^2 kann negativ werden!)

kNN von Hand:

1. Euklidische Distanz zu JEDEM Punkt: $d = \sqrt{\sum (x_i - x_i')^2}$
2. Die k kleinsten auswählen
3. Klassifikation: Modus | Regression: Mittelwert

L.3 Zahlen, die man kennen sollte

Wert	Bedeutung
0.05	üblicher p-Wert-Schwellenwert
$\pm 2 \cdot SE$	95 %-Konfidenzintervall
2^p	Anzahl Modelle bei Brute-Force-Feature-Selection ($p = 40 \rightarrow$ über 1 Billion)
$k - 1$	Anzahl Dummy-Variablen bei k Ausprägungen
0.1^p	Anteil der Daten in einer 10 %-Nachbarschaft je Dimension (Curse of Dimensionality)
$k = 5$ bis 10	empfohlener CV-Bereich
2/3 : 1/3	In-Bag : Out-of-Bag beim Bootstrap
$m \approx \sqrt{p}$	Feature-Anzahl je Split bei Random Forest
$\lambda = 0.01$ / 0.001	typische Boosting-Lernraten (xgboost-Default 0.3 ist zu groß!)
$d \leq 10$	typische Boosting-Baumtiefe
5:1	max. sinnvolles Controls:Cases-Verhältnis
$B = \text{Hunderte-Tausende}$	typische Baumanzahl im Random Forest

L.4 Die Kapitel-Kurzformeln (ohne Formelsammlung merken)

$$\begin{aligned} \text{Error} &= \text{Varianz} + \text{Bias} + \epsilon & f(x) &= E(Y \mid X = x) \\ RSE &= \sqrt{\frac{1}{n-2} RSS} & \hat{\beta}_1 &= \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \\ \hat{\beta}_0^* &= \hat{\beta}_0 + \ln \frac{a}{1-a} - \ln \frac{\bar{a}}{1-\bar{a}} & E &= 1 - \max_k \hat{p}_{mk} \\ \hat{f}_{bag}(x) &= \frac{1}{B} \sum_{i=1}^B \hat{f}^{*i}(x) & \hat{f}_{boost}(x) &= \sum_{b=1}^B \lambda \hat{f}^b(x) \\ z(\vec{x}) &= \vec{w}^T \vec{x} + b, & f(\vec{x}) &= g(z(\vec{x})) \end{aligned}$$

L.5 ⚠ Die 12 Fallen (Checkliste vor der Abgabe)

- ❌ „p-Wert = Wahrscheinlichkeit, dass H_0 wahr ist" → **falsch**
- ❌ „Großer p-Wert beweist, dass kein Zusammenhang besteht" → **falsch** (kein Hinweis dafür UND dagegen)
- ❌ „Das Modell zeigt, dass X Y verursacht" → **Kausalität nie aus dem Modell**
- ❌ „Trainingsfehler < Testfehler $\Rightarrow$ Overfit" → **nur schwacher Indikator**
- ❌ „100 % Wahrscheinlichkeit erreichbar" → **nur als Grenzwert, praktisch unmöglich**
- ❌ Zielvariable als Feature mitzählen → **p ohne Y!**
- ❌ k Dummies statt $k - 1$ → **One-Hot ist bei LinReg problematisch**
- ❌ Interaktion ohne Haupteffekte → **Hierarchieprinzip verletzt**
- ❌ Feature Selection vor der CV → **Selection Bias, 97 % auf reinem Rauschen**
- ❌ Bei unbalancierten Daten Accuracy melden → **Balanced Accuracy / F1**
- ❌ Bei Boosting-Overfit λ **erhöhen** → **verringern!**
- ❌ Jede Featureraum-Aufteilung lässt sich als Baum darstellen → **falsch** (nur rekursiv-achsenparallele)

L.6 Die 7 Kapitel in je einem Satz

Kap.	Kernaussage
2 — Grundlagen	Alles Lernen ist ein Bias-Variance-Tradeoff ; die ideale Regressionsfunktion ist der bedingte Erwartungswert, den wir wegen fehlender Daten nur schätzen können.
3 — Lineare Regression	Least Squares ist analytisch lösbar ; die eigentliche Kunst liegt in Interpretation (p-Werte, Kollinearität, Confounder) und Erweiterungen (Dummies, Interaktionen, Polynome).
4 — Klassifikation	Die logistische Regression liefert echte Wahrscheinlichkeiten über eine Hyperebene ; LDA/QDA/Naive Bayes sind die Alternativen für kleine n , viele Klassen und großes p .
9 — SVM	Der Kernel-Trick erlaubt eine Hyperebene in einem (unendlich-dimensionalen) Raum, ohne diesen je zu berechnen .
5 — Resampling	Drei Datenmengen, nicht zwei; und die ganze Modellentwicklung (inkl. Feature Selection) muss innerhalb der Validierung passieren.
8 — Bäume/Ensembles	Einzelbäume sind interpretierbar, aber ungenau; Bagging senkt Varianz , RF dekorreliert , Boosting lernt langsam aus Residuen — Genauigkeit gegen Interpretierbarkeit.
10 — Neuronale Netze	Ein Neuron ist eine lineare Regression + Nichtlinearität (mit Sigmoid: logistische Regression); Netze verketteten das und werden über Backpropagation + Gradient Descent trainiert.

L.7 Prüfungsstrategie

1. **Zuerst die Formelsammlung überfliegen** — sie verrät oft, welche Rechnung gemeint ist (z. B. Radial-Kernel gegeben $\Rightarrow$ SVM-Aufgabe).
2. **Bei „Begründen Sie“ immer zwei Sätze:** Was + Warum. Nur die Zahl gibt selten volle Punkte.
3. **Bei Rechenaufgaben jeden Zwischenschritt aufschreiben** — Aufgabe 7 verlangt explizit „Dokumentieren Sie Ihr Vorgehen“.
4. **Bei „Zeichnen Sie“:** Achsen beschriften, Werte annotieren, Blätter/Regionen benennen.
5. **Bei Interpretationsfragen:** Vorzeichen prüfen! Ein negativer Koeffizient bedeutet **Abnahme** — auch wenn die Frage „steigt stärker“ lautet.
6. **Fangfragen erkennen:** „100 %?“, „Ist das möglich?“, „richtig oder falsch?“ $\rightarrow$ meist ist die intuitive Antwort falsch.
7. **Spät runden**, sonst weichen die Ergebnisse ab (Warnung des Dozenten in den NN-Folien).
8. **Nicht-programmierbarer Taschenrechner** — e^x , $\ln$, $\sqrt{}$ vorher üben!

Ende Modul 15 — Supervised Learning (KI und ML), Martin Zaefferer, DHBW Ravensburg WDS224.